前言

一次意外的解剖机会

2026 年 4 月，Anthropic 官方发布的 `@anthropic-ai/claude-code` npm 包中意外留存了一个 60MB 的未压缩调试文件。1902 个源文件、约 51 万行 TypeScript 代码全部暴露——Claude Code v2.1.88 的完整核心逻辑、系统提示词、工具定义一览无余。事件持续约 24 小时后被发现并下架，但代码已在 GitHub 社区广泛传播。

这不是一次普通的安全事故。对于一个以 AI 安全为使命的公司，因工程失误泄露自己的核心代码，讽刺之余也揭示了一个事实：即便是 **Anthropic**，其 **AI** 编程工具的竞争力也不在**"魔法 AI"**，而在工程架构。

源码显示，Claude Code 的成功密码是 60% 模型能力 + 40% 精密工程。那 60% 来自 Claude 模型本身，任何人都可以通过 API 调用获得。而那 40%——提示词的分层缓存、5 种上下文压缩策略、对抗性验证智能体、Feature Flag 驱动的自我精简——恰恰是所有人都可以学习、复制、迁移的部分。

这本书就是对那 40% 的完整解剖。

什么是 **Harness** 工程

裸用 LLM API，就像把一匹千里马放进没有围栏的草原——它可能往任意方向跑。Harness（马具）是那套缰绳和马鞍，决定了马能稳定朝目标方向跑多远。

在 AI 工程语境里，Harness 指的是围绕 LLM/Agent 搭建的运行框架：提示词结构、工具配置、上下文管理策略、多智能体协作方式、反馈循环设计。Harness 工程就是研究和实践这套框架设计的学问。

过去一年，Anthropic 和 OpenAI 都发表了深度实践文章。Anthropic 用三智能体架构（规划器 + 生成器 + 评估器）在 6 小时内构建出复古游戏制作器；OpenAI 用 Codex 5 个月写出 100 万行代码。他们路径不同，但核心规律一致：一个伟大的模型配一个糟糕的 **Harness** 会给出糟糕的体验。一个好的模型配一个伟大的 **Harness** 会给出极致的体验。

从源码提炼模式的方法

这本书不逐行解读代码，不罗列功能。它从 51 万行源码中识别 Harness 的核心子系统，逐一回答七个问题：它解决什么问题、核心数据结构是什么、控制流怎样、并发模型如何、状态存储在哪、错误怎么处理、有什么精巧设计。

每个子系统的分析最终收束为一条可迁移的工程原则，格式为「原则 **N.N**：**[**原则名**] — [**一句话阐述**]**」。全书 16 章共提炼了数十条这样的原则，最终在第 16 章综合为 7 条核心原则。

如何阅读本书

结构导览。 全书按架构分层自上而下组织，8 篇 16 章：

篇	章节	主题
第一篇：全景	第 1 章	全局架构分层和 Harness 控制层的完整边界
第二篇：启动与循环	第 2-4 章	启动初始化、QueryEngine 主循环、单轮执行流水线
第三篇：上下文工程	第 5-6 章	系统提示词分层缓存、Skills 懒加载
第四篇：工具编排	第 7-8 章	buildTool 工厂、Plan Mode 先规划后执行
第五篇：护栏与扩展	第 9-10 章	权限渐进谱系、Hooks 事件系统
第六篇：记忆管理	第 11-12 章	5 种压缩策略、forkedAgent 上下文隔离与记忆架构
第七篇：多智能体	第 13-14 章	GAN 式生成/验证分离、Coordinator 并行协调
第八篇：演进	第 15-16 章	Feature Flag 自我精简、7 条通用原则

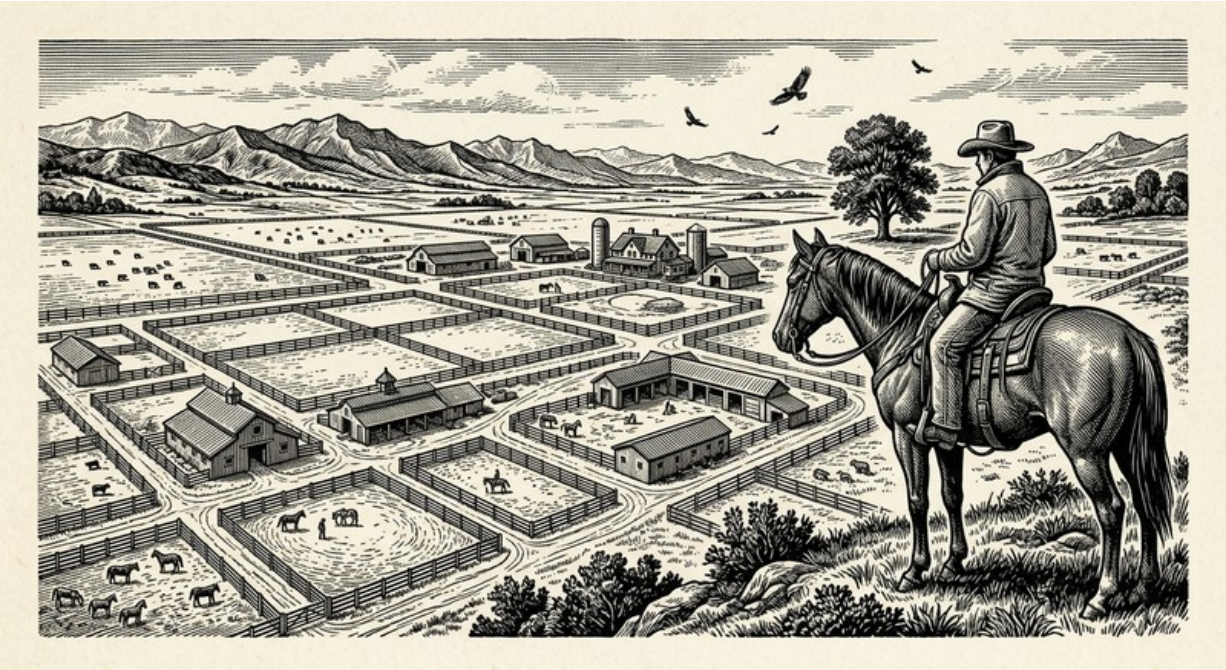
阅读路径。 各篇相对独立，但有前置依赖关系——每章开头标注了前置章节。如果你只想理解某一个问题（如"如何设计上下文压缩"），可以直接跳到对应章节，按前置依赖回溯即可。

术语约定。 全书使用的核心术语及其在 Claude Code 语境下的精确定义，见附录 A 术语表。首次出现的技术术语标注英文原文，如"死代码消除（DCE, Dead Code Elimination）"。

推断标注。 无源码直接依据的结论标注「（推断）」。有源码锚点的结论不标注——锚点本身就是证据。

这本书的最高目标不是教你怎么复制 Claude Code 的实现，而是让你在任何 LLM 应用中回答同一个问题：什么应该由模型管理，什么必须由 **Harness** 控制？ 89 个 Feature Flag 中的每一个都在等待模型变得足够好的那一天。你的 Harness 也应该如此。

第一篇：全景



从 **51** 万行 **TypeScript** 源码中提炼出 **Claude Code** 的全局架构。这一篇建立鸟瞰视角，让你看到 **Harness** 控制层的完整边界——从启动入口到核心循环，从提示词系统到工具编排，所有子系统如何协同工作。

第 1 章:当模型遇见 Harness

> "模型是引擎，Harness 是车身。引擎决定动力，车身决定你能开到哪里。"

51 万行 TypeScript 代码、2000 个源文件、54 个工具注册——这些数字属于 Claude Code v2.1.88，Anthropic 的 AI 编程助手 CLI。但调用 Claude API 完成一次对话，只需要不到 10 行代码。多出来的 51 万行在做什么？它们构成了一个精密的 **Harness**（马具）控制层——围绕 LLM 搭建的运行框架，决定模型能稳定、安全、高效地完成多复杂的任务。读完本章，你将看到这张 Harness 架构的全景地图，并理解后续每一章在全局架构中的位置。

问题——51 万行代码在解决什么问题

裸调 Claude API 只需要几行代码：发送一段文本，收到一段回复。但当你想让 AI 编程助手改完一个文件后记住改动内容、在执行危险命令前先征求许可、在上下文窗口快要爆满时自动压缩历史消息、在主任务之外并行派发子任务——你需要的就不再是一个 API 封装，而是一整套控制架构。

Claude Code 的代码不是功能堆砌，而是围绕一个单一目标构建的精密系统：让模型稳定、安全、高效地完成任务。源码的第一行就揭示了这种工程意识的渗透深度。

启动序列的第一条语句不是解析参数，不是渲染界面，而是一个性能打点。紧随其后的三行代码分别触发 MDM（移动设备管理，Mobile Device Management）子进程读取和 macOS Keychain 预获取，它们的共同特征是：副作用与后续模块加载并行执行。源码注释明确写道："这些副作用必须在所有其他导入之前运行。"这三条并行任务在后续约 135 毫秒的模块加载期间同步完成，将冷启动延迟压到最低。

这种对启动顺序的精密控制，就是 Harness 工程的缩影。模型负责理解代码、生成方案；Harness 负责确保模型运行在一个可靠的环境里。没有 Harness，模型是裸露的引擎；有了 Harness，模型是可驾驶的整车。

这 51 万行代码的分布揭示了一个关键事实：绝大部分代码既不在用户界面，也不在 **API** 通信，而在两者之间的 **Harness** 控制层。

架构层	代码量	核心职责
交互层	~60,000 行	CLI 入口、Ink（React for CLI）TUI 组件、103 个斜杠命令
Harness 控制层	~400,000 行	9 个子系统：启动、核心循环、提示词、工具编排、上下文、权限、任务、多智能体、辅助
模型层	~35,000 行	Anthropic API 流式通信、Bedrock/Vertex 多提供商支持

交互层管"用户看到什么"，模型层管"如何与 API 说话"，**Harness** 控制层管**"所有智能决策"**。工具是否允许执行、上下文何时压缩、子智能体如何派生、权限如何分层——这些决策全部由 Harness 做出，模型只负责"建议"，Harness 负责"审批"。

这些代码的存在不是为了增强模型能力，而是为了管理模型能力的边界。 模型能生成 Bash 命令，但 Harness 决定是否允许执行；模型能读取任何文件，但 Harness 决定是否需要用户确认；模型能处理无限长的上下文（理论上），但 Harness 决定何时压缩、如何压缩、压缩后保留什么。

这就是 Harness 工程的核心问题：什么应该由模型管理，什么必须由 **Harness** 控制。

黄金法则——Harness 的本质是回答一个问题

Claude Code 的全部代码归根结底都在回答一个永恒问题：

什么应该由模型管理，什么必须由 **Harness** 控制？

这个问题没有唯一答案，但 Claude Code 的 `buildTool` 工厂函数给出了一种工程作答方式。`buildTool` 定义了工具的统一接口——每个工具都必须回答 Harness 的控制问题：是否并发安全？是否只读？是否具有破坏性？

`TOOL_DEFAULTS` 的默认值策略编码了 Claude Code 对这个问题的立场：

- `isConcurrencySafe` 默认 `false`——假设工具不安全，除非开发者显式声明
- `isReadOnly` 默认 `false`——假设工具会写入，除非显式标记

这套默认值策略体现了 **fail-closed**（失败关闭） 原则：当信息不完整时，系统选择更安全的方向。一个工具如果开发者忘了标记并发安全性，它会被当作不安全来处理**——**这可能导致性能损失（不能并行），但绝不会导致安全问题（不会并发执行危险操作）。

原则 **1.1**：默认关闭 — 当无法确定操作是否安全时，选择拒绝而非放行。过度保护可能影响性能，但不会造成安全事故。

这个原则在模型管理和 Harness 控制之间划出了一条清晰的界线：模型决定做什么（调用哪个工具、传什么参数），Harness 决定能做什么（是否允许执行、是否需要确认、是否可以并行）。两者的交互构成了 Harness 工程的核心张力。

适用场景——谁需要 Harness 工程

如果你正在构建一个让 AI 执行操作的软件系统——无论它叫 Agent、Copilot、还是 AI 助手——你都在做 Harness 工程。区别只在于你是否意识到。

Claude Code 的工具注册表列出了系统暴露给模型的全部工具，涵盖了五类通用 Agent 需求：

需求类别	工具示例	通用 **Agent** 中的对应
文件操作	FileReadTool、FileEditTool、FileWriteTool	读取/修改知识库、配置文件
命令执行	BashTool	调用外部 API、执行系统命令
搜索	GlobTool、GrepTool、WebSearchTool	信息检索、代码搜索
任务管理	AgentTool、TaskCreateTool、TaskListTool	子任务派发、多智能体协调
用户交互	AskUserQuestionTool、ConfigTool	向用户确认、调整配置

这份工具清单揭示了 Harness 工程的通用性——Claude Code 注册的工具类型，与任何 AI Agent 系统需要的工具类型高度重叠。工具的具体实现不同，但工具的类别和 **Harness** 管理工具的方式是可以迁移的。

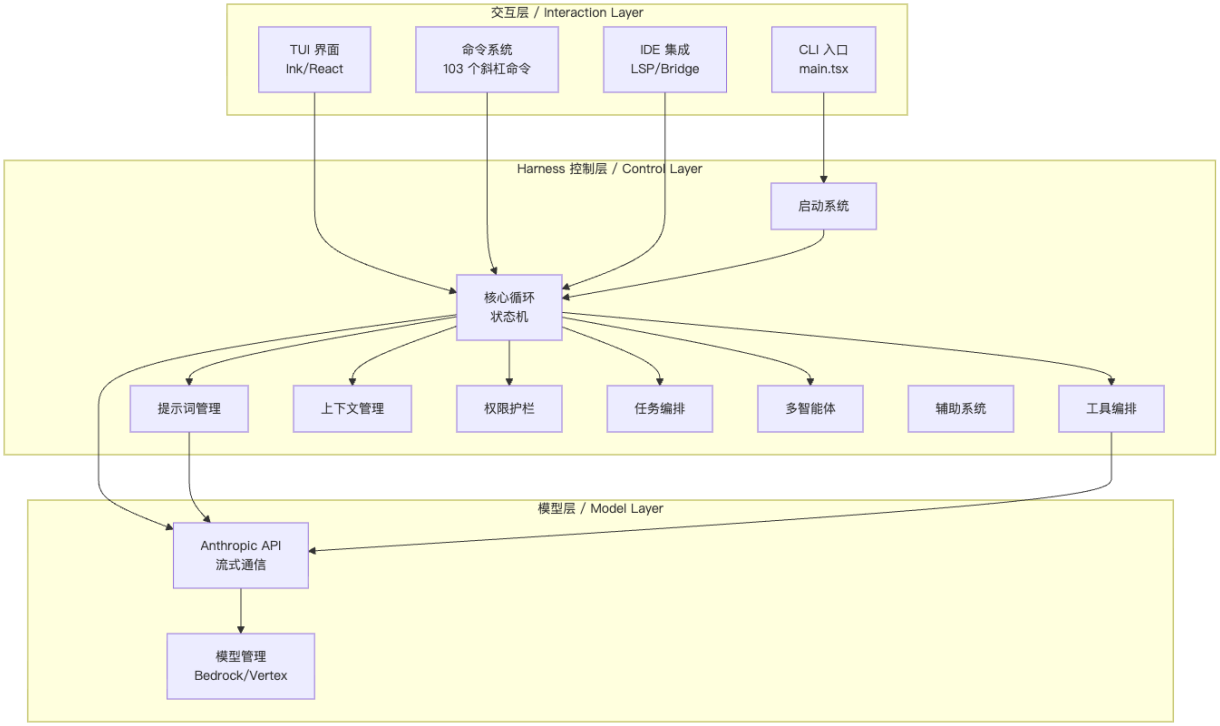
那么，谁需要关注 Harness 工程？

读者类型	场景	从本书获得什么
Agent 框架开发者	构建通用的 Agent 运行框架	工具编排、权限控制、上下文管理的工程模式
AI 应用工程师	在产品中集成 AI 能力	安全护栏、错误处理、性能优化的实践参考
架构师	设计大型 AI 系统的分层结构	三层 + 九子系统架构模型和设计权衡分析方法

工作原理——三层架构与九个子系统

Claude Code 的代码可以清晰地映射为三层架构。交互层处理用户输入和输出渲染，模型层封装与 Anthropic API 的通信，中间的 **Harness** 控制层承担所有"智能"决策。而 Harness 控制层内部，又由 9 个职责明确的子系统构成。

图 **1-1**：**Claude Code** 三层架构总图

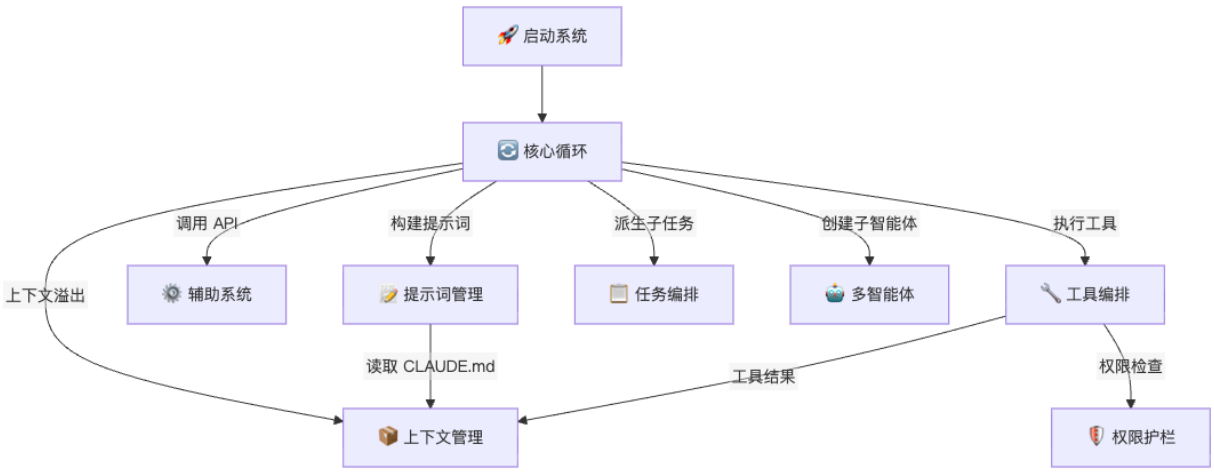


上图展示了三层之间的依赖方向：交互层调用 Harness 控制层，Harness 控制层调用模型层。反向不存在直接依赖——模型层不知道工具是什么，交互层不知道 API 怎么调用。

Harness 控制层的九个子系统

Harness 控制层是 51 万行代码的真正归宿。它内部的 9 个子系统各司其职，通过 `QueryEngineConfig` 的 30+ 个配置字段精确划定子系统间的接口。

图 1-2：Harness 控制层九子系统关系图



以下是每个子系统的一句话定位、源码位置和对应的后续章节：

1. 启动系统（详见第 2 章）

从进程入口到 REPL 就绪的全流程编排。前三行并行触发性能打点、MDM 子进程、Keychain 预获取，将冷启动延迟压到最低。Feature Flag 控制哪些代码被编译加载——每个 flag 都是对"模型在某件事上还不够好"的断言。

2. 核心循环系统（详见第 3 章）

驱动"用户输入 → API 调用 → 工具执行 → 继续或停止"的无限循环。`query()` 是一个 async generator，通过 `while(true)` 循环管理跨迭代的状态：消息列表、工具上下文、压缩追踪、输出 token 恢复计数。`QueryEngine` 类封装此循环为面向 SDK 消费者的接口。

3. 提示词管理系统（详见第 5 章）

动态构建发送给 LLM 的 system prompt。`getSystemPrompt()` 组装分段式提示词（身份、工具描述、安全指令、输出格式），`getSystemContext()` 注入 Git 状态快照，CLAUDE.md 加载器从 4 个优先级层级发现和注入项目指令。提示词分段区分缓存/非缓存段落，服务于 Prompt Cache 最大化命中。

4. 工具编排与执行系统（详见第 7 章）

将 LLM 的工具调用请求转化为实际操作。`runTools()` 按 `isConcurrencySafe()` 将工具调用分为只读（可并发）和写入（串行）两组，默认并发上限 10。`runToolUse()` 处理单个工具的完整流程：权限检查 → pre-tool hooks → execute → post-tool hooks。MCP（Model Context Protocol）集成允许动态加载外部工具。

5. 上下文管理系统（详见第 11 章）

管理对话历史和 token 预算，在上下文溢出时压缩。三级压缩策略：自动压缩（上下文超过阈值时触发）、微压缩（增量压缩最新工具结果）、快照压缩（通过 fork agent 生成摘要）。AppState 通过 React Context + `createStore()` 管理 UI 状态，集中状态模块管理 100+ 个全局状态函数。

6. 权限护栏系统（详见第 9 章）

四层防御确保 LLM 不会执行未授权的危险操作。第一层：配置规则（alwaysAllow/alwaysDeny）；第二层：AI 分类器自动判断（Auto 模式）；第三层：用户自定义 Hook（5022 行代码，6 个触发点）；第四层：交互确认弹窗。组织级策略限制从 API 获取，采用 fail-open 策略。

7. 任务编排系统（详见第 8 章）

管理后台任务生命周期和 Plan 模式。支持 7 种任务类型（LocalAgentTask、InProcessTeammateTask、RemoteAgentTask 等），每种适配不同执行场景。Plan 模式通过工具权限编码"先思考后行动"的约束。

8. 多智能体系统（详见第 13 章）

支持子智能体派生、Swarm/Teammate 协作和 Coordinator 编排三种模式。Fork Subagent 通过 `CacheSafeParams` 共享父级 API 缓存前缀，实现零成本上下文共享。Coordinator 模式通过环境变量启用中心化编排。

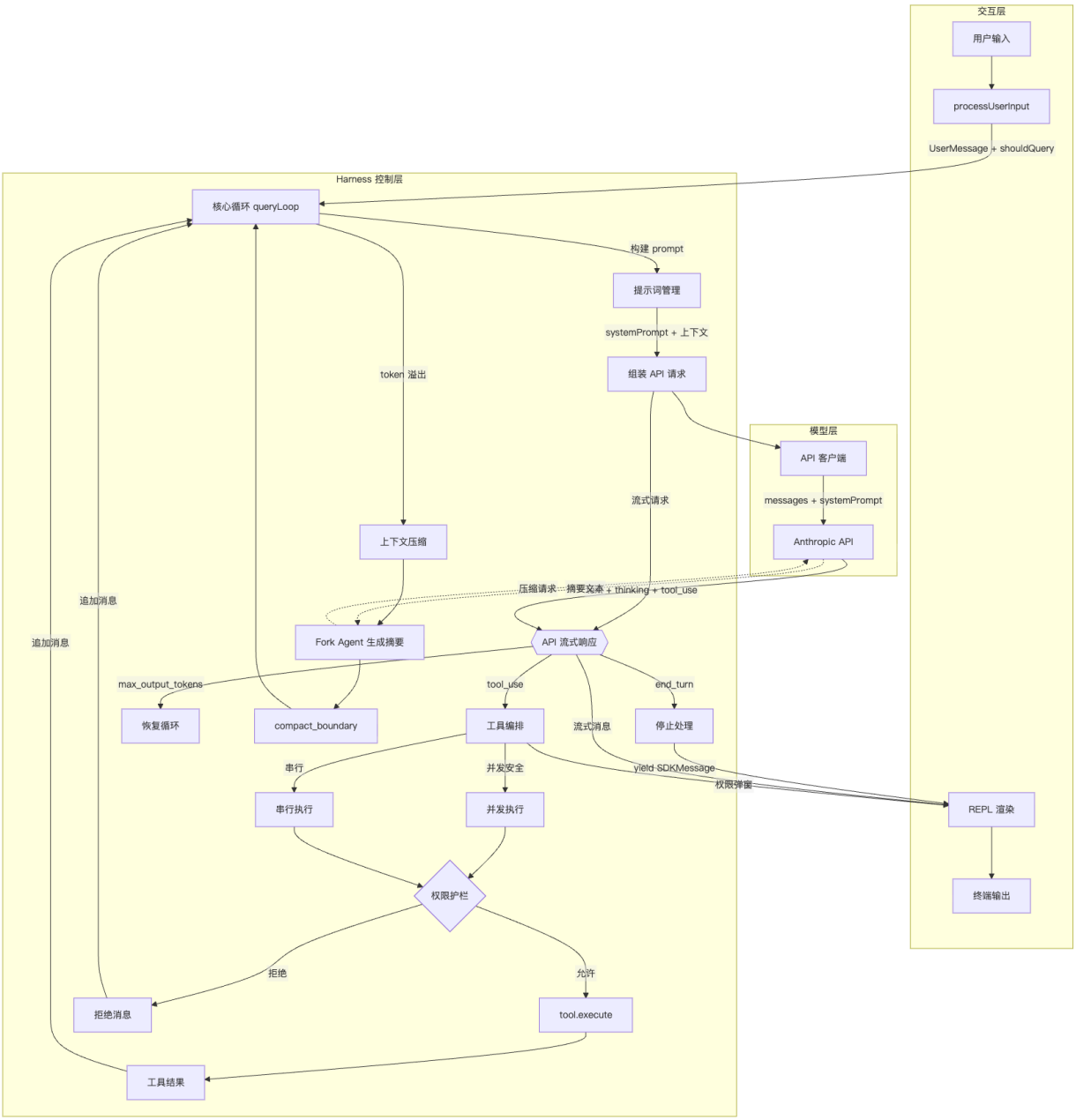
9. 辅助系统

API 客户端封装流式/非流式请求和多提供商支持。费用追踪每次调用的成本。设置管理从 5 个优先级来源加载配置（策略 > 远程管理 > 项目 > 用户 > CLI flag）。模型管理处理模型名称映射和提供商检测。

一次用户请求的完整旅程

上面我们看到了 9 个子系统各自做什么，但还没看到它们如何协作。让我们追踪一次用户输入从进系统到出结果的完整旅程，看三层 × 九子系统如何串成一条管道。

图 **1-3**：用户请求的完整数据流



阶段一：输入解析（交互层 → Harness 控制层）

用户在终端输入文本，`processUserInput()` 解析内容——识别斜杠命令、图片粘贴、@提及，生成 `UserMessage`。如果是斜杠命令（如 `/compact`），直接在 Harness 内处理并返回结果，不调用 API。如果需要模型参与（`shouldQuery = true`），消息被推入核心循环。

阶段二：提示词构建（Harness 控制层 · 提示词管理子系统）

核心循环调用 `fetchSystemPromptParts()` 并行构建三部分：system prompt（工具描述、安全指令、输出格式）、user context（用户环境信息）、system context（Git 状态快照）。CLAUDE.md 文件从 4 个优先级层级动态注入。所有部分组装后加上 API 缓存断点标记，然后发送给模型层。

阶段三：模型调用与响应（Harness 控制层 ↔ 模型层）

API 客户端将消息列表（messages）和系统提示词（systemPrompt）发送给 Anthropic API。响应以 SSE（Server-Sent Events）流的形式返回，包含三类内容块：**text**（模型生成的文本）、**thinking**（内部推理过程）、**tool_use**（工具调用指令）。Harness 控制层在流式接收的同时就开始处理——不等全部响应到达。上下文压缩时的 Fork Agent 也会向同一 API 发起独立请求来生成摘要。

阶段四：工具执行（Harness 控制层 · 工具编排 + 权限护栏）

如果 API 返回 `tool_use` 内容块，工具编排子系统接管。工具调用被分为并发安全（只读）和非并发安全（写入）两组。每个工具执行前经过权限护栏的四层检查。执行结果作为 `tool_result` 消息追加到对话历史，然后回到阶段二重新构建提示词并发起下一轮 API 调用。

这就是核心循环 `while(true)` 的本质——一个不断在提示词构建、API 调用、工具执行之间循环的状态机，直到模型返回 `end_turn` 或触发终止条件。

阶段五：上下文保护（Harness 控制层 · 上下文管理子系统）

如果某轮迭代后 token 使用量超过自动压缩阈值，上下文管理子系统启动压缩：通过 fork agent 生成对话摘要，插入 `compact_boundary` 标记。后续 API 调用只发送边界之后的消息 + 摘要，释放 token 空间。

阶段六：输出交付（Harness 控制层 → 交互层）

输出不是只在最后才回到交互层，而是在整个循环中持续流回：API 流式响应的每个文本块实时送达 REPL 渲染；工具执行前的权限弹窗由交互层的用户确认组件处理；最终 `end_turn` 后，核心循环 yield 完整的 SDKMessage 给交互层渲染为终端输出。交互层是信息的终点，也是用户控制的起点**——**权限弹窗的回复会流回 **Harness** 继续执行。

这 **9** 个子系统不是理论模型，而是源码中可验证的物理边界。 核心循环是它们的交汇点——上面的数据流图展示了每次 `query()` 迭代中，核心循环如何同时与提示词管理、工具编排、上下文管理、权限护栏四个子系统协作。

权衡——三层架构的设计选择

架构不是对错问题，是权衡问题。Claude Code 的每个架构决策都隐含着一个"选 A 不选 B"的理由。理解这些权衡，比记住结构的名字更重要。

权衡一：**Feature Flag vs** 配置文件

Claude Code 使用编译时 Feature Flag（`feature('KAIROS')`）而非运行时配置文件来控制模块加载。这意味着未开启的功能在编译时就被消除——零运行时开销、零包体积占用。但代价是灵活性：修改功能开关需要重新编译，而不是编辑一个 YAML 文件。

这个权衡的取舍方向很明确：对于 **AI Agent** 运行时，不可预测的条件分支本身就是风险源。 模型可能意外触达未完全测试的功能路径，Feature Flag 的编译时消除确保了这些路径在运行时物理上不存在。

权衡二：集中状态 **vs** 分散状态

`bootstrap/state.ts` 集中管理了 100+ 个全局状态函数——`getOriginalCwd()`、`setIsRemoteMode()`、`setMainLoopModelOverride()` 等。这个文件有 1762 行，是一个典型的"上帝模块"。

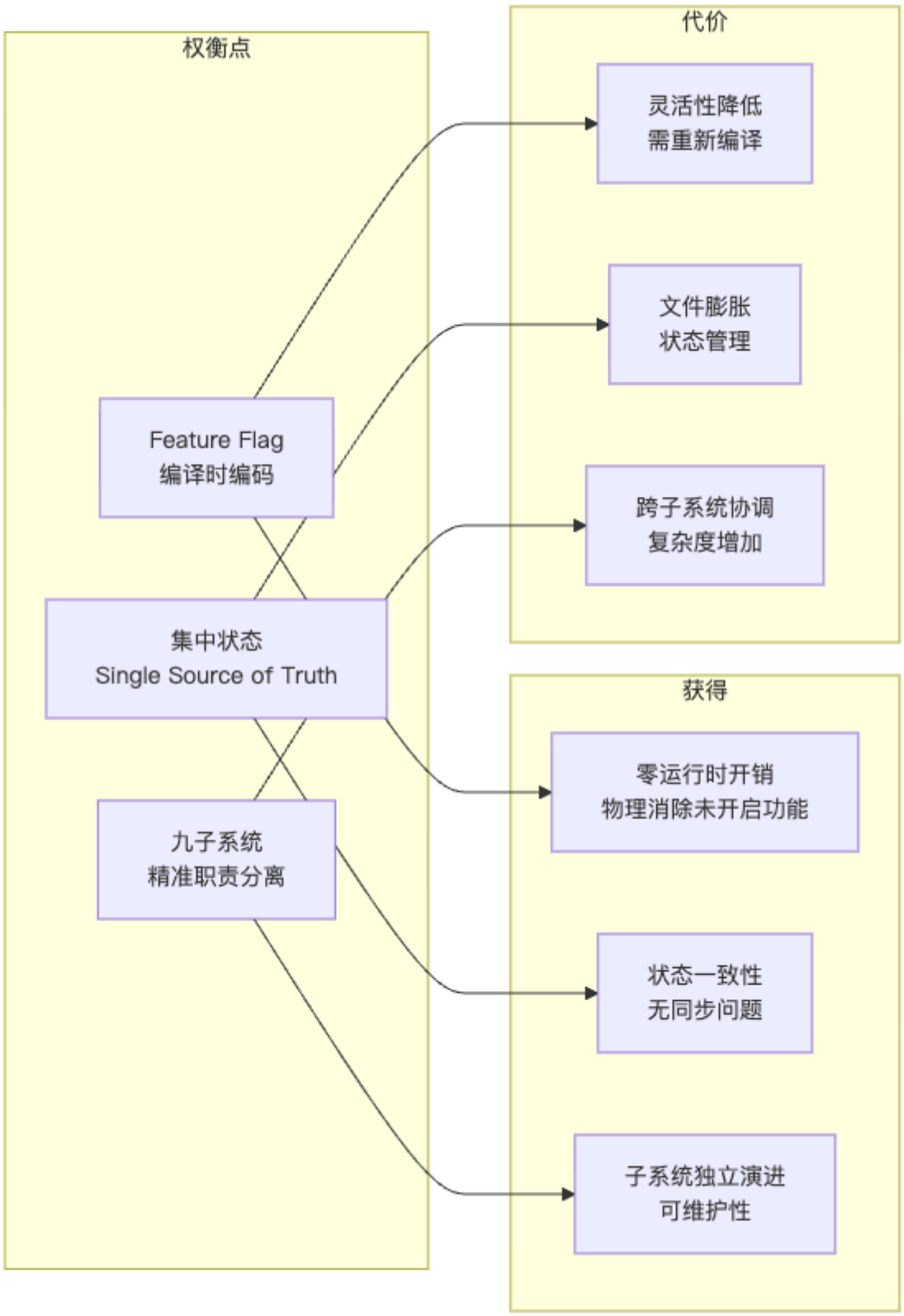
集中状态的好处是单一来源（**Single Source of Truth**）：任何模块需要获取工作目录、远程模式标志、模型覆盖设置，都从同一个文件获取，不会出现两个模块各自维护一份不同步的状态。代价是文件膨胀——1762 行的状态管理文件增加了理解和修改的认知负担。

权衡三：九子系统 **vs** 更粗粒度划分

9 个子系统带来精准的职责分离——提示词管理与上下文管理分开，权限护栏与 Hook 系统分开，任务编排与多智能体分开。但代价是跨子系统协调的复杂度：核心循环在每次迭代中同时与 4-5 个子系统交互。如果采用更粗粒度的划分（例如只分"工具层"、"上下文层"、"安全层"三层），协调更简单但职责边界模糊。

Claude Code 的选择：精准分离优先，因为可维护性比协调简洁性更重要。 当系统演进时，每个子系统可以独立修改而不影响其他子系统——这种解耦在 51 万行代码的项目中是生存必需的。

图 **1-4**：架构决策权衡矩阵



核心权衡总结：

决策维度	选择 A（Claude Code）	选择 B	核心权衡
功能开关	Feature Flag（编译时）	配置文件（运行时）	安全性 > 灵活性

决策维度	选择 **A**（**Claude Code**）	选择 **B**	核心权衡
状态管理	集中（bootstrap/state.ts）	分散（各模块自管）	一致性 > 模块内聚
子系统粒度	9 个精准子系统	3 个粗粒度模块	可维护性 > 协调简洁性

原则 **1.2**：权衡显式化 — 架构决策必须回答"选了什么、放弃了什么、为什么这样选"。没有权衡的架构决策要么是平庸的，要么是没有被认真思考过的。

踩坑指南——从源码中看到的工程陷阱

在学习 Harness 工程的最佳实践之前，先知道最常见的错误是什么。Claude Code 的源码揭示了三个反复出现的工程陷阱。

陷阱一：交互层和 **Harness** 层混淆

裸调 API 没有"架构"的概念，你随时可以调用，随时可以停止。但真实的 Agent 系统需要把"用户看到什么"（交互层）和"系统做什么决策"（Harness 控制层）分开。Claude Code 的交互层是 REPL.tsx——一个纯渲染组件，不含任何业务逻辑。如果你把权限检查写在 UI 组件里，换一个界面（比如 Web UI）就要重写全部权限逻辑。交互层只管渲染，**Harness** 只管决策，两者通过 **AppState** 接口通信。

陷阱二：工具默认不安全

`TOOL_DEFAULTS` 揭示了一个人性化的工程洞察：开发者会忘记标记工具的安全性。`isConcurrencySafe` 默认 `false` 的设计，意味着"忘记标记"的后果是工具不能并行——性能损失，但安全。如果把默认值设为 `true`，"忘记标记"的后果就是危险工具被并发执行——性能提升，但可能删除文件、覆盖数据。默认值的工程含义不是"正常情况"，而是"忘记时的情况"。

陷阱三：上下文无上限增长

LLM 的上下文窗口看似很大，但 Agent 系统的对话历史会持续增长。一个调试会话可能产生数万 token 的工具调用记录和返回结果。如果不主动管理上下文窗口，系统要么因为超出窗口限制而崩溃，要么因为上下文过长导致模型注意力分散而输出质量下降。Claude Code 并存了 5 种压缩策略（详见第 11 章），这本身就是对"上下文增长是 Agent 系统的头号敌人"这个认知的工程作答。

实证——三层架构在源码中的验证

三层 + Harness 层九子系统架构不是理论模型，可以从源码的目录结构、import 关系和类型定义中验证。让我们从交互层到模型层，逐一检查物理边界是否真实存在。

交互层的证据

`main.tsx` 中的 Commander 注册入口（`src/main.tsx:968`）是交互层的分叉点——用户输入在这里被解析为命令或对话，然后分发给 Harness 控制层。5005 行的 REPL.tsx（`src/screens/REPL.tsx:1`）是 TUI 主组件，它的职责是渲染消息和捕获输入——不含任何工具执行、权限检查或 API 调用逻辑。103 个命令目录（`src/commands/`）各自实现一个斜杠命令，全部是"解析输入、路由到 Harness"的模式。交互层不包含任何业务逻辑。

Harness 控制层的证据

`QueryEngineConfig` 类型（`src/QueryEngine.ts:130`）是 Harness 控制层的总接口。这个类型定义了 30+ 个字段，其中 `tools: Tools` 连接工具编排子系统，`mcpClients` 连接 MCP 外部服务，`canUseTool` 连接权限护栏子系统，`initialMessages` 连接上下文管理子系统，`abortController` 连接生命周期管理。**Config** 类型就是子系统之间的契约——它规定了核心循环可以向其他子系统传递什么、期望从它们获得什么。

9 个子系统的源码边界：

1. 启动系统 — `main.tsx` 的前三条语句（`src/main.tsx:12`）是性能打点、MDM 子进程、Keychain 预获取，注释明确标注"这些副作用必须在所有其他导入之前运行"。`feature('COORDINATOR_MODE')` 和 `feature('KAIROS')`（`src/main.tsx:76`）在编译时决定哪些模块被加载——启动系统和后续所有子系统的代码路径都受这些 flag 控制。

2. 核心循环 — `QueryEngine` 类（`src/QueryEngine.ts:184`）封装会话生命周期，`query()` 函数（`src/query.ts:219`）是 `async function*` 生成器，实现 `while(true)` 循环。`QueryEngineConfig` 的 30+ 字段精确定义了核心循环与其他子系统的全部交互接口——不多不少。

3. 提示词管理 — `getSystemPrompt()`（`src/constants/prompts.ts`）组装分段式系统提示词，`getSystemContext()` 和 `getUserContext()`（`src/context.ts`）分别构建系统上下文和用户上下文。CLAUDE.md 加载器（`src/utils/claudemd.ts`）从 4 个优先级层级发现和注入项目指令。提示词的分段结构区分缓存与非缓存段落，服务于 Prompt Cache 命中。

4. 工具编排与执行 — `tools.ts` 的注册列表（`src/tools.ts:195`）包含 54 个工具（19 个始终启用 + 35 个条件启用），全部通过 `buildTool` 工厂（`src/Tool.ts:783`）创建，遵循统一的 `Tool` 接口。`runTools()`（`src/services/tools/toolOrchestration.ts:150`）按 `isConcurrencySafe()` 分区为并发/串行两组。`runToolUse()`（`src/services/tools/toolExecution.ts:337`）处理单个工具的权限检查 → hooks → execute → 结果收集完整流程。

5. 上下文管理 — `AppState` 类型（`src/state/AppStateStore.ts`）通过 `createStore()` 管理 UI 状态。压缩模块（`src/services/compact/compact.ts`）实现

`compactConversation()`，通过 fork agent 生成对话摘要。全局状态模块（`src/bootstrap/state.ts`，1762 行）集中管理 100+ 个状态函数——工作目录、远程模式、模型覆盖等全部从单一来源获取。

6. 权限护栏 — 权限目录（`src/utils/permissions/`）包含 24 个文件、9409 行代码，实现四层防御。Hook 系统（`src/utils/hooks.ts`，5022 行）在 6 个触发点（PreToolUse、PostToolUse、Stop、SessionStart、SessionEnd、Notification）提供外部扩展能力。

7. 任务编排 — `src/tasks/` 目录包含 7 种任务类型（LocalAgentTask、InProcessTeammateTask、RemoteAgentTask、DreamTask、LocalShellTask、LocalWorkflowTask、MonitorMcpTask），每种有独立实现目录。Plan 模式通过 EnterPlanModeTool / ExitPlanModeTool 的工具权限切换实现"先规划后执行"。

8. 多智能体 — `AgentTool`（`src/tools/AgentTool/AgentTool.tsx:196`）通过 `buildTool` 注册，是主循环派发子任务的入口。Swarm 系统（`src/utils/swarm/`，14 个文件）实现进程内 teammate 创建、权限同步、leader-worker 桥接。Coordinator 模式（`src/coordinator/coordinatorMode.ts`）通过环境变量 `CLAUDE_CODE_COORDINATOR_MODE` 启用中心化编排。

9. 辅助系统 — API 客户端（`src/services/api/claude.ts`，3419 行）封装流式/非流式请求和多提供商支持。费用追踪（`src/cost-tracker.ts`）追踪每次调用的成本。设置管理从 5 个优先级来源加载配置（策略 > 远程管理 > 项目 > 用户 > CLI flag）。模型管理（`src/utils/model/`）处理模型名称映射和提供商检测。

模型层的证据

`src/services/api/claude.ts` 是唯一的 API 通信入口，3419 行代码封装了流式/非流式请求、Bedrock/Vertex 多提供商支持、token 使用量追踪。这个文件不知道工具是什么、权限怎么检查、上下文怎么压缩。 它的输入是消息列表和系统提示词，输出是流式响应块。模型管理（`src/utils/model/`）处理模型名称映射和提供商检测，同样不包含任何 Harness 逻辑。

三层之间的依赖方向是单向的：交互层 → Harness 控制层 → 模型层。反向不存在直接依赖。这个单向依赖可以在 `main.tsx` 的 import 链中验证——入口文件引入 `QueryEngine`（Harness），`QueryEngine` 引入 `claude.ts`（模型层），但 `claude.ts` 的 import 列表中既没有 `QueryEngine` 也没有任何交互层组件。

原则 **1.3**：架构的物理性 — 好的架构分层有源码级的物理证据——import 链、类型定义、目录结构。如果分层只存在于架构图上而无法从代码结构中验证，那它就是理论而非工程。

本章主成分：Harness 架构

本质：围绕"什么由模型管理，什么由 Harness 控制"这个核心问题组织的三层控制架构。交互层管输入输出，Harness 控制层管所有智能决策（9 个子系统），模型层管 API 通信。

关键机制：

- Feature Flag 编码模型能力假设，编译时物理消除未开启功能（启动系统）
- `buildTool` 统一工具接口，fail-closed 默认值确保安全（工具编排子系统）
- `QueryEngine` 管理会话生命周期，`QueryEngineConfig` 的 30+ 字段定义子系统间契约（核心循环子系统）
- 四层权限防御从配置到分类器到 Hook 到交互确认（权限护栏子系统）

适用边界：

- ✓ 适合：需要稳定、安全、可扩展的 Agent 系统
- ✓ 适合：需要多工具编排和多智能体协调的复杂 AI 应用
- ✗ 不适合：一次性脚本或简单的 API 封装
- ✗ 不适合：不需要工具执行的纯对话系统

与其他模式的关系：

- Harness 架构是中间层结构模式，对应 ReAct、Plan-and-Execute 等推理模式
- 九子系统各自独立，但通过核心循环的 `QueryEngineConfig` 接口协作

后续章节导航：

- 启动系统的并行初始化与 Feature Flag 机制 → 详见第 2 章
- 核心循环 QueryEngine 的会话状态机 → 详见第 3 章
- 单轮执行从提示词构建到流式工具执行的完整流水线 → 详见第 4 章
- 提示词管理子系统的分段缓存与 Prompt Cache 命中策略 → 详见第 5 章
- Skills 懒加载与上下文窗口的自管理 → 详见第 6 章
- 工具编排子系统的接口统一、并发安全策略与 MCP 扩展机制 → 详见第 7 章
- 任务编排子系统的 Plan Mode 如何通过工具权限编码"先规划后执行"约束 → 详见第 8 章
- 权限护栏子系统的渐进谱系与 AI 分类器的递归安全设计 → 详见第 9 章
- Hook 系统如何让 Harness 行为可配置而不改代码 → 详见第 10 章
- 上下文管理子系统的 5 种压缩策略各编码了什么模型能力假设 → 详见第 11 章
- 上下文重置与缓存命中如何兼顾、MEMORY.md 的分层设计 → 详见第 12 章
- 多智能体子系统的 GAN 式生成/验证协作与验证器提示词设计 → 详见第 13 章
- 后台任务并行执行与 Coordinator 多智能体协调 → 详见第 14 章
- Feature Flag 驱动的 Harness 自我精简机制 → 详见第 15 章

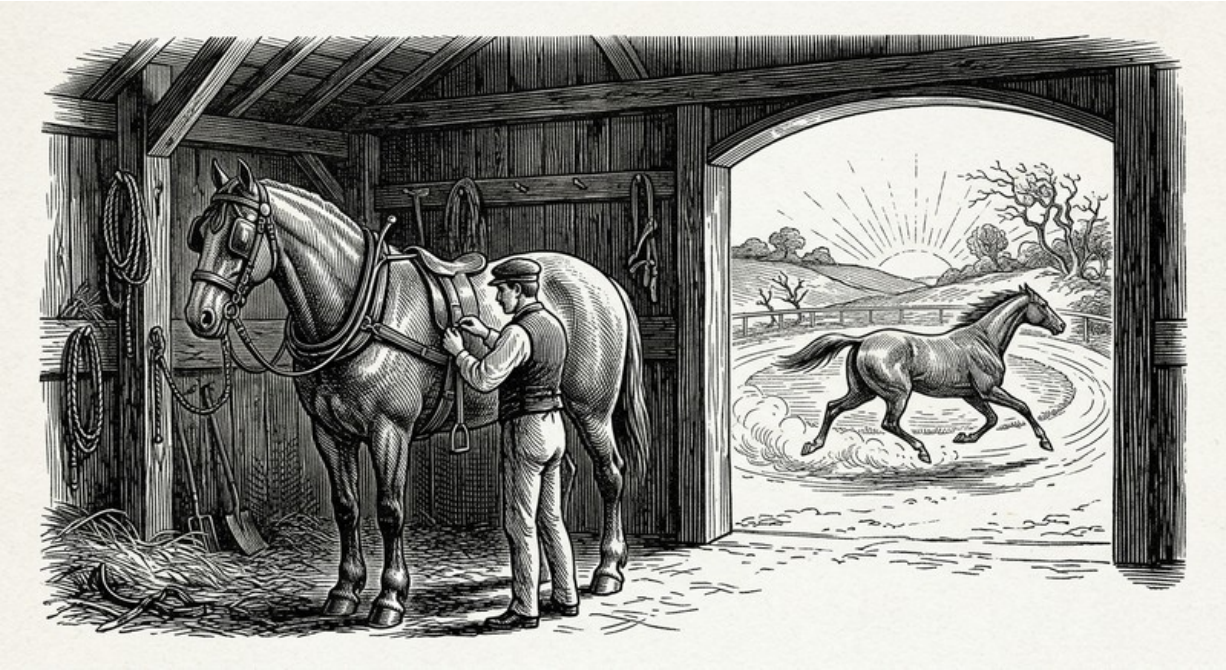
- 可复用的 Harness 工程决策框架，回答"什么由模型管理，什么由 Harness 控制" → 详见第 16 章

你能做什么

- 用三层 **+** 九子系统架构模型审视你的 **AI** 应用。画出你的应用三层图——交互层在哪里？Harness 控制层有哪些子系统？模型层如何封装？如果某层或某个子系统找不到明确的代码边界，说明那里的架构可能需要重构。
- 检查你的工具定义是否有 **fail-closed** 默认值策略。如果工具的默认行为是"允许一切"，一次遗漏标记就可能导致安全事故。
- 识别**"**模型管理**"**和**"Harness** 控制**"**的边界。在你的系统中，列出所有模型可以触发的操作，标注每个操作的 Harness 控制点（权限检查、并发限制、超时控制）。
- 用 **Feature Flag** 的思路评估你应用中硬编码的**"**模型能力假设**"**。你是否在代码中假设"模型一定能正确处理长文本"或"模型一定不会生成危险命令"？这些假设应该被显式编码，而不是隐式存在（详见第 15 章）。
- 画出启动序列的时间线。标注哪些初始化可以并行、哪些有依赖关系。Claude Code 在 135 毫秒内完成三项并行初始化——你的应用有多少启动时间可以利用？（详见第 2 章）
- 对照 `QueryEngineConfig` 的 **30+** 字段，检查你的 **Agent** 系统是否有同样完整的配置契约。缺失的字段往往意味着缺失的工程考量。
- 检查你的上下文管理是否有压缩策略。如果对话历史无限增长，你需要的不是一个更大的上下文窗口，而是一套压缩策略（详见第 11 章）。
- 阅读后续章节前，带着**"**这个子系统解决了什么权衡**"**的问题意识。每章不仅告诉你 Claude Code 做了什么，还会告诉你为什么这样做——权衡才是真正可迁移的工程智慧。

下一章导读：本章看到了三层 + 九子系统架构的全景，但没有深入任何子系统。第 2 章将进入第一个子系统——启动系统。我们会看到 Claude Code 的前三行代码如何通过并行初始化在 135 毫秒内完成所有准备工作，以及 Feature Flag 如何编码"模型能力假设"——每个 flag 都是对"模型在某件事上还不够好"的断言。这些 flag 的存在本身，就是 Harness 工程最诚实的自白。

第二篇：启动与循环



深入 **Harness** 的心脏地带。从第一个函数调用的并行初始化，到永不停止的主循环，再到单次 **API** 调用的完整流水线——理解这三层，就理解了 **Harness** 的心跳节奏。

第 2 章：点火序列

> "启动不是成本，是投资。每一毫秒都在为后续的稳定运行买单。"

你打开终端输入 `claude`，不到一秒就进入交互模式。这一秒内发生了什么？三条并行子进程的启动、~135 毫秒的模块加载、19 次性能打点、11 个版本迁移、Feature Flag 对模块图的裁剪——比大多数应用整个启动过程做的事情更多。而这一切都在回答同一个工程问题：不要让模型等 **I/O**。 读完本章，你将理解 Harness 如何在模型调用之前完成所有基础设施准备，以及 Feature Flag 如何编码"模型能力假设"。

问题——从进程启动到模型就绪，发生了什么

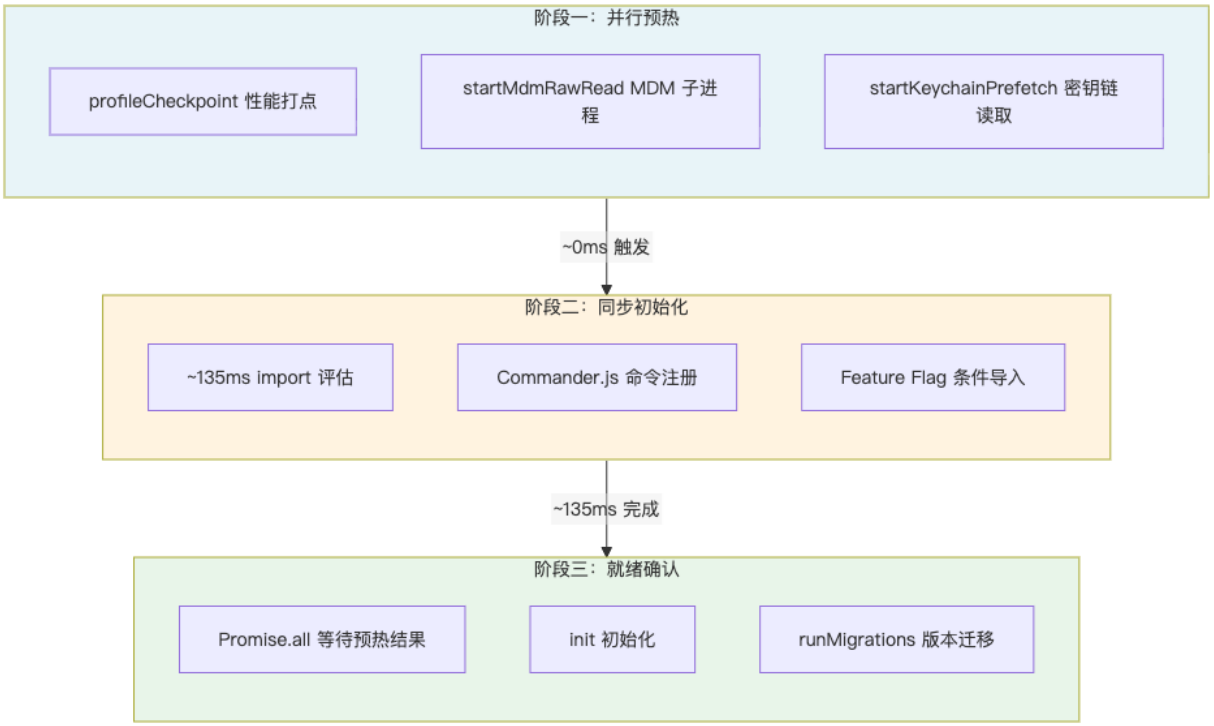
第 1 章的六层架构全景展示了启动层的位置，但没有展开。现在让我们进入第一层内部，看看从进程启动到模型就绪，这 ~200 毫秒内发生了什么。

Claude Code 的启动过程分三个阶段。源码注释精确说明了第一阶段的核心设计：

> "这些副作用必须在所有其他导入之前运行：
>
> 1. profileCheckpoint 在重型模块评估开始前标记入口点；
> 2. startMdmRawRead 触发 MDM 子进程使其与后续约 135ms 的导入并行运行；
> 3. startKeychainPrefetch 触发两次 macOS 密钥链读取的并行执行，否则它们会以同步方式在 applySafeConfigEnvironmentVariables 中顺序执行（每次 macOS 启动约 65ms）。"

这三行注释浓缩了启动层的全部设计哲学：在模块评估期间并行预热一切可能阻塞后续流程的 **I/O**。

图 **2-1**：启动序列三阶段



阶段一几乎零成本——三条子进程启动语句只触发 I/O，不等结果。阶段二是主要耗时——~135 毫秒的 JavaScript 模块评估。阶段三在 preAction 钩子中执行，用 `Promise.all` 收割阶段一种下的并行结果。因为子进程在 ~135 毫秒的 import 期间已经完成，`Promise.all` 几乎是免费的。

这三个阶段的时间分配揭示了一个关键洞察：启动时间不是线性累加的。 阶段一的三个 I/O 操作与阶段二的模块评估并行执行，总启动时间约等于阶段二的时间，而非三者之和。这就是并行预热的威力——用零额外等待时间消除了 ~65 毫秒的顺序 I/O 成本。

黄金法则——并行预热一切可能阻塞模型的 I/O

启动序列的核心原则可以用一句话概括：

不要让模型等 **I/O**。所有可能阻塞后续流程的 **I/O** 操作，都必须在模块评估期间并行启动。

这个原则的源码证据藏在 `keychainPrefetch.ts` 的注释中。这段注释是一份精算报告：密钥链的两次读取——"Claude Code-credentials"（OAuth 令牌）约 32 毫秒、"Claude Code"（遗留 API 密钥）约 33 毫秒——如果顺序执行，总计约 65 毫秒。而如果提前触发，让子进程在 ~135 毫秒的 import 评估期间并行运行，收割时"几乎免费"。

原则 **2.1**：预热先于等待 — 如果一个 I/O 操作的结果在后续一定会被使用，不要等到需要时才启动。提前触发，并行收割。

这个原则的适用条件很明确：操作之间没有依赖关系。MDM 配置读取不依赖密钥链读取，密钥链读取不依赖 import 评估。三者天然可以并行。但有一个微妙的约束——并行操作的 **import** 链必须保持最小化。注释明确警告：密钥链预获取故意不引入 `macOsKeychainStorage` 模块，因为后者会拉入 `execa → human-signals → cross-spawn` 的依赖链，带来约 58 毫秒的同步模块初始化开销。用更轻量的 `macOsKeychainHelpers` 替代，保持了 import 链的纯净。

这就是并行预热的另一面：并行是免费的，但并行操作的模块依赖是有成本的。 每多引入一个模块，就增加了 import 评估的时间——这个时间是同步的、不可避免的。

适用场景——哪些应用需要精心设计启动序列

Claude Code 不仅在启动时做了并行预热，还为此建立了一套专门的性能监控工具——`startupProfiler`。

`startupProfiler` 的采样策略编码了一种工程态度：启动性能值得持续监控。内部用户（Anthropic 员工）100% 上报启动耗时，外部用户 0.1% 采样。这种差异化采样说明 Anthropic 把启动性能当作产品特性来管理，而非开发时的临时调试工具。

源码中 19 个 `profileCheckpoint` 调用构成了一条完整的时间线，覆盖从入口到就绪的每个关键节点。这不是事后添加的——`profileCheckpoint('main_tsx_entry')` 是 `main.tsx` 的第一条可执行语句，在所有 import 之前。

那么，谁需要这种启动设计？

- **AI** 助手和 **Agent** 应用——模型首次响应的延迟直接影响用户体验，用户不会等待一个"正在初始化"的提示
- **CLI** 工具——命令行工具的启动速度感知尤为强烈，超过 500 毫秒用户就会觉得"慢"
- 需要认证的客户端应用——密钥链/OAuth 令牌的获取是典型的启动时 I/O 阻塞点

如果你的应用没有"用户等待首次响应"的压力，精心设计启动序列的收益可能不大——后台批处理任务或服务端长驻进程更关注吞吐量而非冷启动时间。

工作原理——Feature Flag 如何编码模型能力假设

启动层最精巧的设计不是并行预热——那只是经典的性能优化。真正独特的是 **Feature Flag** 对模块图的编译时裁剪。

每个 `feature('FLAG_NAME')` 调用都是一条隐含断言：**"**模型在 **X** 方面还不够好，需要 **Harness** 脚手架。**"** Flag 的存在是 Harness 对模型能力的诚实评估；Flag 的移除（当模型能力增强后）是 Harness 自我精简的信号。

`feature` 函数来自 `bun:bundle`，这是 Bun 运行时的编译时特性。当 Flag 为 `false` 时，`require()` 调用根本不会出现在编译产物中——不仅是条件跳过，而是物理消除（Dead Code Elimination, DCE）。

Claude Code 的关键 Feature Flag 清单揭示了 Anthropic 对模型能力边界的评估：

Feature Flag	功能	编码的能力假设
`KAIROS`	助手模式——自主管理完整工作流	模型还不能自主管理端到端的工作流，需要人类引导
`COORDINATOR_MODE`	多智能体协调模式	单个智能体不足以处理复杂任务，需要协调层
`TRANSCRIPT_CLASSIFIER`	自动权限分类	模型还不能可靠判断自身行为是否安全
`HISTORY_SNIP`	历史消息裁剪	模型在长上下文下的注意力会退化
`OVERFLOW_TEST_TOOL`	上下文溢出测试工具	模型可能生成超出上下文窗口的输出
`WEB_BROWSER_TOOL`	内置浏览器工具	模型需要浏览实时网页但无法直接访问
`AGENT_TRIGGERS`	定时触发器（Cron）	Agent 需要在无用户输入时主动执行任务
`WORKFLOW_SCRIPTS`	预置工作流脚本	模型还不能自行发现最优工作流

这张表是 Harness 工程最诚实的自白。每个 Flag 的名字无关紧要，真正有意义的是 Flag 背后的假设："如果没有这个脚手架，模型会在这个场景下失败。"

原则 **2.2**：**Feature Flag** 编码能力假设 — 每个 Feature Flag 都是对"模型在某方面还不够好"的显式断言。Flag 的存在是脚手架，Flag 的移除是里程碑。

Feature Flag 在启动层的运作方式是条件导入。以 `KAIROS` 为例：如果 Flag 为 `true`，`require('./assistant/index.js')` 加载整个助手模块；如果为 `false`，变量被赋值为 `null`，助手模块的代码根本不会进入 JavaScript 引擎。这种设计的一个直接后果是：**Flag** 关闭时，对应功能不存在运行时开销，也不存在被意外触达的风险。

同样的模式贯穿整个工具编排系统。`src/tools.ts` 中的工具注册使用条件导入：`OVERFLOW_TEST_TOOL`、`CONTEXT_COLLAPSE`、`TERMINAL_PANEL`、`WEB_BROWSER_TOOL` 等——每个工具的注册都受 Feature Flag 控制。模型的工具列表会根据 Flag 状态动态变化，模型甚至不知道自己"少了"哪些工具。

这与第 1 章的六层架构有什么关系？Feature Flag 是启动层的核心机制——它决定了六层架构中哪些层会被实例化、哪些工具会被注册、哪些模块会被加载。启动层是六层架构的开关板，**Feature Flag** 是开关。

权衡——启动序列的设计选择

启动序列不是"越快越好"——它是一组精心校准的权衡。

权衡一：最小 **import** 链 **vs** 开发便利性

密钥链预获取模块故意不引入 `macOsKeychainStorage`，而是用更轻量的 `macOsKeychainHelpers`。注释明确计算了代价差异：`macOsKeychainStorage` 会拉入 `execa → human-signals → cross-spawn` 的依赖链，约 58 毫秒的同步初始化开销。为了节省 **58** 毫秒，开发者放弃了直接调用完整 **API** 的便利性，转而使用一个功能子集的轻量替代。

这个权衡的取舍方向：在启动路径上，每毫秒都是昂贵的。开发便利性在运行时路径上可以追求，但启动路径必须极致精简。

权衡二：同步迁移 **vs** 异步迁移

`runMigrations()` 函数区分了关键迁移和非关键迁移。关键迁移（如默认模型重置、权限配置迁移）同步执行——确保后续代码看到的配置状态是一致的。非关键迁移（如 changelog 格式迁移）用 `fire-and-forget` 异步执行，失败时静默忽略并等待下次启动重试。

迁移类型	执行方式	失败处理	理由
关键迁移（11 个）	同步	阻塞启动	后续代码依赖迁移后的配置状态
非关键迁移	异步	静默忽略	不影响启动后的功能正确性

权衡三：失败开放 **vs** 失败关闭

企业策略加载（`loadRemoteManagedSettings` 和 `loadPolicyLimits`）使用 `void` 前缀——即 `fire-and-forget`。这意味着如果企业策略服务不可用，系统会在没有策略限制的情况下继续启动。

这与第 1 章工具默认值的 fail-closed 策略形成了有趣的对比。工具编排系统选择"失败关闭"——宁可拒绝也不误放行；启动层选择"失败开放"——宁可没有策略也不阻塞启动。两种选择都是正确的，因为代价不同：工具编排系统误放行的代价是安全事故，启动层阻塞的代价是用户无法使用产品。

原则 **2.3**：权衡的代价决定方向 — 选择 fail-closed 还是 fail-open，不取决于组件的重要性，而取决于失败的代价。安全场景拒绝优于放行；可用性场景启动优于等待。

踩坑指南——启动序列中的工程陷阱

Claude Code 的源码注释本身就是最佳实践的教科书。从这些注释中可以提炼出三个常见的启动工程陷阱。

陷阱一：在 **import** 链中引入重模块

`keychainPrefetch.ts` 的注释精确计算了引入 `macOsKeychainStorage` 的代价：`execa → human-signals → cross-spawn`，约 58 毫秒的同步模块初始化。在启动路径上，58 毫秒

不是"微小优化"，而是总启动时间 135 毫秒的 43%。一个看似无害的 **import** 就能吞噬近一半的启动预算。

更危险的是，这种膨胀通常是渐进的：今天引入一个工具函数，明天引入一个格式化库，后天引入一个 HTTP 客户端——每次都是"只多一个"，累计后启动时间翻倍。防御方法是建立启动路径的 import 审查机制。

陷阱二：同步 **I/O** 阻塞事件循环

MDM 配置读取如果改用同步方式（`execSync` 而非 `execFile`），会阻塞整个事件循环约 65 毫秒——在这期间什么都不能做。Claude Code 的做法是启动异步子进程，然后在需要结果时收割。同步 **I/O** 在启动路径上不是**"简单"**，而是**"懒惰"**。

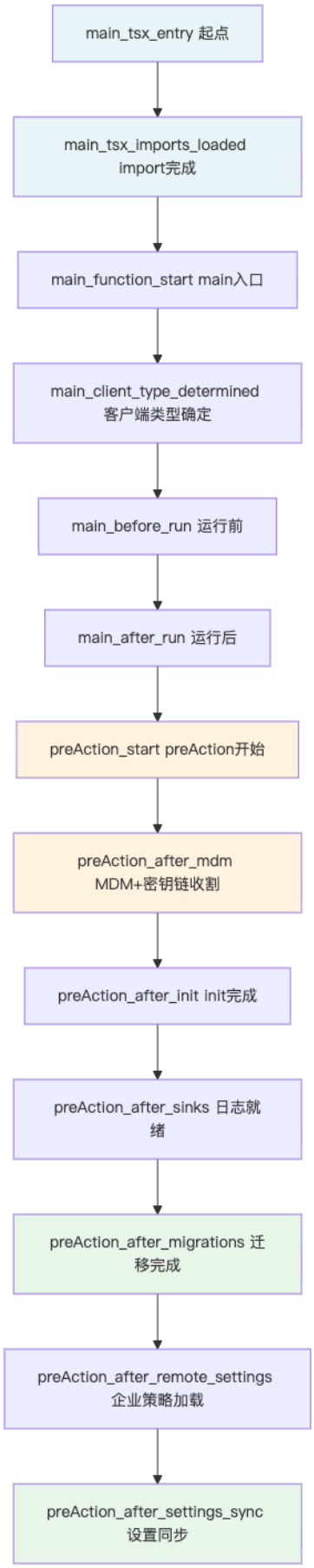
陷阱三：迁移逻辑阻塞启动

`runMigrations()` 同步执行 11 个关键迁移。如果每个迁移涉及文件 I/O 或配置写入，累计时间可能成为新的启动瓶颈。Claude Code 的防御策略是严格区分关键和非关键迁移，非关键的一律异步。如果你的应用有类似的迁移需求，这个区分应该在第一天就建立——等到迁移数量膨胀后再拆分，改造成本远高于一开始就做好。

实证——从 profileCheckpoint 追踪启动时间线

19 个 `profileCheckpoint` 调用构成了一条完整的启动时间线。让我们追踪这条时间线，验证三阶段模型的准确性。

图 **2-2**：**profileCheckpoint** 启动时间线



从这条时间线可以验证三个关键观察：

第一，**import** 评估是启动的主要耗时。 从 `main_tsx_entry`（`src/main.tsx:12`）到 `main_tsx_imports_loaded`（`src/main.tsx:209`）之间的 ~135 毫秒，是整个启动过程中最长的单一段落。这 135 毫秒内，JavaScript 引擎评估了 `main.tsx` 的所有顶层 import——包括 React、Ink、Commander、lodash 等大型库。

第二，并行预热在 **preAction** 中收割几乎免费。 `preAction_start`（`src/main.tsx:908`）到 `preAction_after_mdm`（`src/main.tsx:915`）之间的时间极短——因为 MDM 子进程和密钥链读取在 ~135 毫秒前就已经启动，此时早已完成。`Promise.all` 的等待时间约等于零。

第三，迁移和企业策略加载是 **preAction** 中最重的操作。 `preAction_after_migrations`（`src/main.tsx:951`）和 `preAction_after_settings_sync`（`src/main.tsx:966`）之间的操作包括迁移执行、企业策略加载和设置同步。企业策略使用 `void`（fire-and-forget），所以实际的阻塞时间主要来自 11 个同步迁移。

这条时间线验证了第 1 章提出的六层架构模型——启动层确实是独立的、可验证的物理边界。从 `main_tsx_entry` 到 `preAction_after_settings_sync`，每个检查点都有明确的职责，每个阶段之间的过渡都有源码级的标记。

本章主成分：并行预热

本质：在模型首次调用之前，并行预热一切可能阻塞的 I/O 操作，用编译时 Feature Flag 裁剪模块图。

关键机制：

- import 阶段启动副作用子进程，preAction 阶段用 `Promise.all` 收割
- Feature Flag 编码模型能力假设，编译时物理消除未开启模块
- 启动路径 import 链严格最小化，避免同步模块初始化膨胀

适用边界：

- ✓ 适合：需要快速首次响应的 AI 应用和 CLI 工具
- ✓ 适合：需要在启动时获取认证凭据的客户端应用
- ✗ 不适合：后台批处理任务（更关注吞吐量）
- ✗ 不适合：服务端长驻进程（冷启动频率低）

与其他模式的关系：

- 启动层是六层架构的第一层（详见第 1 章）
- Feature Flag 与压缩策略共享"模型能力假设"的设计哲学（详见第 11 章、第 15 章）
- 启动层的 fail-open 策略与工具编排系统的 fail-closed 策略形成对比（详见第 7 章、第 9 章）

你能做什么

- 审视你的 **AI** 应用启动流程，找出所有同步 **I/O** 点。逐个评估：哪些可以改为异步预热？哪些可以在模块加载期间提前触发？
- 将可以并行的 **I/O** 操作移到应用加载阶段。用 `Promise.all` 在首次需要时收割结果。Claude Code 节省了 ~65 毫秒——你的应用可能节省更多。
- 为你的启动流程添加 **profileCheckpoint** 式的时间标记。在每个关键节点打点，用数据而非直觉驱动优化。
- 列出你应用中硬编码的**"模型能力假设"**。哪些假设应该变成 Feature Flag？哪些 Flag 应该在模型能力增强后移除？
- 检查你的 **import** 链中是否有隐含的重模块依赖。一个 `import` 可能拉入整个依赖树。在启动路径上，每个 import 都应该被审查。
- 评估你的迁移策略是否区分了关键和非关键操作。关键迁移同步执行，非关键迁移异步 fire-and-forget——这个区分应该在第一天建立。
- 对比你的启动时间和 **Claude Code** 的 **~200** 毫秒目标。如果你需要 2 秒才能首次响应用户，差距可能不在模型调用，而在启动流程。

下一章导读：启动层在 ~200 毫秒内完成了所有准备工作，但"准备就绪"只是开始。第 3 章将进入第二层——核心循环层。我们会看到 `QueryEngine` 如何管理一个会话的完整生命周期：何时继续循环、何时压缩上下文、何时终止对话。这个状态机的设计决定了 Harness 能让模型连续工作多长时间而不"失控"。

第 3 章：永不停歇的循环

> "循环不是重复，是收敛。每一轮都在向目标靠近一步——或者向终止条件靠近一步。"

每个用户消息都触发一个状态机。它可能循环一轮，也可能循环数十轮——每轮完成一次"调用模型→执行工具→收集结果"的完整迭代，然后重新开始。驱动这个状态机的不是用户指令，而是 Harness 对"模型在什么条件下会失控"的工程预判。读完本章，你将理解 `QueryEngine` 如何管理一个会话的完整生命周期，以及三层守卫如何决定循环何时继续、何时压缩、何时终止。

问题——用户消息之后发生了什么

大多数开发者假设 AI 对话是"发一条收一条"——发送用户消息，收到助手回复，交互结束。Claude Code 的实际行为颠覆了这个假设。

一个用户消息"帮我重构这个函数"可能触发以下序列：读取文件→分析代码→提出方案→等待用户确认→修改文件→运行测试→修复测试错误→再次运行测试→确认通过。这不是一次 API 调用，而是 8 次或更多次。每次调用的结果都影响下一次调用的输入，消息历史在循环中持续积累。

`QueryEngine` 是这个序列的载体。它是一个有状态的会话级对象——构造一次，跨越整个会话的所有轮次持久存在。`mutableMessages` 字段积累了会话中所有消息，每次 `submitMessage` 调用向其追加新的消息并启动新一轮循环。

这与无状态管道模型的区别是根本性的：

	管道模型	状态机模型（**QueryEngine**）
状态	每次调用独立，无跨调用状态	mutableMessages 跨轮次积累
控制流	线性：输入→处理→输出	循环：工具调用触发下一轮
终止	调用完成即结束	由三层守卫决定何时终止
失败处理	调用失败即错误	可以重试、压缩、优雅降级

原则 **3.1**：会话即状态 — Agent 对话不是无状态的管道，而是有状态的状态机。消息历史、轮次计数、压缩边界——这些状态决定了模型在每一轮能"看到"什么、能"做"什么。

黄金法则——Harness 循环的三层守卫

无限制的循环是 Agent 最危险的失败模式。模型可以永远"再试一次"——每次工具调用失败都触发下一次尝试，每次尝试都消耗更多 token 和费用，最终耗尽预算或上下文窗口。

`QueryEngine` 用三层守卫防止这种失控：

第一层：轮次上限（**maxTurns**）

maxTurns 是最直接的硬终止条件。每完成一轮工具调用循环，`turnCount` 递增 1。当 `nextTurnCount > maxTurns` 时，循环向调用方 yield 一条 `max_turns_reached` 消息，然后立即返回，终止原因为 `'max_turns'`。

关键细节在于这个检查的位置——它在工具结果收集完成、下一轮循环开始之前执行。这意味着最后一轮的工具结果会被处理，但不会触发新的 API 调用。用户看到的是完整的工具执行结果，而非被截断的中间状态。

第二层：费用上限（**maxBudgetUsd**）

maxBudgetUsd 将 token 消耗转换为费用约束，适用于企业客户和 SDK 调用场景。与 maxTurns 不同，这个守卫在每轮 API 响应后检查累计费用，而非在轮次边界。它编码了一个业务判断：宁可让任务未完成，也不要超出预算限制。

第三层：上下文阻塞（**blocking_limit**）

blocking_limit 是三层守卫中最复杂的一层。当上下文窗口中的 token 数接近模型最大值时，继续循环会导致 API 返回 `invalid_request` 错误——这是无法恢复的失败。blocking_limit 检查在每轮循环开始时执行，在 API 调用之前。如果触发，立即返回，原因为 `'blocking_limit'`。

复杂之处在于这个检查有 5 个跳过条件：压缩刚完成时跳过（避免使用过时的 token 计数）、`compact` 和 `session_memory` 来源的查询跳过（这些查询本身在执行压缩，阻塞它们会造成死锁）、reactive compact 开启时跳过（由另一个机制接管），以及 context collapse 开启且自动压缩允许时跳过。这 5 个条件共同防止了守卫在不该触发时误触发。

三层守卫对比：

守卫	触发时机	编码的假设	适用场景
maxTurns	轮次边界	模型可能陷入无效循环	SDK 调用、批处理任务
maxBudgetUsd	API 响应后	费用必须有上限	企业客户、受控环境
blocking_limit	轮次开始前	超出上下文窗口无法恢复	所有会话

原则 **3.2**：多层守卫，各司其职 — 不要用单一的超时或错误处理替代多层终止守卫。每层守卫针对不同的失控场景，缺少任何一层都会留下失控漏洞。

适用场景——谁需要会话级状态机

如果你的 Agent 只做一轮 API 调用——发送消息，接收回复，结束——你不需要 QueryEngine 这样的循环管理。但任何需要"调用工具→看结果→决定下一步"的 Agent 都面临循环管理的问题。

`ask` 函数是 `QueryEngine` 面向外部的接口，它的参数列表是 Anthropic 对"哪些维度需要可配置"的完整回答：`maxTurns`（轮次上限）、`maxBudgetUsd`（费用上限）、`taskBudget`（任务预算）、`tools`（可用工具集）、`canUseTool`（动态权限检查）、`abortController`（外部中断）、`thinkingConfig`（思考模式）、`customSystemPrompt`（自定义系统提示）……超过 25 个参数。

这张参数列表回答了"什么需要由 Harness 控制"：循环的边界条件、工具的权限控制、状态的持久化方式、中断的触发机制。模型控制循环内的决策（调用哪个工具、如何使用工具结果），**Harness** 控制循环本身的边界。

谁需要这种设计？

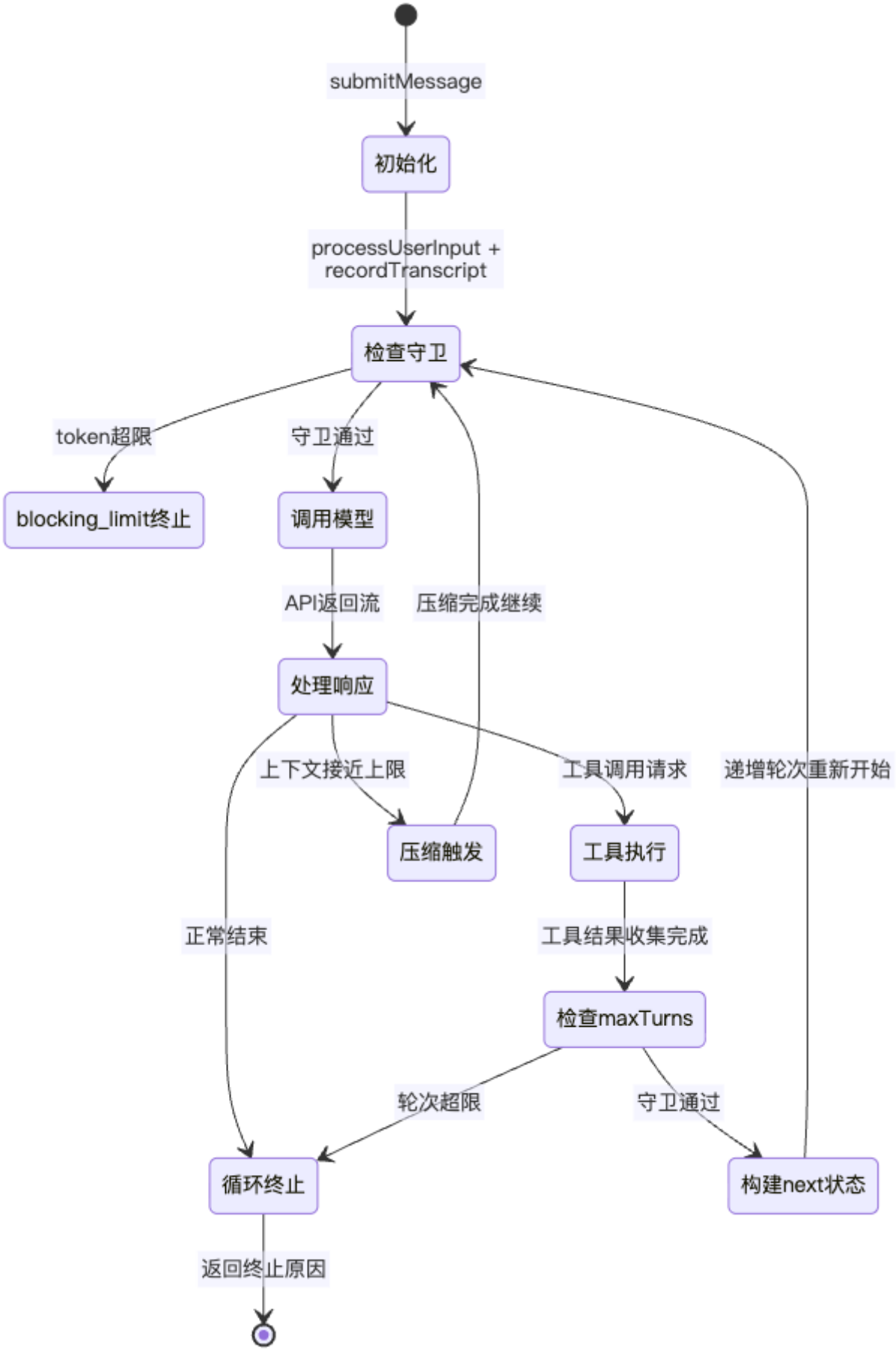
- 代码编辑 Agent——需要多轮工具调用（读文件、分析、修改、验证）
- 研究 Agent——需要迭代搜索和综合（搜索→读取→提炼→再搜索）
- 任务执行 Agent——需要分解→执行→验证的循环

如果你的 Agent 超过 3 轮才能完成任务，你就需要认真设计循环的守卫机制——而不只是让它"一直跑直到完成"。

工作原理——queryLoop 的完整生命周期

`queryLoop` 是 Claude Code 的心脏。它的结构出奇简洁：一个 `while (true)` 无限循环，一个 `State` 对象携带跨迭代状态，每轮迭代结束时用 `state = next` 更新状态并继续。

图 **3-1**：**QueryEngine** 循环状态机



循环的每次迭代经过以下步骤：

步骤一：解构当前状态

每次 `while (true)` 循环开始时，首先解构 `state` 对象，取出 `messages`、`toolUseContext`、`turnCount`、`autoCompactTracking` 等字段。这个设计让每轮迭代都从一个干净的状态快照开始——没有散乱的可变变量，只有 `state` 对象。

步骤二：检查终止条件

解构之后，循环依次检查：自动压缩条件（如果上下文窗口已接近上限，触发压缩后 continue 到下一轮）、blocking_limit（如果确实超限且无法压缩，终止）。

步骤三：调用模型 **API**

通过 `deps.callModel` 发起流式 API 调用，返回一个异步生成器。循环用 `for await` 消费每条流式消息：`message_start`（重置当前消息的 token 计数）、`content_block_start/delta/stop`（收集内容块）、`message_delta`（捕获 stop_reason 和最终 token 用量）、`message_stop`（标记消息结束）。

步骤四：执行工具调用

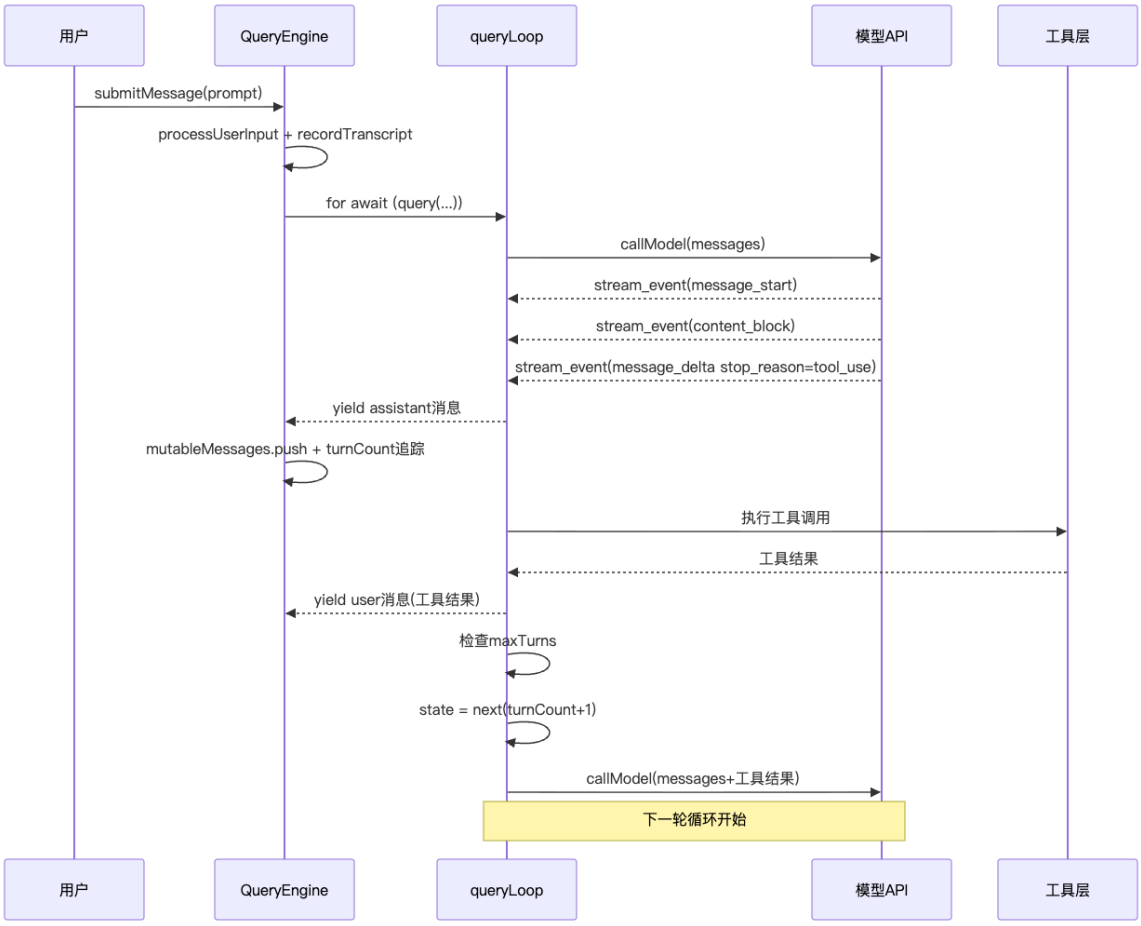
如果助手消息的 stop_reason 是 `tool_use`，循环收集所有工具调用，并发执行只读安全的工具，串行执行其他工具（详见第 7 章）。工具结果作为 user 消息追加到消息列表。

步骤五：构建 **next** 状态，继续循环

工具结果收集完成后，检查 maxTurns。如果未超限，构建新的 `State` 对象 `next`：更新消息列表（包含本轮的 assistant 消息和工具结果）、递增 `turnCount`、重置 `hasAttemptedReactiveCompact` 等临时状态。然后 `state = next`，`while (true)` 进入下一轮。

`taskBudgetRemaining` 是一个特殊的循环局部变量——它不在 `State` 对象中，而是独立声明在循环外。注释解释了原因：State 对象有 7 个 continue 点，如果把 taskBudgetRemaining 加入 State，就需要在 7 处同时更新，极易遗漏。循环局部变量只需在一处维护，以代码复杂度换取正确性。

图 **3-2**：一轮循环的消息流



权衡——循环设计中的工程选择

`queryLoop` 的每个设计决策都在可靠性和复杂度之间做权衡。三个关键权衡尤其值得关注。

权衡一：**transcript** 即时写入 **vs** 延迟写入

用户消息在进入 query 循环之前就写入 transcript（磁盘持久化）。注释解释了为什么：如果进程在 API 响应之前被杀死（比如用户点击停止），`for await` 循环还没有 yield 任何 assistant 消息，transcript 里只有队列操作记录——`--resume` 时无法找到对话，恢复失败。

提前写入确保了"用户消息已被接受"这个事实在任何情况下都不会丢失，即使从未收到 API 响应。

代价是 ~4ms（SSD）到 ~30ms（磁盘竞争时）的额外延迟。在 `--bare` 模式下，这个写入改为 fire-and-forget——脚本调用不需要 resume，不值得为此阻塞。同一个操作，交互模式下是安全优先，脚本模式下是性能优先。

权衡二：**7** 个 **continue** 点 **vs** 中央化状态更新

queryLoop 有 7 个 continue 点——每个都代表一种"本轮循环结束，重新开始"的场景（压缩完成、工具结果收集完成、各类错误恢复等）。每个 continue 点都必须正确设置 `state = next`，否则状态就会损坏。

为什么不把这 7 个 continue 点合并为一个中央化的状态更新？因为每种场景更新的字段不同——压缩完成后需要更新 `autoCompactTracking`，工具执行完成后需要更新消息列表和 toolUseContext，错误恢复后需要更新 `maxOutputTokensRecoveryCount`。强行中央化会引入大量条件分支，反而更难维护。

这个权衡的结果是：复杂度被分散到 7 处，但每处的逻辑是局部的、内聚的。`taskBudgetRemaining` 不加入 State 的决定，正是为了避免让这 7 处更新更加复杂。

权衡三：**blocking_limit 的 5 个跳过条件 vs 简单硬终止**

blocking_limit 检查如果是简单的"token 超限就终止"，代码会非常简洁。但 5 个跳过条件是必要的：

跳过条件	不跳过会发生什么
压缩刚完成	使用过时 token 计数，误触发 blocking_limit
compact/session_memory 查询	死锁：压缩 Agent 无法运行，上下文无法减少
reactive compact 开启	抢占合法的压缩路径，无法恢复
context collapse 开启且自动压缩允许	同上，两种压缩机制互相干扰

原则 **3.3**：守卫的豁免比守卫本身更重要 — 错误的终止比不终止危害更大。终止守卫必须精确——知道"什么时候不该终止"和知道"什么时候该终止"同样重要。

踩坑指南——循环设计中的陷阱

Claude Code 源码注释揭示了几个真实踩过的坑，值得在设计自己的循环时警惕。

陷阱一：只在正常路径检查 **maxTurns**

abort 路径（用户中断工具执行）也需要检查 maxTurns。在工具执行被中断时，代码依然检查 `nextTurnCountOnAbort > maxTurns`，如果超限同样 yield `max_turns_reached`。注释明确写道："Check maxTurns before returning when aborted"。

如果只在正常路径检查，用户可以通过频繁中断来绕过轮次限制——每次中断重新开始，但轮次计数不清零。

陷阱二：延迟写入 **transcript**

上文已提到：延迟写入导致进程被杀后无法 resume。这不是假设场景——用户在编辑器中点击"停止"或网络超时都会触发。transcript 的写入时机决定了 resume 功能的可靠性上限。

陷阱三：在多个 **continue** 点各自维护状态

如果你有多个循环出口（多个 continue/break），每个出口都需要更新的状态字段越多，出错的概率越高。Claude Code 的解决方案是将所有跨迭代状态收入 `State` 对象，每个 continue 点显式构建完整的 `State`。这让状态更新变得可见、可检查，而不是散落在循环体各处的可变变量。

实证——从源码追踪一轮完整循环

让我们追踪一个用户消息从进入 `QueryEngine` 到触发下一轮循环的完整路径，验证状态机模型的准确性。

`submitMessage` 的入口（`src/QueryEngine.ts:209`）首先调用 `processUserInput` 处理用户输入，然后将 `messagesFromUserInput` push 到 `mutableMessages`（`src/QueryEngine.ts:430`）。关键步骤随即发生：在进入 `query` 循环的 `for await` 之前，将消息写入 transcript（`src/QueryEngine.ts:438-449`）——注释用三段话解释了这个"提前写入"的必要性：进程被杀、队列操作记录、`--resume` 失败。

`for await` 循环（`src/QueryEngine.ts:675`）消费 `query()` 生成器的每条消息。当消息类型为 `assistant` 时，push 到 `mutableMessages`，同时追踪 `stop_reason`（`src/QueryEngine.ts:762-769`）。当类型为 `user`（工具结果）时，同样 push 到 `mutableMessages`（`src/QueryEngine.ts:783-786`）。stream_event 消息负责携带实时 token 用量——`message_start` 重置计数，`message_delta` 更新用量并捕获 stop_reason（`src/QueryEngine.ts:791-807`）。

在 `queryLoop` 的循环尾部（`src/query.ts:1679-1729`），工具结果收集完成后，计算 `nextTurnCount = turnCount + 1`，检查 `maxTurns`，然后构建 `next` State 对象，将本轮所有消息（`messagesForQuery + assistantMessages + toolResults`）合并为下一轮的输入，`state = next`，`while (true)` 进入新一轮。

这条路径验证了三件事：mutableMessages 在 `QueryEngine` 层积累所有消息（包括工具结果），queryLoop 在每轮结束时通过 `state = next` 传递状态，终止检查发生在"下一轮是否应该开始"的决策点，而非循环入口。

本章主成分：会话级状态机

本质：一个由三层守卫（maxTurns/maxBudgetUsd/blocking_limit）保护的 `while(true)` 状态机，每轮迭代完成一次"API 调用→工具执行→结果收集"，通过 `state = next` 传递跨迭代状态。

关键机制：

- `mutableMessages` 在 `QueryEngine` 层跨轮次积累，提供完整的消息历史
- `turnCount` 随每轮工具调用递增，是轮次守卫的计数基础
- transcript 在 query 循环前写入，确保进程被杀后 resume 可用
- `State` 对象集中管理 7 个 continue 点的跨迭代状态

适用边界：

- ✓ 适合：需要多轮工具调用的 Agent 系统
- ✓ 适合：需要 resume 恢复能力的长会话
- ✗ 不适合：单轮问答，无需循环管理开销
- ✗ 不适合：无状态 API 封装

与其他模式的关系：

- 启动层为 QueryEngine 准备运行环境（详见第 2 章）
- 单轮执行的完整流水线是每次迭代的内部细节（详见第 4 章）
- 压缩策略是循环的内存管理机制，与 blocking_limit 守卫协作（详见第 11 章）

你能做什么

- 审视你的 **Agent** 循环是否有至少三层终止守卫。轮次上限、费用上限、上下文阻塞——缺少任何一层都会留下失控漏洞。
- 在所有路径检查终止条件，包括 abort、超时、错误恢复路径。"正常路径有守卫"不等于"所有路径有守卫"。
- 在循环关键点前持久化状态。用户消息被接受这个事实，应该在 API 调用之前写入磁盘——不要等 API 响应。
- 用 **State** 对象模式替代散乱的状态变量。如果你的循环有多个 continue/break 点，用一个显式的状态对象让每个出口的状态更新变得可见。
- 为 **blocking_limit** 设计跳过条件，而不是简单的硬终止。在执行压缩期间触发 blocking_limit 会造成死锁——守卫必须知道何时不该触发。
- 区分交互模式和脚本模式的 **I/O** 策略。transcript 写入在交互模式下阻塞（安全优先），在脚本模式下 fire-and-forget（性能优先）。根据你的使用场景做同样的区分。
- 用 **`turnCount`** 追踪调试信息。当 Agent 行为异常时，"它循环了多少轮"是最有价值的第一诊断信息。

下一章导读：本章看到了循环的全貌，但每次迭代内部发生了什么？第 4 章将进入单轮执行的流水线——从提示词的分段构建，到流式 API 调用，到工具并发执行，到上下文窗口保护。这条流水线的每个节点都有独立的设计决策，共同决定了一轮调用的延迟、质量和安全性。

第 4 章：单轮执行的艺术

> "控制不是枷锁，是信任。每一道关卡都在说：'我相信你能完成任务，但我要确保你不会跑偏。'"

一次 API 调用在 Harness 眼里不是"发出去、等回来"。在消息离开进程之前，Harness 完成了提示词的分段缓存组装；在流式响应到达时，Harness 实时调度工具的并发执行；在流结束之后，Harness 触发后采样 Hooks 决定是否允许循环继续。四道关卡，每一道都编码了 Anthropic 对"模型什么时候会失控"的具体预判。读完本章，你将理解这四道关卡如何组成一条串行控制流水线，以及每道关卡的介入条件和权衡逻辑。

问题——一次 API 调用经历了什么

大多数 Agent 框架把 API 调用抽象为一个黑盒：构建消息列表，调用 API，处理返回结果。Claude Code 拒绝这个抽象。

在 `query.ts` 的循环体中，从消息列表准备好到 API 调用返回，中间经过了密度极高的控制逻辑。这些逻辑可以归纳为 4 层串行控制：

层次	触发时机	核心操作	失败时行为
第 1 层：提示词构建	API 调用前	三路并行组装→分段→加缓存控制	拒绝调用
第 2 层：自动压缩检查	API 调用前	检查 token 上限→按需压缩消息列表	替换消息列表后继续
第 3 层：流式工具分区执行	API 响应流中	并发/独占分区→实时执行工具→缓冲结果	丢弃旧执行器重试
第 4 层：后采样 Hooks	流结束后	触发外部 Hooks→判断是否允许循环继续	强制终止循环

每一层都有明确的输入和输出，层与层之间严格串行——前层未通过，后层不执行。这条流水线在每次工具调用循环的每一轮中都会完整执行一遍。

原则 **4.1**：控制流水线 — 对每次 API 调用施加的控制必须是分层的、串行的、可审计的。每层必须有明确的职责边界，层间禁止跨层直接通信，只通过输出状态传递控制权。

黄金法则——最小介入，精确控制

4 层控制听起来很重，实际上每一层的介入条件都经过精心设计——只在必要时才触发。

第 **1** 层介入条件：永远触发，但代价极低。提示词的三个组成部分（defaultSystemPrompt、userContext、systemContext）在每次 API 调用前并行构建，结果被合并并追加到系统提示词末尾。分段缓存控制只在模型支持时附加——`getPromptCachingEnabled` 检查全局禁用标志、模型级禁用标志，任一为真则跳过缓存。

第 **2** 层介入条件：只在上下文窗口接近上限时触发。如果 token 计数低于自动压缩阈值，这一层是透明的——消息列表原封不动进入第 3 层。只有当计数超过阈值时，自动压缩才介入，调用压缩 Agent 重写消息列表（详见第 11 章）。

第 **3** 层介入条件：只在助手消息包含工具调用（`stop_reason = tool_use`）时触发。纯文本回复完全跳过 `StreamingToolExecutor`。工具调用到达时，执行器判断是否可以立即开始——并发安全的工具可以与其他并发安全工具同时执行，非并发工具必须等待所有进行中的工具完成。

第 **4** 层介入条件：只在配置了后采样 Hooks 时触发。如果没有配置，`handleStopHooks` 是一个空操作，零开销。配置后，Hooks 可以执行外部命令、HTTP 请求或提示词注入，并决定是否允许循环继续。

原则 **4.2**：介入条件决定控制成本 — 每个控制层必须有精确的介入条件，在条件不满足时禁止主动干预，确保控制成本不取决于层数，而取决于实际触发频率。

适用场景——哪些系统需要 4 层控制

4 层控制不是"所有 Agent 都应该复制的架构"，而是针对特定场景设计的解决方案。

第 **1** 层（提示词构建）：所有调用 LLM 的系统都需要。提示词构建是最基础的 Harness 职责——它决定模型"看到"什么。如果你的提示词是硬编码的字符串，你已经在做第 1 层，只是没有结构化。

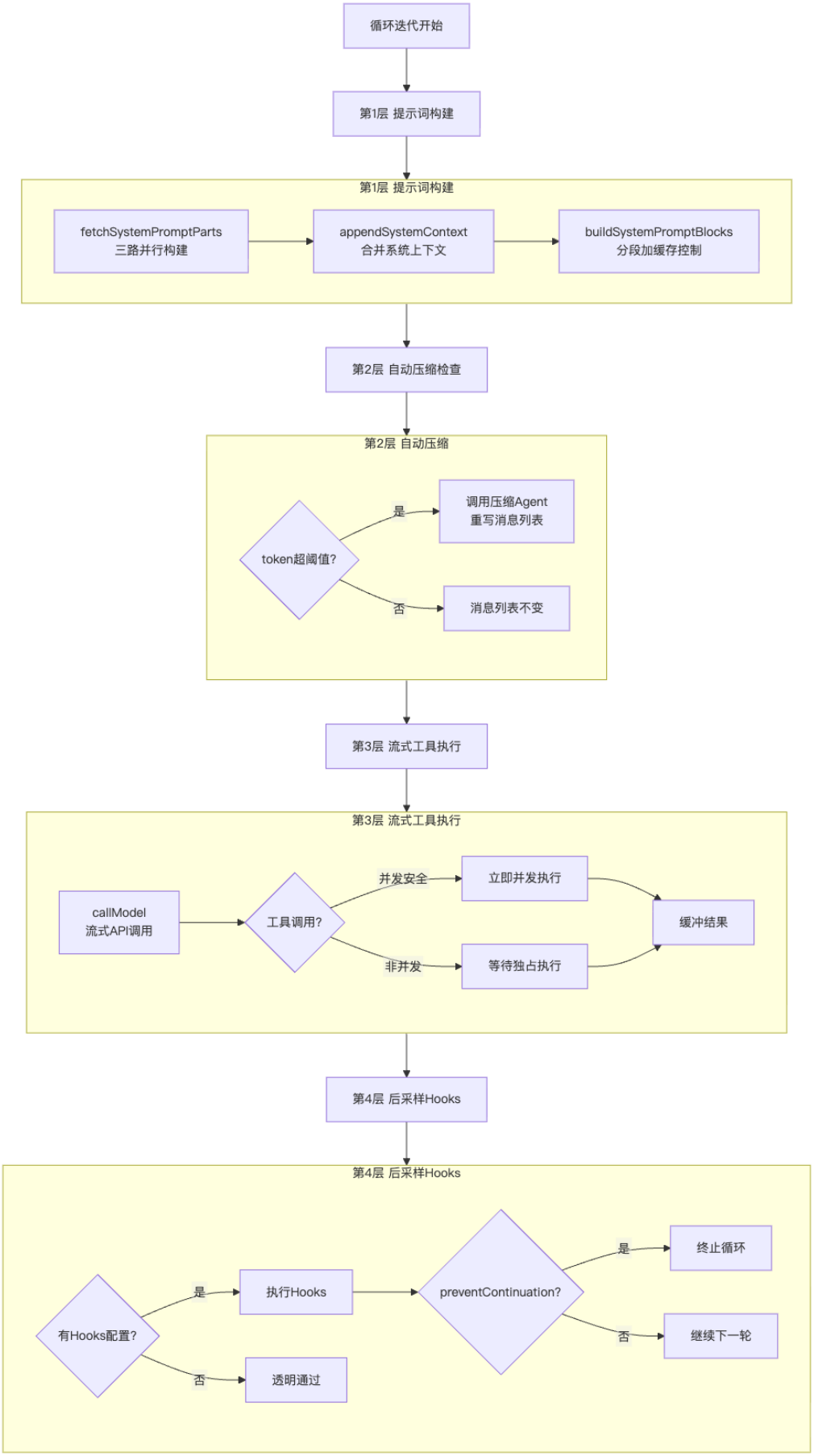
第 **2** 层（自动压缩）：所有可能产生长对话的 Agent 需要考虑。单次问答不需要；超过 10 轮的任务型 Agent 迟早需要面对上下文窗口限制的问题（详见第 11 章）。

第 **3** 层（流式工具分区）：需要同时执行多个工具的 Agent。如果你的 Agent 每次只调用一个工具，顺序执行已经足够。一旦你的任务适合并发（比如同时搜索多个数据源），工具分区带来的延迟收益就变得显著。

第 **4** 层（后采样 **Hooks**）：需要外部审计、安全检查或工作流集成的企业级系统。后采样 Hooks 是 Harness 向外部世界开放的唯一扩展点——它让你可以在不修改核心循环的情况下插入自定义逻辑（详见第 10 章）。

工作原理——4 层控制的完整流水线

图 **4-1**：单轮执行 **4** 层控制流水线


循环迭代开始
第1层 提示词构建
第1层 提示词构建
fetchSystemPromptParts
三路并行构建
appendSystemContext
合并系统上下文
buildSystemPromptBlocks
分段加缓存控制
第2层 自动压缩检查
第2层 自动压缩
token超阈值?
是
调用压缩Agent
重写消息列表
否
消息列表不变
第3层 流式工具执行
第3层 流式工具执行
callModel
流式API调用
工具调用?
并发安全
立即并发执行
非并发
等待独占执行
缓冲结果
第4层 后采样Hooks
第4层 后采样Hooks
有Hooks配置?
是
执行Hooks
否
透明通过
preventContinuation?
是
终止循环
否
继续下一轮

第 1 层：提示词构建

提示词的三个组成部分使用 `Promise.all` 并行获取——`defaultSystemPrompt`（工具定义和系统指令）、`userContext`（Claude.md 内容和日期）、`systemContext`（Git 状态和缓存破坏标记）三路同时请求，再合并。这三部分各自有独立的缓存策略，memoized 版本在会话内不变，重复调用直接命中内存缓存。

合并后，`appendSystemContext` 将 systemContext 追加到系统提示词末尾，产生 `fullSystemPrompt`。随后 `buildSystemPromptBlocks` 将系统提示词按缓存范围（cacheScope）分段，每段独立附加 `cache_control` 标记。分段策略遵循一条硬约束：最多 4 个缓存块——这是 API 的硬限制，超过会立即返回 400 错误。

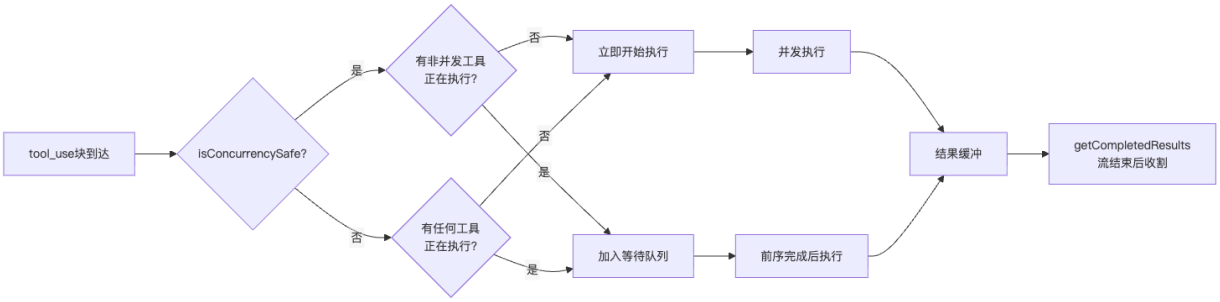
第 **2** 层：自动压缩检查

在 API 调用前，循环检查当前消息列表的 token 估算值是否超过自动压缩阈值。如果触发，`autocompact` 调用一个独立的压缩 Agent，将完整对话历史压缩为摘要，返回一个更短的消息列表替换原列表。压缩完成后，循环从检查守卫重新开始（而非直接进入 API 调用），确保压缩结果有效。

第 **3** 层：流式工具分区执行

这是 4 层中最精妙的设计。`StreamingToolExecutor` 在流式响应开始前初始化，流式传输期间实时检测每个 `tool_use` 内容块的到达，并立即决定执行策略。

图 **4-2**：**StreamingToolExecutor** 工具调度策略



注释对并发策略有精确描述：并发安全工具可以与其他并发安全工具并行执行，非并发工具需要独占访问。这保证了只读工具（文件读取、代码搜索）可以并发，而写操作工具（文件编辑、命令执行）串行执行。

流式响应结束时，`getCompletedResults` 收割所有已完成的工具结果，合并为 user 消息追加到消息列表。这条消息成为下一轮循环的输入。

如果 API 流中途失败触发流式回退（streaming fallback），`discard()` 方法立即将当前执行器标记为废弃——所有排队中的工具不再启动，进行中的工具收到合成错误。然后创建新的执行器重试，避免旧执行器的孤立 tool_result 污染新一轮的消息历史。

第 **4** 层：后采样 **Hooks**

流式传输完全结束、所有工具结果收集完毕后，`handleStopHooks` 触发。Hooks 收到完整的当前轮次上下文，可以执行三类操作：外部命令（shell 脚本）、HTTP 请求（外部服务）、提示词注入（向下一轮注入额外指令）。

Hooks 执行完成后，返回 `preventContinuation` 标志。如果为 `true`，循环立即返回，终止原因为 `'stop_hook_prevented'`，不再进入下一轮——无论模型是否还有工具要调用。

权衡——流水线设计中的 3 个关键选择

权衡一：**Prompt Cache** 分段上限（**4** 块）

API 允许最多 4 个带 `cache_control` 的文本块。Claude Code 的分段策略优先保护命中率最高的块：系统提示词的"全局"部分（工具定义、系统指令）使用 `cacheScope: 'global'`，几乎所有会话共享，命中率最高（推断）；用户上下文和动态内容放在后面，按需缓存。

注释「不得再添加更多缓存控制块，否则会触发 400 错误（原文："IMPORTANT: Do not add any more blocks for caching or you will get a 400"）」是一条踩坑记录——这个上限不是软警告，是 API 硬拒绝。分段策略必须在设计阶段就固定，而不是在运行时动态增减。

权衡二：流式工具并发 **vs** 顺序执行

顺序执行更简单——等第一个工具完成，再发第二个。并发执行更快，但需要管理执行状态、缓冲结果、处理部分失败。`StreamingToolExecutor` 选择了精确的中间路径：按并发安全性分区，而非全并发或全顺序。

这个选择的代价是 `StreamingToolExecutor` 本身的复杂度——它需要维护工具队列、执行状态跟踪、discard 机制、结果缓冲四套逻辑。好处是：只读工具（通常占绝大多数）（推断）全部并发，总执行时间接近最慢单个工具的时间，而非所有工具时间之和。

策略	延迟	实现复杂度	安全性
全顺序执行	最高（时间累加）	最低	最高
全并发执行	最低	中等	有风险（写操作竞争）
按并发安全性分区	接近最低	最高	高（写操作隔离）

权衡三：后采样 **Hooks** 的强制终止能力

`preventContinuation` 是一把双刃剑。它让外部系统可以终止任何循环——合规检查、安全审计、预算控制都可以通过 Hook 实现。但一旦配置错误，所有请求都会被终止，Agent 彻底失去工具调用能力，而错误原因藏在 Hooks 执行日志中，不易排查。

Claude Code 的设计选择是：第 4 层默认不存在（无 Hooks 配置时是空操作），强制终止能力需要显式激活。这遵循了第 1 章介绍的 fail-closed 原则的另一面——扩展点默认关闭，需要显式开启。

踩坑指南——流水线中的工程陷阱

陷阱一：缓存分段超过 **4** 块导致 **400**

系统提示词构建函数中的"IMPORTANT"注释（原文："IMPORTANT: Do not add any more blocks for caching or you will get a 400"，译：不得再添加更多缓存控制块，否则会触发 400 错误）不是礼貌提示，是血泪教训。Anthropic API 对带缓存控制的文本块有硬上限，超过立即返回 400 错误。这类错误不会出现在 API 错误消息中（它只说 "invalid_request"），需要在请求体中检查 `cache_control` 块的数量。

❌ 错误做法：在系统提示词中动态追加 `cache_control` 块，以为可以按需扩展。
✓ 正确做法：在设计阶段固定分段策略，确保带缓存控制的文本块总数 ≤ 4。测试时验证所有代码路径下的分段计数。

陷阱二：工具错误声明为并发安全

一个工具声明了 `isConcurrencySafe: true` 但实际上会修改共享状态——这不会在单工具测试中暴露，只在多工具并发调用时出现竞争条件。症状是随机的数据损坏或不一致的文件状态，极难复现。

❌ 错误做法：为了追求并发性能，对未经严格审查的写操作工具声明 `isConcurrencySafe: true`。
✓ 正确做法：保守默认（`TOOL_DEFAULTS` 中 `isConcurrencySafe` 默认 `false`，如第 1 章所述），只对经过严格审查的纯只读工具显式开启并发。

陷阱三：流式回退不调用 **discard()**

当 API 流中途失败需要重试时，如果忘记调用 `discard()` 丢弃旧的 `StreamingToolExecutor`，旧执行器中已开始执行的工具会产生 tool_result 消息，但对应的 tool_use 消息在重试时被替换了——API 在收到孤立的 tool_result 时会返回 "tool_result without matching tool_use" 错误。

❌ 错误做法：流式重试时复用旧的 `StreamingToolExecutor` 实例，或忽略 `discard()` 调用。
✓ 正确做法：在任何触发流式重试的路径上，先调用旧执行器的 `discard()` 标记废弃，再创建新执行器，确保 tool_use 与 tool_result 的对应关系在重试后保持一致。

实证——从源码追踪一次完整的单轮执行

一次单轮执行的完整路径可以通过 `queryCheckpoint` 调用串联起来。

第 1 层从 `fetchSystemPromptParts` 三路并行返回后开始（`src/utils/queryContext.ts:44`），`fullSystemPrompt` 在 `query.ts` 的组装步骤完成后确定（`src/query.ts:449`）。此时提示词已经是最终形态，包含分段缓存控制标记。

第 2 层紧随其后——`autocompact` 函数检查当前消息列表的 token 估算（`src/query.ts:454`），如果触发压缩，整个消息列表被替换后从检查守卫重新开始，否则直接穿透到第 3 层。

第 3 层从 `query_api_streaming_start` 打点后开始（`src/query.ts:658`），`callModel` 发起流式请求，`StreamingToolExecutor` 在请求前已初始化，开始实时处理流中的内容块。流结束时，`streamingToolExecutor.getCompletedResults()` 收割所有已完成工具的结果（`src/query.ts:851`），合并为工具结果列表。

第 4 层在流式结束后触发（`src/query.ts:1267`），`handleStopHooks` 接收当前会话上下文，执行配置的 Hooks，返回 `stopHookResult`。如果 `stopHookResult.preventContinuation` 为真（`src/query.ts:1278`），循环立即返回，单轮执行在此终止而非进入下一轮。

这条追踪路径验证了一个核心观察：4 层控制确实是严格串行的——每层的 checkpoint 打点之间没有交叉，层与层的边界清晰可测。

本章主成分：4 层串行控制流水线

本质：一条 4 层串行控制流水线，每层精确地只在必要时介入，共同保护上下文窗口不超限、工具调用不越界、循环不失控。

关键机制：

- 提示词三路并行构建后串行分段，最多 4 个缓存块
- `StreamingToolExecutor` 按并发安全性实时分区调度工具执行
- 后采样 Hooks 是循环外部世界唯一的扩展点，可强制终止

适用边界：

- ✓ 适合：需要工具调用、缓存优化、外部审计的 Agent 系统
- ✓ 适合：多工具并发执行场景
- ✗ 不适合：简单单轮问答（第 1 层足够）
- ✗ 不适合：无状态函数式调用（4 层控制面向有状态会话）

与其他模式的关系：

- 单轮执行是 QueryEngine 循环（第 3 章）的每轮迭代内部
- 提示词构建是第 5 章（系统提示词分层缓存）的执行层
- 工具分区执行是第 7 章（工具编排）的运行时实现
- 后采样 Hooks 是第 10 章（Hooks 系统）的执行入口

你能做什么

- 审视你的 **Agent** 每次 **API** 调用是否经过提示词最终组装层。硬编码的提示词字符串无法享受分段缓存，也无法动态注入系统上下文——这是第 1 层控制最直接的价值。
- 检查你的提示词缓存分段数量是否超过 **API** 上限。如果你使用 Anthropic API 的 `cache_control`，上限是 4 个带缓存标记的文本块。超过会触发 400 错误，而错误消息不会告诉你原因。
- 为你的工具建立并发安全分类，并在工具定义中显式声明。默认保守（不声明为并发安全），只对经过审查的只读工具开放并发。
- 为你的流式工具执行添加 **discard** 机制。任何需要重试的流式调用，都应该在重试前清理旧的执行状态，防止孤立 tool_result 污染后续调用。
- 把后采样 **Hooks** 当作审计点而非控制点。`preventContinuation` 很强大，也很危险——错误配置会静默地杀死所有循环。优先用 Hooks 收集数据，只在完全确定后才启用强制终止。
- 用 **queryCheckpoint** 式的时间线埋点定位延迟。4 层控制中，第 2 层（压缩）是最大的延迟来源，第 3 层（工具执行）是最大的并发优化空间。埋点是找到优化方向的前提。
- 评估你的 **Agent** 需要哪几层。不要为了架构完整性引入不需要的复杂度——只调用一个工具的 Agent 不需要 `StreamingToolExecutor`，没有审计需求的系统不需要后采样 Hooks。

下一章导读：本章看到了提示词构建的执行过程——三路并行、合并、分段加缓存控制。第 5 章将深入这条路径，展示系统提示词是如何通过分段缓存策略实现 Prompt Cache 最大化命中的，以及 Claude.md 记忆注入如何参与上下文构建。这是 Harness 最精妙的成本优化之一。

第三篇：上下文工程

上下文窗口是 **Harness** 最稀缺的资源。这一篇揭示 **Claude Code** 如何通过分层缓存、按需加载和智能压缩，让有限的窗口承载无限的信息需求。

第 5 章：提示词的分层构建

> "一段提示词的价值不在于它说了什么，而在于下次调用时它是否还在缓存中。"

系统提示词不是一整块文本——它是 20 多个 section 拼成的数组，每个 section 有独立的缓存语义。这个"多此一举"的拆分不是代码组织的偏好，而是一条精确计算的成本原则：静态内容在前、动态内容在后，每一个分段边界都是一个缓存命中率的决策点。读完本章，你将理解 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 这条分界线如何把系统提示词切成两个缓存世界，以及 MEMORY.md 的记忆注入为什么必须落在动态区。

问题——系统提示词为什么要拆成 20 多段

如果系统提示词在每次 API 调用时完全重建，Prompt Cache 命中率为零——每个 token 都要重新计费。一个典型会话中，系统提示词约占 40,000 个 token（推断）（包括工具定义、安全指令、环境信息、记忆内容等）。如果这些内容全部在缓存外，每轮调用的输入成本约为缓存命中的 2.5 倍（按 Anthropic API 定价，缓存命中的 token 单价约为未命中的 0.1 倍（推断））。

问题在于：系统提示词的某些部分在每次调用间不变（工具定义、系统指令），而某些部分每轮都可能变（用户上下文、MCP 工具列表、记忆文件）。如果把所有内容打包成一个字符串，缓存系统只能将整个字符串视为一个单元——任何一个字符变化，整个缓存失效。

Claude Code 的解决方案是 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY`。源码注释（原文："Boundary marker separating static (cross-org cacheable) content from dynamic content. Everything BEFORE this marker in the system prompt array can use scope: 'global'. Everything AFTER contains user/session-specific content and should not be cached."，译：分界标记，分隔静态（可跨组织缓存的）内容与动态内容。此标记之前的所有内容可使用全局缓存作用域，之后的所有内容包含用户/会话专属信息，不应缓存）定义了一条硬分界线：

区域	位置	缓存语义	典型内容
静态区	BOUNDARY 之前	`cacheScope: 'global'`	工具定义、系统指令、安全规则
动态区	BOUNDARY 之后	`cacheScope: null`（不缓存）	记忆文件、环境信息、会话指引、MCP 指令

这条分界线的位置不是随意的。源码中有一条 WARNING 注释（原文："WARNING: Do not remove or reorder this marker without updating cache logic in: src/utils/api.ts (splitSysPromptPrefix), src/services/api/claude.ts (buildSystemPromptBlocks)"，译：不得移除或重新排列此标记，除非同步更新 `splitSysPromptPrefix` 和

`buildSystemPromptBlocks` 中的缓存逻辑）——任何对分界线位置的改动都会影响缓存分段的正确性。

原则 **5.1**：缓存分段是成本结构的核心 — 系统提示词必须按变化频率分段，不变的内容使用全局缓存，可变的内容不缓存。分段边界禁止随意移动——每个边界位置都对应 API 缓存策略的精确计算。

黄金法则——静态在前、动态在后

Prompt Cache 的最大化命中率依赖一个简单原则：在同一会话内不变的内容必须始终出现在会话内可能变化的内容之前。

API 缓存不是自动的。Claude Code 在每次调用时通过 `splitSysPromptPrefix` 精确控制哪些块进缓存、用哪种缓存作用域。该函数的注释文档揭示了 4 种不同的缓存策略，每种对应一个场景：

模式	触发条件	缓存布局	块数
MCP 工具存在	`skipGlobalCacheForSystemPrompt = true`	归属头（null）+ 前缀（org）+ 其余（org）	3 块，全部 org 级
全局缓存 + 边界标记	1P 场景，边界标记存在	归属头（null）+ 前缀（null）+ 静态区（global）+ 动态区（null）	4 块
默认模式	3P 提供商或边界缺失	归属头（null）+ 前缀（org）+ 其余（org）	3 块，全部 org 级
缓存禁用	全局/模型级禁用	不分段，不附加 `cache_control`	0 块

原则 **5.2**：缓存作用域选择决定成本边界 — 全局缓存（global）必须只用于跨所有会话绝对不变的内容；组织级缓存（org）用于同一组织内不变的内容；动态内容禁止附加任何缓存控制。

为什么 MCP 工具存在时不能使用全局缓存？因为 MCP 服务器随时可能连接或断开——如果缓存了包含 MCP 工具指令的系统提示词，当 MCP 服务器断线后，模型仍然会尝试调用已经不存在的工具（"幽灵工具"问题）。所以在 MCP 工具存在时，整个系统提示词降级为 org 级缓存，牺牲命中率换取正确性（详见第 6 章）。

适用场景——哪些应用需要分段提示词

任何需要在 API 调用间复用大量相同系统提示词内容的应用，都应该考虑分段缓存策略。

适合分段的场景：会话内需要多轮调用的 Agent——每轮调用的系统提示词大部分相同，只有少量动态部分更新。典型例子是代码编辑 Agent，工具定义和安全规则在会话内不变，只有环境信息和记忆文件可能更新。

不需要分段的场景：单次调用就完成的任务（如一次性翻译、单轮问答）。这类场景没有缓存复用的机会，分段只会增加实现复杂度。

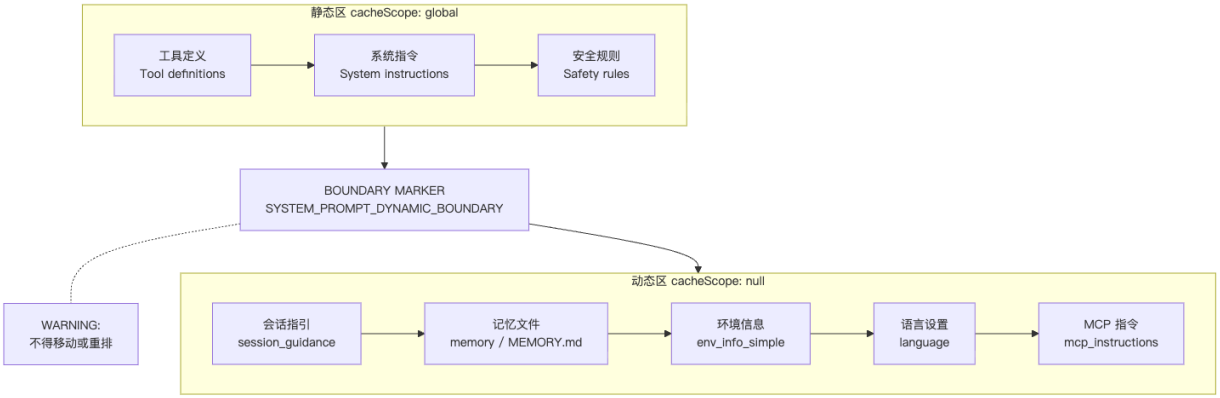
分段策略的前提是三路并行构建——系统提示词、用户上下文、系统上下文同时获取。`fetchSystemPromptParts` 通过 `Promise.all` 并行获取三个组成部分，各自独立缓存和更新。这种并行设计是分段策略的前提——如果三个部分串行构建，任何一个部分的延迟都会阻塞整条提示词管道。

应用特征	是否需要分段	理由
多轮对话 Agent，工具定义固定	✓	大量可缓存内容，分段后命中率显著提升
MCP 工具动态变化的 Agent	✓	分段后静态区仍可缓存，动态区正确降级
单轮问答服务	✗	无缓存复用机会
无状态函数式调用	✗	每次调用独立，无跨调用状态

工作原理——提示词构建的完整管道

系统提示词从原始 section 数组到 API 请求的最终分块，经过 5 步管道。每一步都有明确的输入输出和设计意图。

图 **5-1**：系统提示词分层结构



步骤 1：**section** 数组构建。`getSystemPrompt` 异步构建 20 多个 section，按静态/动态分区排列。静态区包含工具定义、系统指令、安全规则等固定内容。到达 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 后，进入动态区——`systemPromptSection('memory', () => loadMemoryPrompt())` 将 MEMORY.md 的内容作为一个动态 section 注入。

步骤 2：优先级决策。`buildEffectiveSystemPrompt` 在 5 个级别中选择提示词基础：override（强制覆盖）→ coordinator（协调器模式）→ agent（Agent 模式）→

custom（用户自定义）→ default（默认完整版）。这个决策决定了系统提示词的"骨架"——不同模式下，section 数组的内容和长度不同。

步骤 3：三路并行补充。`fetchSystemPromptParts` 通过 `Promise.all` 并行获取三个组成部分：`getSystemPrompt`（系统提示词本体）、`getUserContext`（Claude.md 和日期信息）、`getSystemContext`（Git 状态和缓存破坏标记）。三路的结果在 `src/query.ts` 中合并为 `fullSystemPrompt`（详见第 4 章）。

步骤 4：缓存语义分块。`splitSysPromptPrefix` 按缓存语义将系统提示词数组分块。全局缓存模式下，它沿 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 分割——边界之前的静态内容获得 `cacheScope: 'global'`，边界之后的动态内容 `cacheScope: null`。MCP 工具存在时，跳过全局缓存，全部降级为 org 级。

步骤 5：添加 **cache_control** 字段。`buildSystemPromptBlocks` 将每个分块转换为 API 的 `TextBlockParam` 格式，对 `cacheScope` 不为 null 的块附加 `cache_control` 字段。源码注释（原文："IMPORTANT: Do not add any more blocks for caching or you will get a 400"，译：不得再添加更多缓存控制块，否则会触发 400 错误）——这是 API 的硬上限，最多 4 个带 `cache_control` 的文本块。

记忆注入的位置值得特别注意。MEMORY.md 的内容通过 `systemPromptSection('memory', ...)` 注入到动态区——在 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 之后。这个位置是刻意选择的：记忆文件每次 mount 可能不同（用户可能编辑了 MEMORY.md），如果放在静态区，任何记忆变化都会使全局缓存失效。牺牲这个局部缓存，换取整个静态区的缓存稳定性。

权衡——分段策略的 3 个代价

分段缓存策略在成本效率上有明显优势，但也引入了 3 个工程代价。

决策维度	选择 **A**（本系统）	选择 **B**	核心权衡
缓存块数上限	最多 4 块，超出 400 报错	不限块数，自由分段	API 硬约束限制了分段的精细度——必须合并某些动态内容
MCP 指令缓存	`DANGEROUS_` 前缀标记不缓存	所有 section 统一缓存	正确性优先——缓存 MCP 指令会导致幽灵工具调用
记忆文件大小	40,000 字符上限，超出截断	不限大小	超长记忆文件占用的缓存空间超过命中率收益

代价一：缓存块数上限（**4** 块）

API 允许最多 4 个带 `cache_control` 的文本块。Claude Code 的策略是把所有动态内容合并为最后 1 块（不附加 `cache_control`），静态区最多占 3 块。全局缓存模式下：归

属头（null）+ 前缀（null）+ 静态区（global）+ 动态区（null）= 4 块，恰好达到上限。分段策略在设计阶段就必须固定——运行时动态增减会触发 400 错误。

代价二：**MCP** 动态内容的特殊处理

`DANGEROUS_uncachedSystemPromptSection` 是一个命名约定——`DANGEROUS_` 前缀标记那些绝对不能缓存的 section。MCP 工具指令使用这个标记，因为 MCP 服务器随时可能连接或断开。如果缓存了 MCP 指令，模型在服务器断线后仍会尝试调用已不存在的工具。`DANGEROUS_` 前缀是对开发者的警告：这里有一个你可能忽略的正确性风险。

代价三：记忆文件大小上限（**40,000** 字符）

`MAX_MEMORY_CHARACTER_COUNT = 40000` 是一个截断上限。超过此大小的记忆文件会被截断。这个限制不是随意的——超长记忆文件如果在每次调用中被完整注入动态区，会占用大量 token 预算，挤占工具调用和对话历史的空间。40,000 字符约等于 10,000 token（推断），对于一个典型会话的 200,000 token 上限来说，记忆文件最多占 5%。这是一个工程权衡——保留足够的记忆空间让 Agent 个性化，同时不至于让记忆挤占工作空间。

踩坑指南——提示词分段中的常见错误

陷阱一：把动态内容放到静态区（缓存 **bust**）

最常见也最隐蔽的错误。时间戳、用户 ID、会话 ID、随机 nonce——这些每轮都变的内容如果放在 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 之前，整个静态区的缓存立即失效。症状是 `cache_read_input_tokens` 始终为零，但代码逻辑看起来完全正确。

❌ 错误做法：在系统提示词中嵌入当前时间或会话 ID，放在 BOUNDARY 之前，以为这些"很短不会影响缓存"。
✓ 正确做法：所有每轮可能变化的内容必须放在 BOUNDARY 之后。如果不确定一个字段是否每轮变化，默认放在动态区。

源码中的 WARNING 注释明确列出了依赖这条分界线的两个函数——移动分界线时必须同步更新这些缓存逻辑。

陷阱二：添加第 **5** 个 **cache_control** 块

全局缓存模式下已经用了 4 块（归属头 + 前缀 + 静态区 + 动态区）。如果开发者新增一个 section 并给它附加 `cache_control`，第 5 个缓存块会立即触发 400 错误。更糟的是，这个错误不会告诉你"缓存块太多了"——它只说 `invalid_request`。

❌ 错误做法：新增 section 后给它附加 `cache_control`，忘记检查总块数。
✓ 正确做法：新增 section 时，确认当前缓存布局。如果 4 块已满，新 section 必须合并到已有的某个块中，不能独立分块。

陷阱三：对 **MCP** 工具使用全局缓存

MCP 工具连接/断开是运行时事件。如果 MCP 指令被全局缓存，断线后的工具列表仍然留在缓存中——模型会尝试调用不存在的工具，返回错误，然后重试，形成循环。

❌ 错误做法：MCP 工具指令使用 `systemPromptSection` 而非 `DANGEROUS_uncachedSystemPromptSection`，导致 MCP 指令被缓存。
✓ 正确做法：MCP 相关的 section 使用 `DANGEROUS_` 前缀标记，确保在任何缓存模式下都不被缓存。MCP 工具存在时，整个系统提示词降级为 org 级缓存。

实证——从 MEMORY.md 到 API 请求的完整路径

一个 MEMORY.md 文件从磁盘读取到最终出现在 API 请求中，经过了 4 次变换。这条路径揭示了记忆注入如何参与上下文构建。

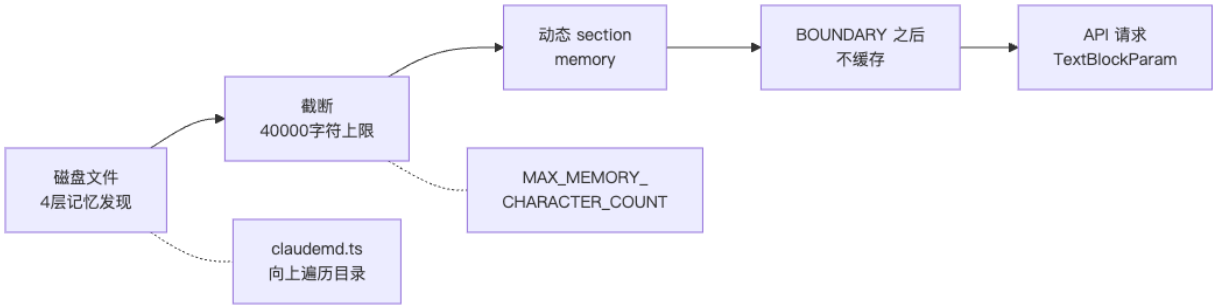
第一次变换：四层记忆发现。`claudemd.ts`（`src/utils/claudemd.ts:1`）定义了四层记忆体系——Managed memory（全局系统级）、User memory（用户主目录）、Project memory（项目目录中的 CLAUDE.md 及 .claude/rules/*.md）、Local memory（CLAUDE.local.md）。文件发现过程从当前目录向上遍历到根目录，越靠近当前目录的文件优先级越高（后加载覆盖先加载）。

第二次变换：大小截断。发现的记忆文件内容合并后，如果超过 `MAX_MEMORY_CHARACTER_COUNT = 40000`（`src/utils/claudemd.ts:92`），超出部分被截断。这个截断发生在 section 构建之前——截断后的内容作为 `systemPromptSection('memory', ...)` 的返回值（`src/constants/prompts.ts:495`）。

第三次变换：动态区注入。记忆 section 位于 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 之后，不附加 `cache_control`。这意味着记忆内容的变化不会影响静态区的缓存命中率——无论用户如何编辑 MEMORY.md，工具定义和安全规则的缓存始终有效。

第四次变换：三路并行合并。`fetchSystemPromptParts`（`src/utils/queryContext.ts:44`）中，`getUserContext` 负责处理 claudeMd 文件——它与 `getSystemPrompt` 和 `getSystemContext` 并行执行。合并后的完整系统提示词进入第 4 章描述的管道：`splitSysPromptPrefix` 分块 → `buildSystemPromptBlocks` 添加缓存控制 → API 调用。

图 **5-2**：**MEMORY.md** 到 **API** 请求的路径



这条路径验证了一个关键设计：记忆注入被刻意放在动态区，确保记忆的灵活性（随时编辑）不会破坏静态区（工具定义、系统指令）的缓存效果。这是"静态在前、动态在后"原则在记忆系统中的具体体现。

本章主成分：提示词分层构建

本质：把系统提示词按变化频率分段——不变的内容使用全局/组织级缓存，可变的内容不缓存。分段边界是成本决策点，不是代码组织偏好。

关键机制：

- `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 标记分界线，静态区前、动态区后
- `splitSysPromptPrefix` 按缓存语义分块，最多 4 块 `cache_control`
- `DANGEROUS_uncachedSystemPromptSection` 标记绝对不能缓存的动态内容
- MEMORY.md 注入动态区，`MAX_MEMORY_CHARACTER_COUNT = 40000` 截断上限
- `buildEffectiveSystemPrompt` 5 级优先级决策选择提示词骨架

适用边界：

- ✓ 适合：高频 API 调用的多轮 Agent（缓存命中率直接降低成本）
- ✓ 适合：MCP 工具动态变化的系统（正确降级策略）
- ✗ 不适合：单次调用的无状态服务（无缓存复用机会）
- ✗ 不适合：系统提示词极短（<1000 token）的场景（分段收益可忽略）

与其他模式的关系：

- 本章是第 4 章（单轮执行）提示词构建步骤的深入展开
- 第 6 章（按需加载的智慧）是动态内容管理的延伸——Skills 如何进一步减少动态区的体积
- 第 11 章（上下文的五把剪刀）涉及记忆文件在压缩中的处理

你能做什么

- 审视你的系统提示词，识别哪些内容在每次调用间不会变化。工具定义、安全规则、系统指令通常是静态的——把它们提取出来，作为缓存区的基础。

- 把静态内容放在动态内容之前，添加 **`cache_control`** 边界。确保 API 请求中的缓存块按变化频率排列——不变的最前，每轮变化的内容最后。
- 检查你是否超过了 **API** 的缓存块数上限。Anthropic API 最多 4 个带 `cache_control` 的文本块。如果你使用其他 API 提供商，确认其缓存块限制。
- 对需要动态更新的工具列表，使用不缓存标记。MCP 工具、动态加载的插件——任何可能在两次调用间变化的内容，都不应该被缓存，避免"幽灵工具"调用。
- 为你的记忆**/**上下文文件设置合理的大小上限。Claude Code 使用 40,000 字符上限——超过会被截断。测量你的 Agent 记忆文件的平均大小和增长速度，设定一个不挤占工作空间的上限。
- 测量优化前后的 **`cache_read_input_tokens`** 变化，验证分段策略的效果。如果没有缓存命中数据，你无法确认分段策略是否真正降低了成本。
- 学习第 **6** 章如何通过 **Skills** 懒加载进一步优化动态内容的管理，减少每次调用的提示词体积。

下一章导读：本章看到了系统提示词如何通过分段缓存策略最大化 Prompt Cache 命中率。但静态区与动态区之间的平衡点不是固定的——动态区中有些内容可以按需加载而非每次注入。第 6 章将展示 Skills 系统如何通过懒加载和指令压缩，进一步减少动态区的体积，让更多内容留在缓存命中的静态区。

第 6 章：按需加载的智慧

> "把所有东西都放进上下文，和让模型自己决定需要什么——这两种策略的 token 成本可以相差 50 倍。"

Claude Code 有 50 多个技能（Skills），但系统提示词中只暴露它们的名称和摘要——全文在模型决定使用时才拉取。这个"让模型自己管理上下文窗口"的设计，不是延迟加载的性能技巧，而是 Harness 控制每次 API 调用 token 预算的核心策略。读完本章，你将理解三层懒加载架构——发现、决策、执行——如何让技能库的元信息只占用上下文窗口的 1%。

问题——50+ 技能如何在上下文中"不占位"

如果把 50 多个技能的完整提示词全部注入系统提示词，每个技能平均 400-800 字符，总计 20,000-40,000 字符（约 5,000-10,000 token）（推断）。在 200,000 token 的上下文窗口中，技能提示词一项就占去 2.5%-5%——留给工具定义、对话历史和工作空间的内容被大幅压缩。

Claude Code 的解法是一个反直觉的设计：只注入元数据摘要，不注入全文。系统提示词中，技能列表占用上下文窗口的 1%（`SKILL_BUDGET_CONTEXT_PERCENT = 0.01`），默认 8,000 字符（`DEFAULT_CHAR_BUDGET = 8_000`，计算方式：200k token × 4 字符/token × 1%）。每条技能的描述截断为 250 字符（`MAX_LISTING_DESC_CHARS = 250`），确保 50 个技能的摘要列表控制在预算内。

策略	技能列表 **token** 消耗	上下文占比	留给工作空间的 **token**
全文注入	~5,000-10,000	2.5%-5%	减少 5,000-10,000
元数据摘要（1% 预算）	~2,000	1%	仅减少 2,000
不注入任何技能信息	0	0%	模型无法发现和使用技能

注释写出了精确的计算逻辑："Skill listing gets 1% of the context window (in characters)"（译：技能列表占用上下文窗口的 1%，以字符为单位）。8,000 字符不是随意设定的数字，而是从上下文窗口大小推导的精确预算。`MAX_LISTING_DESC_CHARS` 的注释进一步解释："The listing is for discovery only — the Skill tool loads full content on invoke, so verbose whenToUse strings waste turn-1 cache_creation tokens without improving match rate."（译：列表仅用于发现——Skill 工具在调用时加载完整内容，冗长的 whenToUse 字符串会浪费第一轮的缓存创建 token，却不能提高匹配率）。

原则 **6.1**：预算驱动的上下文分配 — 每种上下文内容必须有明确的 token 预算上限。技能列表禁止超过上下文窗口的 1%——超出预算的信息在发现阶段被截断。

黄金法则——让模型决定何时拉取全文

Skills 懒加载的核心原则是"发现时只给名称和摘要，调用时才给全文"——把上下文管理的决策权从 Harness 转交给模型。

`DiscoverSkillsTool` 只做一件事：告诉模型当前有哪些技能可用（`DISCOVER_SKILLS_PROMPT = 'Discover skills'`）。这个"无操作"的发现阶段是懒加载的关键——模型在需要时调用发现工具，获得技能列表，然后根据任务需求决定是否调用具体技能。

`estimateSkillFrontmatterTokens` 函数只统计技能的 frontmatter 元数据（name + description + whenToUse），而非全文。注释说明了原因："since full content is only loaded on invocation"（译：因为完整内容只在调用时加载）。`FRONTMATTER_REGEX` 正则解析 YAML frontmatter，提取元数据用于发现阶段的摘要生成。

维度	预加载（所有技能全文）	懒加载（先发现后调用）
首次调用 token 成本	极高（全部技能全文）	极低（仅 1% 预算的摘要）
调用技能时延迟	零（已在上下文中）	一轮 API 调用（先发现后执行）
缓存命中率	低（技能内容变化频繁）	高（摘要列表稳定可缓存）
上下文窗口利用率	差（大量不相关技能占位）	好（只加载需要的技能）

原则 **6.2**：模型即上下文管理者 — Harness 禁止主动决定哪些技能内容进入上下文——只提供摘要，让模型根据任务需求自行判断。模型是唯一有权决定"我需要什么信息"的参与者。

适用场景——哪些功能适合懒加载

任何"存在但不常用"的能力都适合懒加载。核心判断标准是：每次任务中，有多少比例的技能实际被使用？

适合懒加载：

- 技能库超过 20 个，每次任务只用 1-3 个——典型场景是代码编辑 Agent，拥有代码分析、重构、测试、文档等多种技能，但单个任务只需其中一两种
- 技能内容较长（每个 400 字符以上），全文注入会显著挤占上下文
- 技能列表在不同会话间变化不大——摘要可以缓存，提高 Prompt Cache 命中率

不适合懒加载：

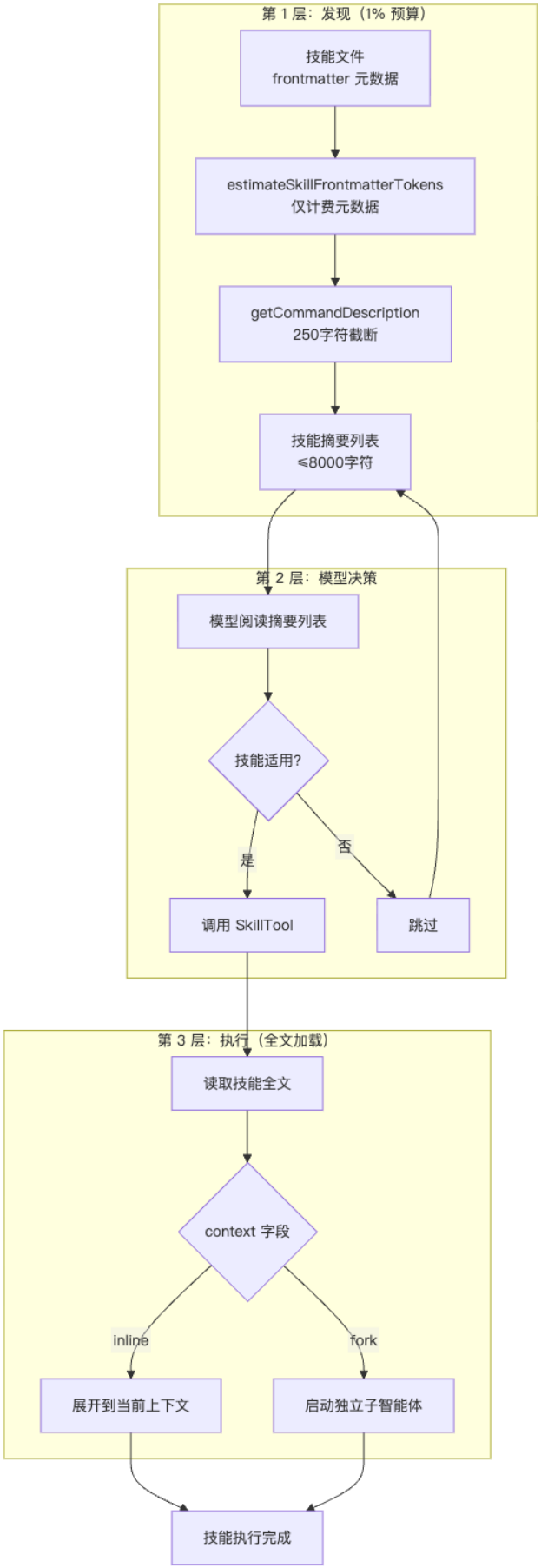
- 核心工具（文件读写、代码搜索）——每次任务几乎都用，懒加载反而增加延迟
- 极短的内容（<50 字符）——懒加载的管理开销超过节省的 token

`BundledSkillDefinition` 类型中 `userInvocable` 字段区分了两类技能：用户可直接调用的（需要在发现阶段显示）和仅供内部使用的（不需要在列表中暴露）。这个区分确保了发现阶段的列表只包含对用户有价值的信息。

工作原理——三层架构的完整流程

Skills 懒加载不是一个工具，是三个协作模块构成的完整系统：发现层、决策层、执行层。

图 **6-1**：**Skills** 懒加载三层架构


第 1 层：发现（1% 预算）
技能文件
frontmatter 元数据
estimateSkillFrontmatterTokens
仅计费元数据
getCommandDescription
250字符截断
技能摘要列表
≤8000字符
第 2 层：模型决策
模型阅读摘要列表
技能适用?
是
否
调用 SkillTool
跳过
第 3 层：执行（全文加载）
读取技能全文
context 字段
inline
fork
展开到当前上下文
启动独立子智能体
技能执行完成

发现层：`DiscoverSkillsTool` 在模型需要时被调用，返回技能摘要列表。列表的生成经过三步过滤：首先 `estimateSkillFrontmatterTokens` 只提取每个技能的 name、description、whenToUse 三个字段；然后 `getCommandDescription` 将 description 和 whenToUse 拼接为完整描述；最后截断到 250 字符。整个列表控制在 8,000 字符预算内。

决策层：模型阅读摘要列表后，根据任务需求判断哪个技能适用。这个决策完全由模型做出——Harness 不干预。如果模型判断需要某个技能，调用 `SkillTool` 并传入技能名称和参数。

执行层：`SkillTool` 调用时才读取技能全文。根据技能的 `context` 字段选择执行模式：`inline`（技能内容展开到当前上下文中）适合需要访问当前会话状态的轻量技能；`fork`（启动独立子智能体执行）适合需要隔离上下文的重型任务。fork 模式的详细设计见第 12 章（隔离与交接）。

`getAllCommands` 函数合并了所有技能来源——本地技能和 MCP 技能。注释解释了过滤逻辑："Only include MCP skills (loadedFrom === 'mcp'), not plain MCP prompts"（译：只包含 MCP 技能，不包含普通 MCP prompts）——这是为了防止模型猜测 MCP prompt 名称绕过发现阶段。

权衡——三层架构的设计代价

三层懒加载在 token 效率上有显著优势，但引入了 3 个工程代价。

决策维度	选择 **A**（本系统）	选择 **B**	核心权衡
调用模式	两步调用（先发现后执行）	一步直接调用	多一轮 API 调用，换取上下文窗口的 95% 以上节省
描述精度	250 字符截断	不限长度	短摘要可能降低发现精度，但防止列表超出预算
MCP 技能管理	过滤非 MCP skill 的 prompts	允许所有 MCP 内容	增加约束复杂度，但防止绕过发现阶段的安全管控

代价一：两步调用增加延迟

模型需要先调用 `DiscoverSkillsTool` 获得技能列表，再调用 `SkillTool` 执行——这增加了一轮 API 调用。但代价是可接受的：技能列表在第一次发现后留在模型的上下文中，后续调用无需重复发现。在多轮对话中，首次发现的固定成本被后续所有技能调用分摊。

代价二：描述截断降低发现精度

250 字符的截断上限意味着复杂的技能用途可能无法在摘要中完整表达。模型可能因为描述不够精确而跳过适用的技能，或调用了不适用的技能——这是懒加载固有的精确度/效率权衡。缓解方法是让 `whenToUse` 字段聚焦于触发场景而非功能描述。

代价三：**MCP** 过滤增加约束复杂度

`getAllCommands` 中的 MCP 过滤逻辑防止模型通过猜测 prompt 名称直接调用——这确保所有技能调用都经过发现阶段。但这也意味着 MCP 提供商需要按照 BundledSkillDefinition 的格式注册技能，增加了集成复杂度。

踩坑指南——懒加载中的常见错误

陷阱一：**frontmatter** 格式不规范

`FRONTMATTER_REGEX` 要求严格的 `---\n` 开头和结尾。常见的错误包括：`---` 后缺少换行符、使用 `~~~` 代替 `---`、frontmatter 块内有未转义的冒号。解析失败的结果是整个技能文件被当作正文处理——技能不会出现在发现列表中，模型无法调用。

❌ 错误做法：在技能文件中随意使用 YAML 格式，不验证 frontmatter 是否被正确解析。
✓ 正确做法：使用严格的 `---\n` 格式包裹 frontmatter，提交前用 `FRONTMATTER_REGEX` 验证解析结果。

陷阱二：**whenToUse** 描述过于泛化

"完成各种任务"或"帮助用户解决问题"——这类泛化的 whenToUse 在截断到 250 字符后，无法帮助模型判断技能是否适用。模型面对 50 个"帮助用户"的摘要，只能猜测。

❌ 错误做法：whenToUse 写成功能简介——"这个技能用于代码分析和重构"。
✓ 正确做法：whenToUse 写成触发场景——"当用户要求分析项目架构、重构超过 50 行的函数、或解释复杂类型推导时使用"。

陷阱三：**description** 和 **whenToUse** 的信息重叠

`getCommandDescription` 会把 description 和 whenToUse 拼接为一条摘要。如果两者内容重叠，250 字符中会有大量冗余，导致有效信息被截断。

❌ 错误做法：description 写"代码搜索工具"，whenToUse 也写"用于搜索代码"。
✓ 正确做法：description 写"代码搜索工具"，whenToUse 写"当用户要求查找某个函数的所有调用点、搜索特定模式的代码、或定位变量定义时使用"。

实证——从 Skills 列表到技能执行的完整路径

一个 bundled skill 从代码注册到模型调用，经过 4 个阶段。这条路径验证了三层懒加载架构的实际运作。

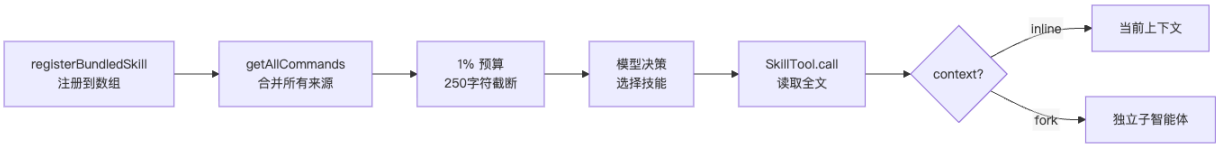
注册阶段：`registerBundledSkill`（`src/skills/bundledSkills.ts:47`）将技能定义注册到内部数组 `bundledSkills`。注册时，如果技能有附属文件（`files` 字段），这些文件会在首次调用时按需解压到磁盘——不是启动时解压，而是调用时。

发现阶段：模型调用 `DiscoverSkillsTool`，触发 `getAllCommands`（`src/tools/SkillTool/SkillTool.ts:81`）合并所有来源的技能列表。每个技能的元数据通过 `estimateSkillFrontmatterTokens`（`src/skills/loadSkillsDir.ts:100`）估算 token 数，然后截断到 250 字符（`src/tools/SkillTool/prompt.ts:29`），总列表控制在 8,000 字符预算内（`src/tools/SkillTool/prompt.ts:21`）。

决策阶段：模型阅读摘要列表，选择适用的技能。这个决策完全由模型做出——Harness 不干预。模型传入技能名称和参数调用 `SkillTool`。

执行阶段：`SkillTool` 读取技能全文。如果技能声明了 `context: 'fork'`，通过 `executeForkedSkill`（`src/tools/SkillTool/SkillTool.ts:119`）启动独立子智能体执行。否则以 inline 模式将技能内容展开到当前上下文。

图 **6-2**：**bundled skill** 的注册→发现→选择→执行路径



这条路径验证了核心设计：技能全文只在调用时加载——注册时只存定义，发现时只暴露摘要，执行时才读取完整内容。三个阶段各自的 token 成本递减：注册零成本（内存数组），发现 1% 预算，执行仅当前需要的技能。

本章主成分：Skills 懒加载

本质：把上下文管理的决策权从 Harness 转交给模型——Harness 只维护 1% 的发现层，模型自己决定何时加载全文。

关键机制：

- 1% 上下文预算（8,000 字符默认值），每条技能截断到 250 字符
- `FRONTMATTER_REGEX` 解析 YAML 元数据，`estimateSkillFrontmatterTokens` 只计费元数据
- `DiscoverSkillsTool` 发现 + `SkillTool` 执行的两步调用模式
- `context: 'inline' | 'fork'` 执行模式选择

适用边界：

- ✓ 适合：技能库超过 20 个，每次任务只用少数几个
- ✓ 适合：技能内容较长，全文注入会挤占上下文
- ✗ 不适合：每次任务必用的核心工具（应预加载）
- ✗ 不适合：极短内容（懒加载管理开销超过节省）

与其他模式的关系：

- 本章是第 5 章（提示词分层构建）动态区管理的实例——技能摘要列表属于动态区内容
- 第 7 章（最小工具基座）的工具注册采用了类似的按需声明策略
- 第 12 章（隔离与交接）详细展开 fork 执行模式的上下文隔离设计

你能做什么

- 为你的技能/工具库建立发现层。只暴露名称和简短描述，不暴露全文。计算上下文预算的 1%，以此为发现层的 token 上限。
- 为每个技能写精确的 **whenToUse** 字段。不超过 200 字符，聚焦于触发场景而非功能描述——确保模型能从摘要判断技能适用性。
- 检查技能 **frontmatter** 格式是否符合严格的 `---\n` 规范。解析失败意味着技能不会出现在发现列表中，模型无法调用。
- 为需要隔离上下文的重型技能设置 `context: 'fork'`。fork 模式启动独立子智能体，防止技能执行的副作用污染当前会话（详见第 12 章）。
- 对照第 **5** 章的缓存策略，确保技能元数据列表在静态区。技能摘要列表在会话内不变，应该利用 Prompt Cache——如果你的技能列表每轮都变，说明摘要中有动态内容，需要排除。
- 评估你的技能库大小。超过 20 个时特别需要懒加载策略；超过 50 个时，1% 预算可能不够——考虑对技能分组，按任务类型分批发现。
- 测量技能发现和执行的 **token** 消耗。验证发现阶段的实际消耗是否在预算内，执行阶段的全文加载是否值得。

下一章导读：本章看到了 Skills 如何通过懒加载控制上下文窗口的技能列表占用。但技能只是 Harness 工具系统的一部分——更基础的问题是：哪些工具应该存在，它们如何注册、分类、分组？第 7 章将进入工具注册的最小基座设计，展示 Harness 如何用"最小工具集 + 按需扩展"的策略平衡能力与安全。

第四篇：工具编排

工具是 **Harness** 的手和脚。从统一接口工厂到并发安全策略，从最小权限到规划式执行——这一篇展示工具编排如何让模型安全而有效地与外部世界交互。

第 7 章：最小工具基座

> "最小不是最少，是最精。一套简单的接口，撑起了 54 个工具，也撑起了无数第三方扩展。"

Claude Code 有 50 多个工具——文件读写、代码搜索、命令执行、权限检查——但它们共用一套 7 个字段的接口。`buildTool` 工厂把所有工具约束到同一张行为合同上，`partitionToolCalls` 按一条规则决定并发还是串行。复杂度不在基座里，在 MCP 生态里。读完本章，你将理解"最小工具基座"的含义——核心接口越简单，扩展能力越强。

问题——50+ 工具如何共享一套行为约定

50 个工具如果各自实现安全检查、并发控制、权限验证，出错概率是单工具的 50 倍。更危险的是，一个工具的遗漏声明会破坏整个调度系统的安全假设——如果一个声明为并发安全的工具实际上修改了共享文件，所有与之并发执行的工具都会遭遇竞争条件。

Claude Code 的解决方案是 `buildTool` 工厂。它定义了 7 个可默认字段（DefaultableToolKeys），每个工具通过工厂获得统一的安全默认值，开发者只需覆盖与默认值不同的字段。源码注释写明了这个设计的核心意图："Build a complete Tool from a partial definition, filling in safe defaults for the commonly-stubbed methods. All tool exports should go through this so that defaults live in one place."（译：从部分定义构建完整的 Tool，为常用桩方法填充安全默认值。所有工具导出都应经过此函数，确保默认值集中在一处）。

7 个可默认字段分两类：

类别	字段	默认值	作用
行为类	`isEnabled`	`true`	工具是否可用
行为类	`isConcurrencySafe`	`false`（假定不安全）	是否可与其他工具并发
行为类	`isReadOnly`	`false`（假定写操作）	是否只读
行为类	`isDestructive`	`false`	是否有破坏性
控制类	`checkPermissions`	`allow`（交由通用权限系统）	权限检查逻辑
控制类	`toAutoClassifierInput`	`''`（跳过分类器）	AI 安全分类器输入
控制类	`userFacingName`	工具名	面向用户的显示名

原则 **7.1**：集中默认值，分散覆盖 — 工具安全默认值必须集中在一处（`TOOL_DEFAULTS`），所有工具通过 `buildTool` 工厂继承。开发者禁止手动设置默认值——只覆盖与默认不同的字段。

黄金法则——fail-closed 默认值策略

`buildTool` 的 7 个默认值全部选择 fail-closed（保守安全）方向。源码注释写明了这个原则："Defaults (fail-closed where it matters): isConcurrencySafe → false (assume not safe), isReadOnly → false (assume writes)."（译：默认值在关键处采用失败关闭：isConcurrencySafe 默认 false，假定不安全；isReadOnly 默认 false，假定写操作）。

三个关键默认值的工程逻辑：

`isConcurrencySafe → false`：除非工具开发者显式声明为并发安全，否则工具不可与其他工具同时执行。这意味着新工具默认串行——即使开发者忘记声明，也不会导致并发竞争。

`isReadOnly → false`：假定工具会修改状态。在权限系统中，写操作工具需要更严格的审查（详见第 9 章）。默认为写操作确保新工具不会被意外绕过写操作权限检查。

`toAutoClassifierInput → ''`：AI 安全分类器收不到这个工具的描述，无法自动判断风险等级。注释明确标注："security-relevant tools must override"（译：安全相关工具必须覆盖）。代价是遗漏覆盖的工具会被分类器跳过，好处是分类器不会被无关工具的信息淹没。

默认策略	对 **50+** 个工具的影响	如果选 **fail-open** 的风险
fail-closed（当前）	开发者需显式声明安全性	无风险——遗漏声明=最安全行为
fail-open	无需声明，默认允许	遗漏声明=最危险行为——并发竞争、权限绕过

原则 **7.2**：安全默认值优先于便利性 — 工具默认值必须选择最安全的方向——`false` 优先于 `true`。开发者为了便利必须显式覆盖默认值，这个"不便"是安全的代价。

适用场景——哪些系统需要工具基座

任何 Agent 系统只要有 2 个以上需要不同权限的工具，就需要统一的工具接口。

需要工具基座：工具类型多样（读/写/破坏性），来源多样（内置/第三方/协议扩展），需要统一的并发控制和权限检查。典型场景是代码编辑 Agent——同时拥有文件读取（安全、可并发）、文件编辑（不安全、需串行）、命令执行（破坏性、需确认）三类工具。

不需要工具基座：只有 1-2 个工具的简单 Agent，或者所有工具行为完全相同的系统。如果每个工具都是"接收输入、返回输出"的无状态函数，统一接口的收益可忽略。

Tool interface 的 `isMcp`、`isLsp`、`shouldDefer`、`alwaysLoad` 字段揭示了基座的扩展性设计——核心接口不变，通过这些标记字段区分不同来源的工具：

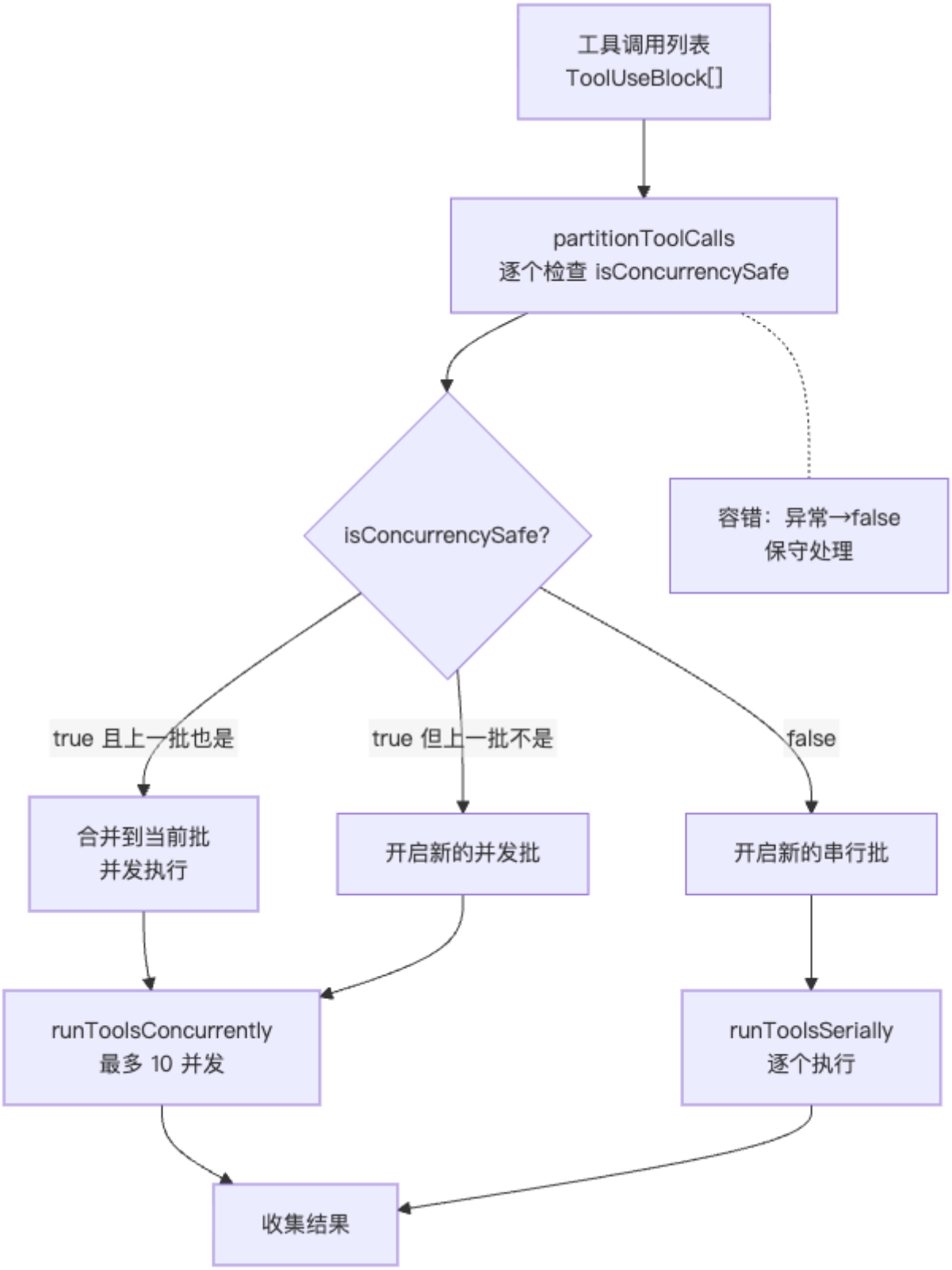
工具来源	标记字段	注册方式	调度方式
内置工具	默认	`buildTool()` 工厂	与所有工具统一调度
MCP 工具	`isMcp: true`	对象展开复用基座	同上
LSP 工具	`isLsp`	延迟加载	需要时按需启动

工作原理——只读并发、写操作串行

`toolOrchestration.ts` 实现了一个简单但有效的调度策略：把所有工具调用分批——连续的并发安全工具合并为并发批次，其余串行执行。

源码注释精确定义了分批规则："Partition tool calls into batches where each batch is either: 1. A single non-read-only tool, or 2. Multiple consecutive read-only tools."（译：将工具调用分为批次，每个批次要么是一个非只读工具，要么是多个连续的只读工具）。

图 **7-1**：工具调度分批策略



分批逻辑的执行过程：

`partitionToolCalls` 用 reduce 逐个处理工具调用。对每个工具，先通过 `findToolByName` 查找工具定义，再用 `inputSchema.safeParse` 解析输入参数，然后调用 `isConcurrencySafe(parsedInput.data)` 判断并发安全性。如果返回 `true` 且上一个批次也是并发安全的，合并到当前批次；否则开启新批次。

容错逻辑是设计的关键。`isConcurrencySafe` 调用被 try-catch 包裹——如果抛出异常（例如 shell-quote 解析失败），保守处理为 `false`（不可并发）。源码注释说明了意图："If isConcurrencySafe throws, treat as not concurrency-safe to be conservative."（译：如果 isConcurrencySafe 抛出异常，保守地视为不可并发）。

分批完成后，`runTools` 对每个批次选择执行策略：并发安全批次调用 `runToolsConcurrently`（最多 `getMaxToolUseConcurrency` 个并发，默认 10，可通过环境变量 `CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY` 覆盖），非并发安全批次调用 `runToolsSerially`（逐个执行）。

这个调度策略是贪心的——它只合并"连续"的并发安全工具，不做全局依赖分析。例如，如果工具调用序列是 [读, 写, 读]，两个读操作不会被合并，因为中间夹了一个写操作。代价是偶尔无法识别非连续的可并发机会；收益是 O(n) 复杂度、无依赖分析开销。

权衡——工具基座的 3 个设计选择

决策维度	选择 **A**（本系统）	选择 **B**	核心权衡
MCP 工具复用	对象展开继承基座	MCP 工具独立实现接口	统一调度逻辑 vs MCP 工具必须适配内部接口
并发安全声明	readOnlyHint 自动映射	第三方工具自行声明 isConcurrencySafe	降低接入门槛 vs 映射精度依赖 MCP 注解质量
异常处理	保守处理为不可并发	让异常传播到调用方	调度层健壮性 vs 可能损失并发机会

选择一：**MCP 工具通过对象展开复用内置基座**

MCP 工具不是另起一套接口——它通过 `...MCPTool` 展开运算符继承内置工具基座的所有默认值和行为，然后覆盖 name、description、isConcurrencySafe 等字段。这意味着 `partitionToolCalls` 不需要区分内置工具和 MCP 工具——它们走同一条调度路径。代价是 MCP 工具必须适配 Tool interface 的 7 个字段结构。

选择二：**readOnlyHint 自动映射到 isConcurrencySafe**

MCP 协议定义了 `readOnlyHint` 注解——MCP 工具只需声明自己是否只读，Claude Code 自动将 `readOnlyHint` 映射为 `isConcurrencySafe`。第三方工具开发者无需理解 Claude Code 的内部调度机制，只需遵循 MCP 协议标准。如果 MCP 工具没有设置 `readOnlyHint`，默认 `false`——又是 fail-closed。

选择三：**isConcurrencySafe 异常保守处理**

调度层不信任工具的 `isConcurrencySafe` 实现——如果它抛出异常，调度层不传播异常，而是静默降级为串行执行。这是一个防御性设计：一个工具实现的 bug 不应该导致整个调度层崩溃。代价是偶尔损失并发机会，收益是调度层的健壮性。

踩坑指南——工具基座中的常见错误

陷阱一：错误声明 isConcurrencySafe

一个工具声明了 `isConcurrencySafe: true` 但实际上会修改共享状态——不会在单工具测试中暴露，只在多工具并发调用时出现随机竞争条件。

❌ 错误做法：为了并发性能，对"大多数时候只读"的工具声明 `isConcurrencySafe: true`。
✓ 正确做法：只对经过严格审查的纯只读工具声明 `isConcurrencySafe: true`。如果有任何修改状态的可能，保持默认 `false`。

陷阱二：忘记声明 isDestructive

`isDestructive` 的默认值是 `false`。如果删除文件或执行不可逆操作的工具没有声明 `isDestructive: true`，权限系统不会触发额外的确认步骤——用户可能在不知情的情况下批准了不可逆操作。

❌ 错误做法：只关注 isConcurrencySafe，忽略 isDestructive 声明。
✓ 正确做法：对每个工具逐一检查——如果有任何修改、删除、覆盖操作，声明 `isDestructive: true`。

陷阱三：安全相关工具遗漏 toAutoClassifierInput

`toAutoClassifierInput` 默认返回空字符串，意味着 AI 安全分类器跳过这个工具。如果安全敏感工具（如命令执行、网络请求）遗漏了这个声明，分类器无法评估风险等级（详见第 9 章）。

❌ 错误做法：只在工具的 description 里写"谨慎使用"，不实现 toAutoClassifierInput。
✓ 正确做法：安全相关工具必须实现 `toAutoClassifierInput`，返回描述工具用途和风险的关键词字符串。

实证——MCP 工具如何复用内置基座

一个 MCP 工具从服务器注册到被调度执行，完全走内置工具的相同路径。这条路径验证了"最小基座"的扩展能力。

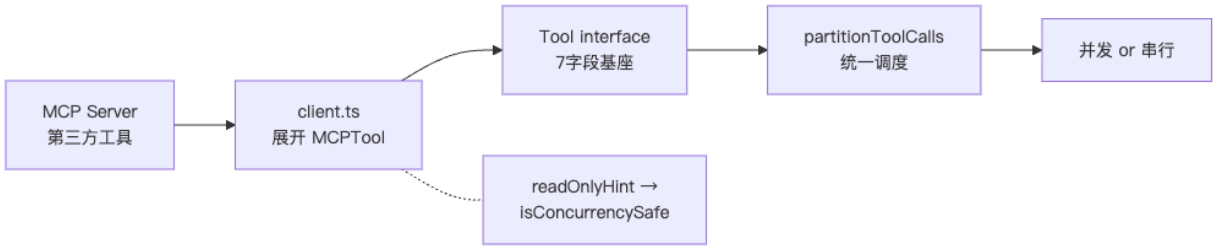
注册阶段：MCP 客户端连接到 MCP 服务器后，获取工具列表。每个 MCP 工具通过对象展开继承基座——`...MCPTool`（`src/services/mcp/client.ts:1770`）将内置工具的所有

默认值复制过来，然后覆盖特定字段：`isMcp: true`（`src/tools/MCPTool/MCPTool.ts:28`）标记为 MCP 工具，`mcpInfo` 附加服务器名和工具名身份信息（`src/services/mcp/client.ts:1774`）。

映射阶段：MCP 协议的 `readOnlyHint` 注解自动映射到 `isConcurrencySafe`。MCP 工具的并发安全声明来自 `tool.annotations?.readOnlyHint ?? false`（`src/services/mcp/client.ts:1780`）——如果 MCP 工具声明了只读，自动获得并发调度优化；如果没有声明，默认 `false`（fail-closed）。

调度阶段：`partitionToolCalls`（`src/services/tools/toolOrchestration.ts:91`）不区分工具来源——内置工具和 MCP 工具走同一条 reduce 逻辑。isConcurrencySafe 为 true 的工具合并到并发批次，false 的串行执行。MCP 工具无需特殊路径。

图 **7-2**：**MCP** 工具复用内置基座的完整路径



这条路径验证了基座设计的核心价值：第三方工具通过 MCP 协议接入，自动获得内置工具的全部安全默认值和调度优化。54 个内置工具和无数 MCP 工具共用同一条调度路径——这就是"最小基座"的扩展力。

本章主成分：最小工具基座

本质：一套 7 个字段的 fail-closed 接口，统一约束所有工具的行为。调度决策压缩为"连续只读并发、其余串行"一条规则。

关键机制：

- `TOOL_DEFAULTS` 集中管理 fail-closed 安全默认值
- `buildTool` 工厂让所有工具继承默认值，开发者只覆盖差异
- `partitionToolCalls` 贪心分批——连续并发安全工具合并，其余串行
- MCP 工具通过对象展开复用基座，`readOnlyHint` 自动映射到 `isConcurrencySafe`

适用边界：

- ✓ 适合：多类型工具（内置+第三方）的 Agent 系统
- ✓ 适合：需要统一并发控制和权限检查的工具库
- ✗ 不适合：只有 1-2 个无状态工具的简单系统
- ✗ 不适合：所有工具行为完全相同的系统

与其他模式的关系：

- 工具基座是第 4 章（单轮执行）工具调度的实现基础
- 第 8 章（先规划后执行）通过工具权限编码约束工具使用
- 第 9 章（渐进式安全）在 `checkPermissions` 上构建权限评估层

你能做什么

- 为每个工具声明 **`isConcurrencySafe`**。只读工具设为 `true` 以获得并发优化，有写操作的工具保持 `false`。用 `buildTool` 工厂而非手动设置默认值。
- 为破坏性操作工具声明 **`isDestructive=true`**。删除、覆盖、不可逆操作必须声明，确保权限系统触发额外确认。
- 为安全相关工具实现 **`toAutoClassifierInput`**。命令执行、网络请求等高风险工具必须提供描述信息，让 AI 安全分类器评估风险（详见第 9 章）。
- 通过环境变量调整并发上限。默认最大并发数为 10，可通过 `CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY` 覆盖——在 I/O 密集型场景中适当增大，在资源受限环境中减小。
- 为你的 **MCP** 工具设置 **`readOnlyHint`** 注解。声明只读的工具自动获得并发调度优化，无需实现 Claude Code 内部接口。
- 检查工具库中遗漏的 **fail-closed** 声明。逐一审查每个工具的 7 个默认字段——任何"大多数时候安全"的工具都应该保持默认 `false`。
- 重新排列工具调用顺序以最大化并发批次。`partitionToolCalls` 是贪心的——把只读工具连续排列可以增加并发机会，夹在写操作之间会打断并发批次。

下一章导读：本章看到了工具基座如何用统一接口约束 50+ 个工具的行为和调度。但工具注册只是第一步——模型在调用工具前需要知道"我能用哪些工具，什么时候用"。第 8 章将展示 Plan Mode 如何通过工具权限编码（Tool Permissions Encoding）约束模型的工具使用策略——从"什么工具可用"到"什么时候用什么工具"。

第 8 章：先规划后执行

> "提示词说'请先规划'，模型可以假装听了。权限撤销说'不能写文件'，模型没有选择。"

"先规划再执行"听起来像一条建议，但 Claude Code 的实现是一套机制。进入 Plan Mode 意味着写文件的工具权限被暂时撤销——模型想写代码，物理上办不到。语言劝导失败的地方，工具权限不会失败。读完本章，你将理解 Plan Mode 如何通过 `ToolPermissionContext.mode` 的状态切换，把"先规划"从建议升级为不可绕过的约束。

问题——"先规划后执行"为什么需要机制而非指令

同样的"先规划"要求，用提示词写和用权限约束写，结果截然不同。

提示词层面的"先规划后执行"指令，模型可以"理解"但在压力下跳过——面对一个紧急 bug 修复任务，模型可能认为"直接修改更快"而跳过规划步骤。这不是模型的错误——它做出了一个合理的效率权衡，只是这个权衡不符合安全策略。

约束方式	可靠性	灵活性	可绕过性
提示词指令（"请先规划"）	低——模型可以选择忽略	高——可以区分紧急和非紧急场景	完全可绕过
工具权限撤销（写文件权限移除）	高——模型物理上无法写文件	低——所有写操作一律阻止	不可绕过

Claude Code 的解法是 `EnterPlanModeTool`。它的 `call()` 方法做了两件事：切换权限模式（`setAppState` 将 `mode` 设为 `'plan'`）和在工具结果中注入指令文本（"Remember: DO NOT write or edit any files yet"）。两者的优先级不同——权限模式切换是机制层（模型物理上无法写文件），注入的指令是信息层（告知模型当前处于规划阶段）。机制层不会被绕过，信息层只是辅助。

原则 **8.1**：机制约束优先于语言劝导 — 能用工具权限编码的行为约束，禁止退化为提示词指令。模型对权限约束没有选择权——它只能在允许的工具集内行动。

黄金法则——用工具权限编码行为约束

Plan Mode 的核心原则是"能用权限约束的行为，不要用提示词劝导"。这个原则的设计证据在 `prepareContextForPlanMode` 函数中——进入规划模式时，它修改 `ToolPermissionContext` 的 `mode` 字段，而非修改系统提示词。

`ToolPermissionContext.mode` 是权限系统的核心控制面（详见第 9 章）。当 mode 设为 `'plan'`，权限系统自动限制可用工具集——写文件、执行命令等修改性操作的工具不再可用。模型只能使用只读工具（文件读取、代码搜索）来探索和理解代码库，然后输出规划文本。

`handlePlanModeTransition` 管理模式转换的副作用。注释说明了状态管理逻辑："If switching TO plan mode, clear any pending exit attachment. This prevents sending both plan_mode and plan_mode_exit when user toggles quickly."（译：如果切换到 plan 模式，清除任何待处理的退出附件。这防止用户快速切换时同时发送 plan_mode 和 plan_mode_exit）。这个细节揭示了工程考量——快速切换（Enter→Exit→Enter）需要仔细的状态清理，否则会出现不一致的附件状态。

约束层级	实现方式	**Plan Mode** 效果
机制层	`ToolPermissionContext.mode = 'plan'`	写文件工具权限撤销，模型物理上无法修改代码
信息层	工具结果中的指令文本	告知模型处于规划阶段，应专注分析而非修改
状态层	`handlePlanModeTransition`	记录模式转换，管理附件状态

原则 **8.2**：约束的可审计性 — 行为约束必须通过可审计的机制实现——权限模式切换留下明确的状态记录，提示词劝导不留任何可追踪的痕迹。

适用场景——哪些任务需要 Plan Mode

不是所有任务都需要 Plan Mode。`EnterPlanModeTool` 的 `searchHint` 给出了清晰的适用范围："switch to plan mode to design an approach before coding"（译：切换到规划模式，在编码前设计方案）。这个描述揭示了 Plan Mode 的本质——不是防止模型做坏事，而是让模型先想清楚再动手。

适合 **Plan Mode** 的场景：

- 大规模重构——影响超过 5 个文件的结构调整，先规划修改范围和顺序
- 架构变更——修改核心接口或数据流，先分析影响面和兼容性
- 数据库迁移——不可逆操作，先确认迁移脚本和回滚方案
- 多步骤任务——超过 3 步且步骤间有依赖的操作链

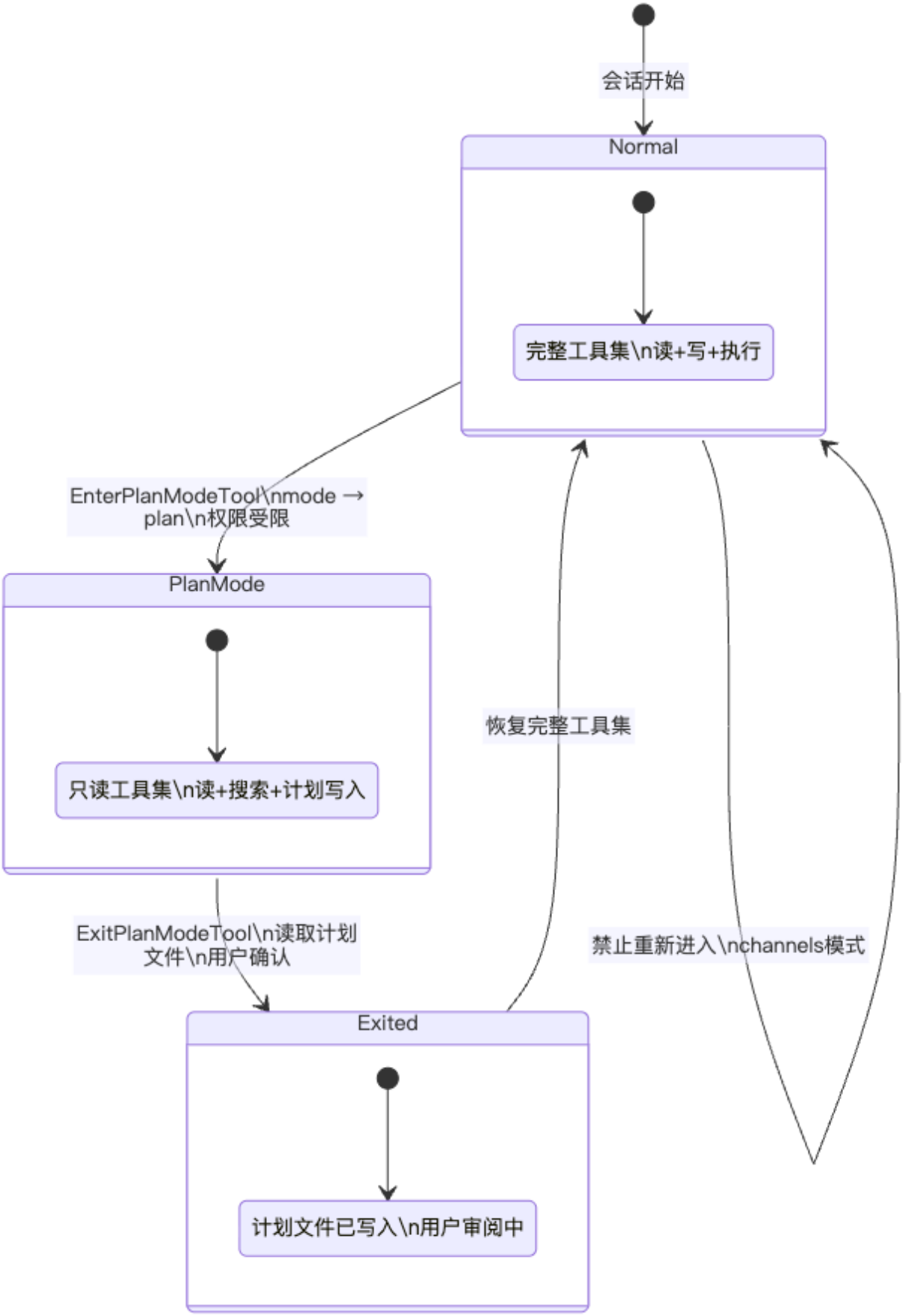
不需要 **Plan Mode** 的场景：

- 单文件小修改——影响范围明确，规划成本高于执行成本
- 紧急 hotfix——时间敏感，规划延迟不可接受
- 探索性任务——本身就是"看看代码是什么样"，没有明确的执行目标

工作原理——Plan Mode 的完整状态机

Plan Mode 是一个三态状态机：正常 → 规划中 → 已退出。每次状态转换都有对应的工具权限变更和磁盘操作。

图 **8-1**：**Plan Mode** 三态状态机



进入阶段：模型调用 `EnterPlanModeTool`，触发 `prepareContextForPlanMode` 修改权限上下文。如果当前处于 auto 模式，`prepareContextForPlanMode` 会先恢复危险权限

（`restoreDangerousPermissions`），保存 `prePlanMode` 状态用于退出时恢复。权限切换后，写文件和执行命令的工具从可用列表中移除。

规划阶段：模型在只读工具集下探索代码库。同时，`getPlanSlug` 生成唯一的计划文件名——使用可读的 word slug（如 "happy-fox-42"）而非 UUID，最多重试 10 次（`MAX_SLUG_RETRIES`）处理文件名冲突。`getPlanFilePath` 拼接完整路径，计划内容写入磁盘。

退出阶段：模型调用 `ExitPlanModeV2Tool`。它读取磁盘上的计划文件（`getPlanFilePath(context.agentId)`），供用户审阅。用户可以在 CCR（Continuous Code Review）场景下编辑计划——注释说明了覆盖逻辑："CCR web UI may send an edited plan via permissionResult.updatedInput"（译：CCR Web UI 可以通过 permissionResult.updatedInput 发送编辑后的计划）。确认后，`setHasExitedPlanMode(true)` 标记会话级退出状态，权限模式恢复到 `prePlanMode`。

状态	可用工具	磁盘操作	权限模式
正常	完整工具集	无	原始 mode
规划中	只读 + 计划写入	生成 slug → 写计划文件	`plan`
已退出	完整工具集	读取计划文件	恢复 `prePlanMode`

权衡——计划审批流的 3 个设计选择

决策维度	选择 **A**（本系统）	选择 **B**	核心权衡
计划存储	写入磁盘文件	存储在内存 state 中	用户可审批前编辑 vs 文件管理复杂度
进入门控	channels 模式下同时禁用 Enter 和 Exit	只禁用 Exit	防止死锁 vs 允许进入但不允许退出
文件命名	可读的 word slug	UUID	调试可读性 vs 冲突处理开销

选择一：计划文件持久化到磁盘

计划内容写入磁盘文件（`getPlanFilePath` 指向实际文件路径），而非存储在内存 state 中。这个选择的直接收益是 CCR 场景——用户可以在 Web UI 中查看和编辑计划文件，修改后的内容通过 `permissionResult.updatedInput` 覆盖磁盘版本。如果存在内存中，外部编辑是不可能的。代价是需要管理文件的生命周期——创建、读取、清理。

选择二：进入/退出门控的对称性

`isEnabled()` 中有一段关键的注释："When --channels is active, ExitPlanMode is disabled (its approval dialog needs the terminal). Disable entry too so plan mode isn't a trap."（译：channels 模式活跃时，ExitPlanMode 被禁用（审批对话框需要终

端）。同时禁用进入，以免 Plan Mode 变成一个陷阱）。如果只禁用 Exit 而不禁用 Enter，模型可以进入 Plan Mode 但永远无法退出——一个死锁。对称门控防止了这种"陷阱"。

选择三：**word slug 代替 UUID**

`getPlanSlug` 生成如 "swift-raven-7" 的可读文件名，而非 `550e8400-e29b-41d4-a716-446655440000`。在调试和日志中，"swift-raven-7" 比 UUID 可读性高得多。代价是需要最多 10 次（`MAX_SLUG_RETRIES`）重试处理文件名冲突——多会话同时创建计划时可能出现 slug 碰撞。

踩坑指南——Plan Mode 中的常见错误

陷阱一：在 **agent** 子任务中调用 **EnterPlanMode**

`EnterPlanModeTool.call()` 的第一行就检查了上下文——如果在 agent 子任务中被调用，直接抛出异常。Plan Mode 是会话级机制，不应该在子智能体的上下文中使用——子智能体没有独立的权限上下文来管理 plan mode 的状态转换。

❌ 错误做法：在 agent 子任务中让模型进入 Plan Mode，以为子智能体也可以"先规划"。
✓ 正确做法：Plan Mode 只在主会话中使用。子智能体有自己的任务编排机制（详见第 12 章）。

陷阱二：计划文件目录不存在时退出失败

`getPlanFilePath` 依赖 `getPlansDirectory`（memoized，只初始化一次）。如果计划目录在规划阶段没有正确创建，退出时读取计划文件会失败——模型完成了规划但无法交付计划内容。

❌ 错误做法：假设计划目录已经存在，不在退出前检查路径有效性。
✓ 正确做法：在进入 Plan Mode 时确保目录创建，退出时验证计划文件路径可读。

陷阱三：快速切换导致附件状态不一致

`handlePlanModeTransition` 专门处理快速切换（Enter→Exit→Enter）的边界情况。如果状态清理不完整，系统可能同时发送 `plan_mode` 和 `plan_mode_exit` 附件——导致下游系统行为不一致。

❌ 错误做法：忽略快速切换场景，只处理正常的 Enter→Exit 单次流程。
✓ 正确做法：每次进入 plan mode 时清除 `needsPlanModeExitAttachment`，确保状态干净。

实证——从工具调用到计划文件的完整路径

一次完整的 Plan Mode 循环涉及 3 个工具调用、2 次权限模式切换、1 次文件写入。这条路径验证了三态状态机的实际运作。

进入：模型调用 `EnterPlanModeTool`（`src/tools/EnterPlanModeTool/EnterPlanModeTool.ts:77`）。`prepareContextForPlanMode`（`src/utils/permissions/permissionSetup.ts:1462`）修改 `ToolPermissionContext`，保存 `prePlanMode`，切换 `mode = 'plan'`。`handlePlanModeTransition`（`src/bootstrap/state.ts:1349`）清除待处理的退出附件。

规划：模型在只读工具集下探索代码库。同时，`getPlanSlug`（`src/utils/plans.ts:32`）生成唯一文件名，`getPlanFilePath`（`src/utils/plans.ts:119`）拼接路径，计划内容写入磁盘。`getPlansDirectory`（`src/utils/plans.ts:79`）使用 memoize 确保目录只初始化一次。

退出：模型调用 `ExitPlanModeV2Tool`。读取磁盘上的计划文件供用户审阅。用户确认后，`setHasExitedPlanMode(true)`（`src/tools/ExitPlanModeTool/ExitPlanModeV2Tool.ts:359`）标记会话级退出状态，权限模式恢复到 `prePlanMode`。

图 **8-2**：**Plan Mode** 完整调用链

这条路径验证了核心设计：Plan Mode 通过修改权限上下文来约束模型行为，通过磁盘文件来交付规划结果。进入/退出的对称性确保模型不会陷入死锁，word slug 提升了调试效率。

本章主成分：**Plan Mode**

本质：通过修改 `ToolPermissionContext.mode` 撤销写文件工具的权限，把"先规划"从语言劝导升级为物理约束。

关键机制：

- `prepareContextForPlanMode` 切换权限模式，保存 `prePlanMode` 用于恢复
- `handlePlanModeTransition` 管理快速切换的状态清理
- 计划文件通过 `getPlanFilePath` + word slug 持久化到磁盘
- 进入/退出门控的对称性防止 channels 模式下的死锁

适用边界：

- ✓ 适合：不可逆操作前的强制规划（重构、迁移、架构变更）

- ✓ 适合：多步骤依赖任务链
- ✗ 不适合：单文件小修改（规划成本高于执行成本）
- ✗ 不适合：紧急 hotfix（时间敏感）

与其他模式的关系：

- 本章是第 7 章（工具基座 `checkPermissions`）的应用实例
- 第 9 章（渐进式安全）展示了 `ToolPermissionContext` 的完整权限体系
- 第 12 章（隔离与交接）的子智能体排除了 Plan Mode（agentId 检查）

你能做什么

- 为你的 **Agent** 中不可逆操作前添加 **Plan Mode** 入口。用权限约束（而非提示词指令）来强制执行"先探索后行动"的行为顺序——模型无法绕过它不具备的工具权限。
- 把计划文件写入磁盘而非内存。让用户可以在执行前审阅和编辑计划——外部编辑能力是文件持久化的核心价值。
- 设计进入/退出的对称门控。禁用进入时同时禁用退出——防止模型进入后无法退出的死锁。
- 在多智能体场景中，为每个 **agent** 维护独立的计划文件路径。基于 `agentId` 区分计划文件，避免多智能体写入冲突。
- 用可读的文件名（如 **word slug**）代替 **UUID**。在调试和日志中提升可读性——接受最多 10 次重试的冲突处理开销。
- 对照第 **9** 章权限系统，理解 **`ToolPermissionContext.mode`** 的完整状态空间。Plan Mode 只是 mode 字段的一个值——了解完整的状态空间才能设计更丰富的行为约束。
- 为快速切换场景添加状态清理。每次进入 Plan Mode 时清除待处理的退出状态，防止不一致的附件发送。

下一章导读：本章看到了 Plan Mode 如何通过权限模式切换约束模型行为。但 Plan Mode 只是权限系统的一个应用——`ToolPermissionContext` 的完整体系如何运作？权限评估器如何分层决策？第 9 章将深入渐进式安全设计，展示 Harness 如何用多层评估器实现"默认拒绝、逐级放行"的权限策略。

第五篇：护栏与扩展

安全不是开关，而是光谱。从最严格的白名单到最宽松的自动批准，从权限分级到外部钩子——这一篇揭示 **Harness** 如何在安全与灵活之间找到精妙的平衡点。

第 9 章:渐进式安全

> "最好的安全护栏是透明的--用户感知不到它,但越界的模型行为不可能穿过它。"

安全不是一个开关,是一条连续谱。从"每步都问用户"到"完全不问"之间,Claude Code 设计了 5 个精确的档位,每个档位有不同的守卫机制。最精妙的一档用 Claude 自己来判断 Claude 的行为是否安全--这是 Harness 在安全性上的递归应用。读完本章,你将理解 4 个子系统如何构成纵深防御,以及"默认拒绝、逐级放行"的安全谱系如何覆盖从个人开发到企业 CI 的全部场景。

问题--单一权限开关为什么不够

不同使用场景需要不同的安全级别。个人开发者可能希望文件编辑自动通过但命令执行需要确认;企业 CI/CD 管道需要完全自主执行不需要人工干预;代码审查场景需要模型只读不写。一个简单的"开/关"无法覆盖这些截然不同的需求。

Claude Code 的解法是一个权限谱系--`EXTERNAL_PERMISSION_MODES` 定义了 5 个面向用户的权限模式(`acceptEdits`、`bypassPermissions`、`default`、`dontAsk`、`plan`),`INTERNAL_PERMISSION_MODES` 在此基础上增加了 2 个内部模式(`auto`、`bubble`),共 7 个模式覆盖从"完全手动"到"完全自主"的全部场景。

模式	安全级别	用户交互	典型场景
`default`	最高	每次操作都确认	首次使用、高风险操作
`acceptEdits`	高	文件编辑自动通过,命令需确认	日常开发
`plan`	限制性	只读探索,无写权限(详见第 8 章)	架构分析、方案设计
`dontAsk`	中低	静默拒绝代替询问	批量非交互场景
`bypassPermissions`	无	无任何护栏	受控 CI 环境
`auto`(内部)	中	AI 分类器自动判断	内部用户半自主模式

原则 **9.1:**安全谱系覆盖全场景 - Agent 系统的安全策略必须提供连续的权限档位而非单一开关。不同场景的安全需求是量变的,不是质变的--用一个模式覆盖所有场景等于用一种安全级别应对所有风险。

黄金法则--安全谱系是从严到宽的单向门

权限谱系的设计原则是“从最严开始，用户主动解锁更宽”——默认值是 `default`（最严格），用户通过 Shift+Tab 循环解锁更宽松的模式。`getNextPermissionMode` 的 switch-

case 结构是谱系顺序最直接的源码证据：`default → acceptEdits → plan → bypassPermissions → default`（循环）。

向右移动解锁更多自主性,向左移动增加安全护栏。注释说明了内部用户的谱系差异:"Ants skip acceptEdits and plan - auto mode replaces them"(译:内部用户跳过 acceptEdits 和 plan--auto 模式替代它们)。内部用户的谱系是 `default → auto → bypassPermissions`,更激进但也更有 AI 监督。

谱系的关键设计:系统始终可以从任何模式回到 **default**。这不是说用户不能手动选择更严格的模式,而是系统的默认行为是"从严开始"。即使当前处于 `bypassPermissions`(无护栏),下一次循环回到 `default` 时所有安全机制重新生效。

原则 **9.2:**默认值是最安全的选项 - 系统的初始权限模式必须是最严格的。向宽松方向移动需要用户主动操作(如 Shift+Tab),向严格方向移动是默认行为。禁止将宽松模式设为默认值。

适用场景--哪个权限档位用于哪个场景

5 个权限档位各有精确的适用场景,从交互式开发到无人值守 CI 全覆盖。

`PERMISSION_MODE_CONFIG` 为每个模式配置了 title、shortTitle 和 color。`color = 'error'` 标记高危模式--`bypassPermissions` 和 `dontAsk` 都是 error 级,提醒用户当前处于危险区域。这不是装饰性的颜色编码,而是 UI 层面对安全等级的视觉表达。

`default`--每次操作需确认:适合首次使用 Claude Code 或执行高风险操作。代价是频繁的权限弹窗打断工作流--每读一个文件、执行一个命令都需要确认。

`acceptEdits`--文件编辑自动通过:日常开发的最佳平衡。读文件和编辑文件不需要确认,但执行命令(如 `npm test`)仍需确认。这是大多数开发者选择的档位。

`plan`--只读探索:详见第 8 章。模型只能读文件和搜索代码,不能写任何内容。

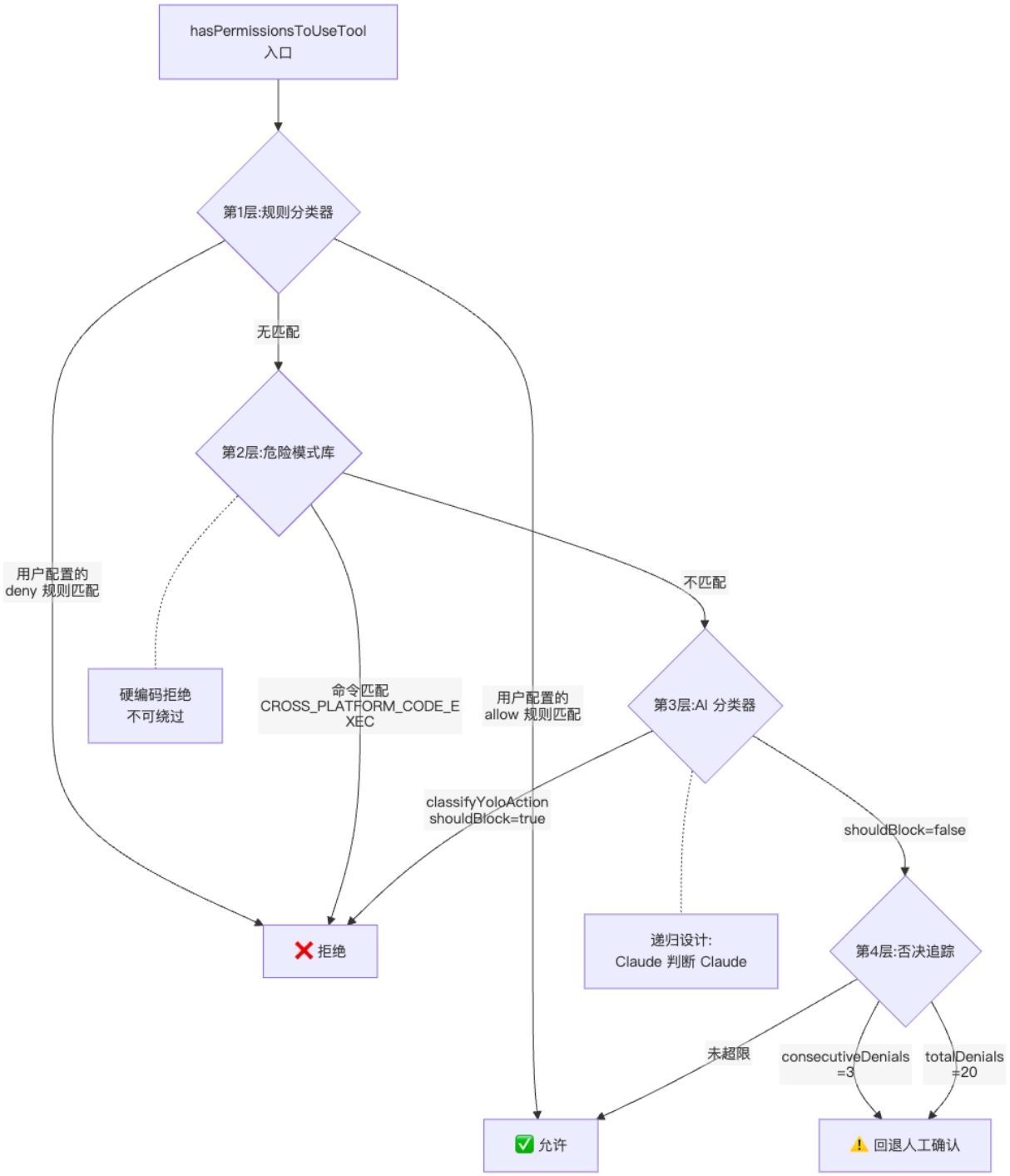
`dontAsk`--静默拒绝:模型尝试执行不允许的操作时不弹窗询问用户,而是直接拒绝并继续。适合批量非交互场景,避免弹窗阻塞管道。

`bypassPermissions`--无护栏:所有 4 层防御全部关闭。仅在完全受控的 CI 环境(无用户交互、操作可回滚)中使用。

工作原理--4 个子系统构成的纵深防御

Harness 的权限判断不是一个 if-else,而是 4 个子系统串联:规则分类器 → 危险模式库 → AI 分类器 → 否决追踪。`hasPermissionsToUseTool` 是 4 层判断的入口。

图 **9-1:auto** 模式 **4** 层纵深防御



第 1 层:规则分类器

检查用户配置的允许/拒绝规则。用户可以通过配置文件声明"允许 Bash(git:*)"或"拒绝 *Edit()*"--这些规则是最高优先级的判断。如果用户配置了明确的 allow 规则,后续 3 层检查全部跳过。这是"用户主权"的体现--用户的显式配置高于一切自动判断。

第 2 层:危险模式库

`CROSS_PLATFORM_CODE_EXEC` 是一个硬编码的危险模式列表--包含 `python`、`node`、`bash`、`sh` 等跨平台代码执行入口点。源码注释说明了这个库的用途:"An allow rule like Bash(python:*) lets the model run arbitrary code via that interpreter, bypassing the*

auto-mode classifier. These lists feed the isDangerousBashPermission predicates, which strip such rules at auto-mode entry."(译:一条 Bash(python:) 的允许规则让模型可以通过该解释器运行任意代码,绕过 auto 模式分类器。这些列表提供给 isDangerousBashPermission 断言,在进入 auto 模式时剥离此类规则)。

这层的关键特性是不可绕过--即使用户配置了 `Bash(python:*)` 的 allow 规则,进入 auto 模式时这条规则也会被自动剥离。硬编码拒绝是 AI 分类器的前置守卫。

第 3 层:AI 分类器

`classifyYoloAction` 是最精妙的设计--它把整个对话历史发给 Claude 模型,让 Claude 判断当前操作是否安全。这是 Harness 在安全性上的递归应用:用 Claude 监督 Claude。`AutoModeRules` 定义了 AI 分类器的规则配置,规则可配置而非硬编码,说明分类器的行为可以按场景调整。

AI 分类器能理解上下文--它看到完整的对话历史,知道"这个 Bash 命令是在执行用户明确要求的重构任务"还是"模型在尝试绕过安全限制"。这种上下文理解是纯规则匹配无法实现的。

第 4 层:否决追踪

`DENIAL_LIMITS` 定义了精确的否决上限:`maxConsecutive: 3`(连续否决不超过 3 次)、`maxTotal: 20`(总计否决不超过 20 次)。`shouldFallbackToPrompting` 检查否决计数--如果超限,强制退出 auto 模式,回退到人工确认。

防御层	输入	输出	失败时行为
规则分类器	用户配置	allow/deny	无匹配时传递给下一层
危险模式库	工具名+参数	硬拒绝	不在危险列表中时传递给下一层
AI 分类器	完整对话历史	shouldBlock	shouldBlock=false 时传递给下一层
否决追踪	否决计数	是否超限	超限时强制回退人工确认

权衡--AI 分类器的 3 个设计代价

AI 分类器在安全性和自主性之间找到了独特平衡点,但引入了 3 个不可忽视的代价。

决策维度	选择 A(本系统)	选择 B	核心权衡
AI 监督	用 Claude 判断 Claude	纯规则匹配	上下文理解能力 vs 对抗性风险
否决阈值	3 次连续 / 20 次总计	无上限或更低阈值	自主模式持续性 vs 安全兜底频率
auto 模式暴露	内部专用(Feature Flag)	对所有用户开放	验证可靠性 vs 功能可达性

代价一**:AI** 分类器的对抗性

用 Claude 判断 Claude 的行为,在提示词注入场景下可能被绕过--攻击者可能在对话历史中注入误导性内容,让 AI 分类器误判操作为安全。这个代价的缓解方案是危险模式库--代码执行入口点(python/node/bash)的硬编码拒绝不经过 AI 分类器,即使在 auto 模式下也有效。

代价二**:**否决阈值的精确校准

`maxConsecutive: 3` 和 `maxTotal: 20` 是精确校准的数字。如果阈值太低(如 1 次连续否决就退出),auto 模式频繁退出,用户体验差;如果阈值太高(如 100 次总计才退出),连续错误操作无法被人类干预,安全风险大。3 和 20 的比例设计编码了一个判断--"连续 3 次错误操作"比"分散的 20 次错误操作"更危险。

代价三**:auto** 模式不对外开放

`auto` 模式通过 `USER_TYPE === 'ant'` 判断和 Feature Flag 控制,仅内部用户可用。这暗示 AI 分类器的可靠性尚未达到对所有用户开放的水平(推断)。内部用户对系统行为有更高的容忍度和理解力,外部用户遇到误判会造成信任损失。

踩坑指南--权限系统中的常见错误

陷阱一**:**通配 **allow** 规则绕过 **AI** 分类器

源码注释明确警告:`Bash(python:*)` 这样的通配允许规则让模型可以通过 python 解释器运行任意代码,绕过 auto 模式的 AI 分类器。虽然 auto 模式进入时会自动剥离此类规则,但在非 auto 模式下这些规则仍然生效。

❌ 错误做法:配置 `Bash(python:*)` 或 `Bash(node:*)` 的通配允许规则,以为"我只在受控环境中用"。 ✓ 正确做法:如果需要允许 python 执行,使用精确的命令匹配(如 `Bash(python:my_script.py)`),避免通配符。理解通配 allow 规则会完全绕过危险模式库。

陷阱二**:auto** 模式下高频操作触发否决累积

auto 模式下,每次 AI 分类器拒绝操作都会增加否决计数。如果一个任务类型频繁触发 AI 分类器的误判,否决计数迅速累积到上限(3 次连续或 20 次总计),auto 模式被强制退出。

❌ 错误做法:在 auto 模式下执行大量高风险操作(如频繁的 rm 命令),不监控否决计数。 ✓ 正确做法:监控 `shouldFallbackToPrompting` 的触发频率。高频触发说明 AI 分类器在当前任务类型上不可靠,应切换到更宽松或更严格的模式。

陷阱三**:**误以为 **bypassPermissions** 只是**"**不问用户**"**

`bypassPermissions` 不是跳过确认弹窗--它完全移除了 4 层防御。规则分类器、危险模式库、AI 分类器、否决追踪全部失效。模型可以执行任何操作,没有安全网。

❌ 错误做法:在交互式开发中使用 bypassPermissions "减少打扰"。 ✓ 正确做法:bypassPermissions 仅在完全受控的 CI 环境中使用--没有用户交互、所有操作可回滚、输出经过审计。

实证--一次 auto 模式工具调用的决策链

在 auto 模式下,一次工具调用在执行前经过了至少 4 次独立判断。以 `Bash("rm -rf /tmp/old")` 为例追踪完整决策路径。

入口:`hasPermissionsToUseTool`(`src/utils/permissions/permissions.ts:473`)接收工具名(Bash)、参数(rm -rf /tmp/old)和当前权限上下文(mode=auto),启动 4 层判断。

第 1 层(规则分类器):检查用户配置的 allow/deny 规则。如果用户配置了 `Bash(rm:*)` 的 allow 规则,直接允许,跳过后续 3 层。假设无匹配规则,传递给第 2 层。

第 2 层(危险模式库):检查 `rm -rf /tmp/old` 是否匹配 `CROSS_PLATFORM_CODE_EXEC`(`src/utils/permissions/dangerousPatterns.ts:15`)中的代码执行入口点。`rm` 不在 python/node/bash 列表中--通过。传递给第 3 层。

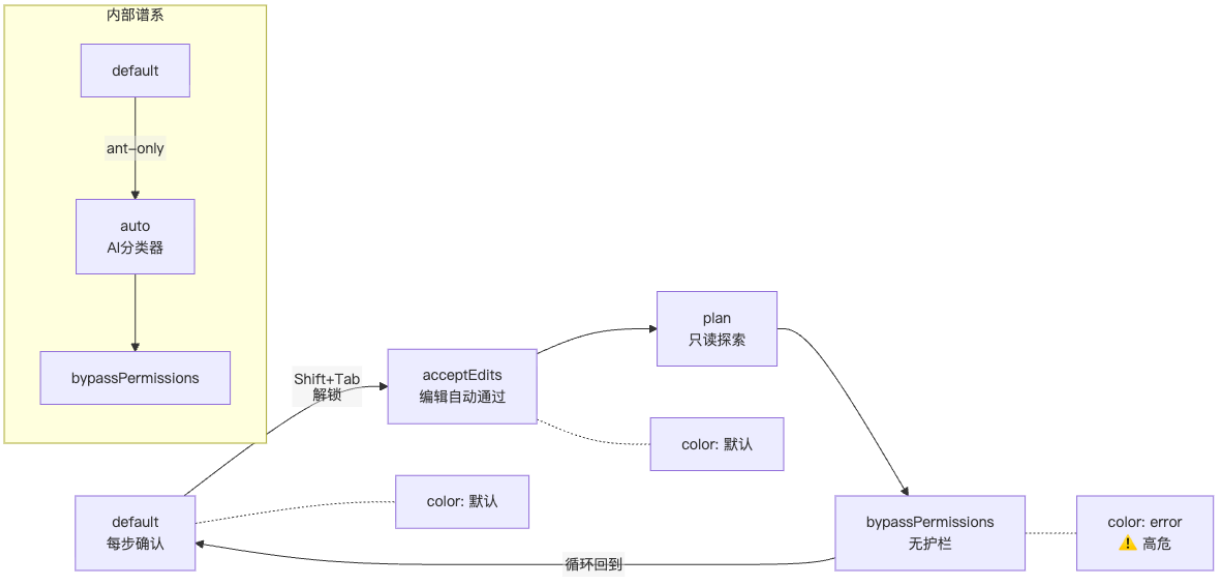
第 3 层(AI 分类器):`classifyYoloAction`(`src/utils/permissions/yoloClassifier.ts:1012`)把完整对话历史(包括用户最初的请求、模型之前的操作、当前要执行的命令)发给 Claude API。Claude 看到用户要求"清理临时文件",判断 `rm -rf /tmp/old` 是合理的清理操作,返回 `shouldBlock=false`。传递给第 4 层。

第 4 层(否决追踪):`shouldFallbackToPrompting`(`src/utils/permissions/denialTracking.ts:40`)检查否决计数。`consecutiveDenials=0 < 3`,`totalDenials=0 < 20`--未超限。最终判定:允许执行。

如果第 3 层 AI 分类器返回 `shouldBlock=true`(例如模型尝试执行 `rm -rf /`),否决计数增加到 `consecutiveDenials=1`。如果连续 3 次被阻止,`shouldFallbackToPrompting=true`,auto 模式强制退出,回退到人工确认。

这条路径验证了纵深防御的核心价值:每一层独立判断,层与层之间不共享决策逻辑。一层的误判可以被下一层纠正--规则分类器的遗漏被危险模式库兜底,危险模式库的遗漏被 AI 分类器兜底,AI 分类器的误判被否决追踪兜底。

图 9-2:权限谱系从严到宽的循环



本章主成分:渐进式安全

本质:5 个权限档位构成从严到宽的单向解锁谱系。auto 档位通过 AI 递归判断实现"让 Claude 监督 Claude"--4 层纵深防御确保即使 AI 分类器被绕过,硬编码的危险模式库和否决追踪仍能兜底。

关键机制:

- 5 个外部模式 + 2 个内部模式覆盖全部安全场景
- `getNextPermissionMode` 的谱系循环:default → acceptEdits → plan → bypassPermissions
- 4 层纵深防御:规则分类器 → 危险模式库 → AI 分类器 → 否决追踪
- `DENIAL_LIMITS`(maxConsecutive=3, maxTotal=20)作为 auto 模式的安全网

适用边界:

- ✓ 适合:需要不同安全级别的多场景 Agent 系统
- ✓ 适合:需要在安全性和自主性之间找到平衡的半自主模式
- ✗ 不适合:所有操作安全等级完全相同的简单系统

与其他模式的关系:

- 本章是第 7 章(工具基座 `checkPermissions`)的运行时实现
- 第 8 章(Plan Mode)是权限谱系中 `plan` 档位的具体应用
- 第 10 章(Hooks 系统)在权限决策后提供进一步的行为拦截

你能做什么

- 审视你的 **Agent** 使用了哪个权限档位。生产环境应避免 `bypassPermissions`--如果需要自主执行,考虑 auto 模式(需要实现 AI 分类器)而非完全移除护栏。
- 为你的工具实现 **`toAutoClassifierInput`**。AI 分类器能看到的安全信息越多,判断越准确--告诉分类器工具的用途和风险等级(详见第 7 章)。
- 检查你的 **allow** 规则是否包含代码执行入口点。`Bash(python:*)` 或 `Bash(node:*)` 会绕过危险模式库和 AI 分类器--使用精确匹配而非通配符。
- 设置合理的否决上限。参考 `maxConsecutive=3` 和 `maxTotal=20` 的比例设计--连续否决比分散否决更危险,连续阈值应远低于总计阈值。
- 为高危操作构建专用的危险模式库。代码执行入口点(python/node/bash)应在 AI 分类器之前提供硬拒绝--这是不可绕过的安全底线。
- 监控 **`shouldFallbackToPrompting`** 的触发频率。高频触发说明 AI 分类器在当前任务类型上不可靠,应调整分类器规则或切换模式。
- 理解 **auto** 模式的递归性质。AI 分类器本身也可能被误导--否决追踪是最后的安全网,确保即使分类器失效,系统也不会无限执行危险操作。

下一章导读:本章看到了权限系统如何通过 4 层纵深防御保护每次工具调用。但权限决策只是 Harness 控制模型行为的一种方式--第 10 章将展示 Hooks 系统如何在权限决策之后、工具执行前后提供额外的行为拦截和审计能力。这是 Harness 的"神经末梢"--最靠近执行层的控制点。

第 10 章：Harness 的神经末梢

> "好的扩展点不是'打开代码随便改'，而是'在正确的时机、用定义好的方式注入行为'。"

Harness 的核心代码不可修改，但它在 27 个时机点留了"钩子"（Hooks）。任何 shell 命令、HTTP 请求或 AI Agent 都可以挂上去，观察行为、改写决策、注入上下文。这些钩子是 Harness 向外延伸的神经末梢——唯一的、有设计的双向接口。读完本章，你将理解 4 种执行方式如何按能力层次递进，以及 Hooks 如何在不修改核心代码的情况下"重新接线" Harness 的行为。

问题——如何在不修改核心代码的情况下改变 Harness 行为

Hooks 解决的核心问题是扩展性。源码注释定义了 Hooks 的边界："Hooks are user-defined shell commands that can be executed at various points in Claude Code's lifecycle."（译：Hooks 是用户定义的 shell 命令，可以在 Claude Code 生命周期的各个时机点执行）。

`HOOK_EVENTS` 数组列出了 27 个触发点，覆盖 Harness 的完整生命周期。按功能分组：

类别	触发点	数量
工具调用	PreToolUse、PostToolUse、PostToolUseFailure	3
会话边界	SessionStart、SessionEnd、Setup	3
停止条件	Stop、StopFailure	2
权限	PermissionRequest、PermissionDenied	2
上下文压缩	PreCompact、PostCompact	2
多智能体	SubagentStart、SubagentStop、TeammateIdle、TaskCreated、TaskCompleted	5
用户交互	UserPromptSubmit、Notification、Elicitation、ElicitationResult	4
配置与环境	ConfigChange、CwdChanged、FileChanged、InstructionsLoaded、WorktreeCreate、WorktreeRemove	6

27 个触发点意味着 Harness 的行为在 27 个位置可以被外部代码观测和修改。这些触发点不是事后添加的"插件接口"——它们是 Harness 设计之初就预留的扩展点。内部机制（如第 4 章的后采样 Hooks）和外部用户配置的 Hooks 走同一套调度系统。

原则 **10.1**：预定义扩展点优于开放修改 — Agent 系统必须在核心代码中预定义有限的扩展点，而非允许外部代码直接修改核心逻辑。扩展点数量禁止无限增长——每个新触发点都有维护成本和安全审计成本。

黄金法则——Hook 是配置文件控制 Harness 行为的唯一接口

Hooks 是 Harness 与外部世界之间唯一的、有设计的双向接口。外部可以读取 Harness 状态（通过 Hook 的输入），也可以改变 Harness 决策（通过 Hook 的输出）。

`registerPostSamplingHook`（内部 Hook 注册机制）与用户配置的 Hooks 并列运行——这说明 Hooks 不是"用户专用"的接口，而是 Harness 内部扩展的通用模式。内部功能（如第 4 章的 `handleStopHooks`）通过 `registerPostSamplingHook` 注册到同一调度系统，与用户配置的 Hooks 走同一条执行路径。

Hooks 的双向能力通过返回值实现：

方向	能力	示例
读取	Hook 收到当前会话状态	Stop Hook 读取完整对话历史
读取	Hook 收到触发事件上下文	PostToolUse Hook 收到工具执行结果
写入	返回 `permissionBehavior` 覆盖权限决策	PermissionRequest Hook 返回 allow/deny（与第 9 章联动）
写入	返回 `additionalInput` 注入上下文	PreToolUse Hook 注入当前环境信息
写入	返回 `preventContinuation` 终止循环	Stop Hook 判断任务未完成，阻止 Harness 结束

`hook.type` 的 4 种类型（`command`、`prompt`、`agent`、`http`）在统一调度点过滤——所有 Hook 类型走同一条分发逻辑，只是执行方式不同。

原则 **10.2**：扩展接口必须双向 — Hook 禁止设计为只读接口。如果外部只能观察但不能影响决策，Hook 就退化为日志系统。真正的扩展需要"读状态 + 写决策"的双向能力。

适用场景——哪些场景需要 Hooks

一个 Hook 可以是一行 shell 命令，也可以是一个完整的 AI 对话循环。4 类典型场景：

审计日志：为 `PostToolUse` 注册 command Hook，每次工具调用后写入外部审计日志。延迟极低（毫秒级），不影响 Harness 性能。适合合规要求严格的企业环境。

外部权限系统集成：为 `PermissionRequest` 注册 http Hook，将权限决策转发到企业内部的权限管理系统。返回 `allow` 或 `deny`，与第 9 章的内部权限系统协同工作。

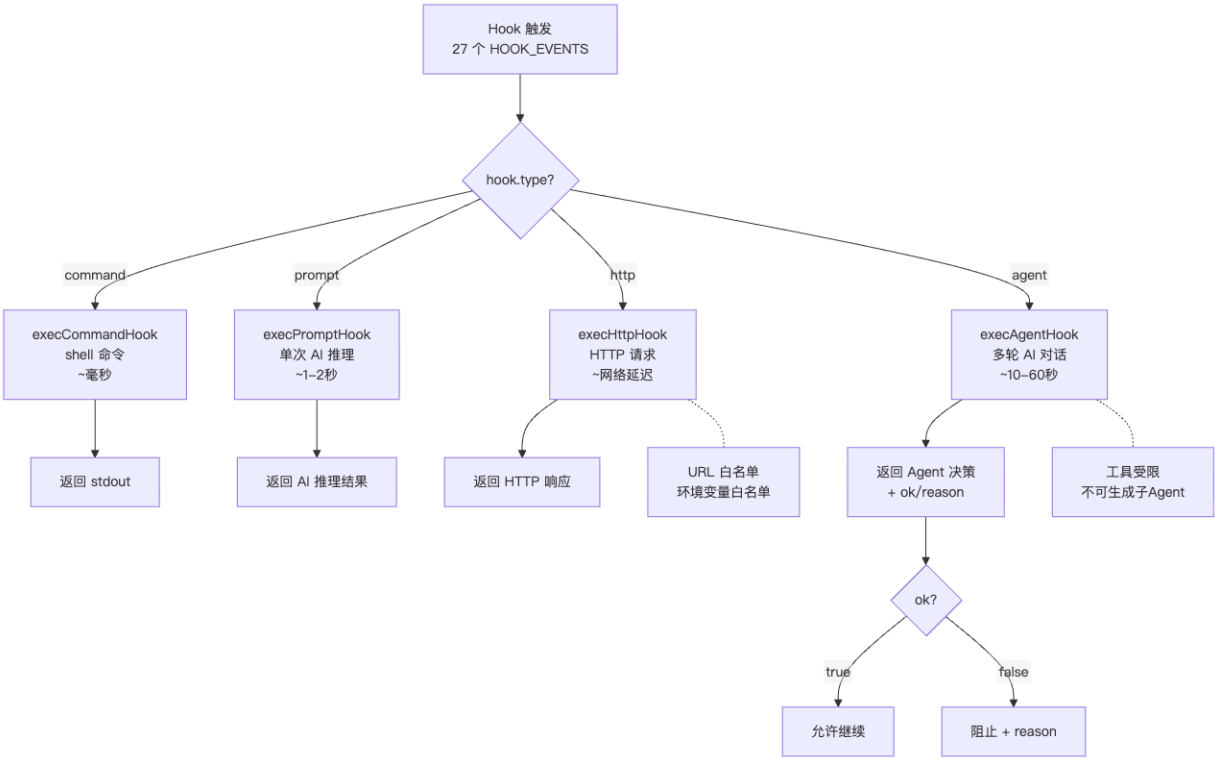
动态上下文注入：为 `PreToolUse` 注册 prompt Hook，在工具调用前通过单次 AI 推理注入当前环境信息（如 CI 构建状态、代码审查意见）。延迟约 1-2 秒，但提供了 Harness 静态上下文无法获得的动态信息。

任务完成验证：为 `Stop` 注册 agent Hook——这是能力最强的用法。`execAgentHook` 注释说明了其角色："You are verifying a stop condition in Claude Code. Your task is to verify that the agent completed the given plan."（译：你正在验证 Claude Code 的停止条件。你的任务是验证 Agent 是否完成了给定的计划）。Agent Hook 可以读取完整对话历史和代码库，判断任务是否真正完成（而非表面完成）。

工作原理——4 种执行方式的完整机制

4 种 Hook 执行方式不是功能重叠，而是能力层次递进——从最快的 shell 命令到最强的 AI Agent 对话，每种方式有不同的延迟和功能边界。

图 10-1：4 种 Hook 执行方式的调度关系



command——shell 命令

最快的执行方式。Harness 执行用户配置的 shell 命令，捕获 stdout 作为返回值。适合简单的日志记录和环境检查。延迟取决于命令本身，通常毫秒级。

prompt——单次 AI 推理

`execPromptHook` 调用 Claude API（`querySource: 'hook_prompt'`）执行单次推理。适合需要 AI 理解上下文但不需工具调用的场景——如"根据当前对话上下文，判断是否应该注入额外的安全提醒"。延迟约 1-2 秒。

http——外部服务调用

`execAgentHook` 发送 HTTP 请求到外部服务。适合连接企业内部系统——权限管理、审计日志、通知系统。`getHttpHookPolicy` 实现了安全策略：`allowedUrls`（URL 白名单）和 `allowedEnvVars`（允许传递的环境变量白名单）。这个策略防止对抗性提示注入修改 HTTP 请求的目标 URL 或注入恶意 header。

agent——完整 AI 对话

`execAgentHook` 启动完整的多轮 Agent 对话。这是能力最强但延迟最高的方式——Agent 可以访问对话历史、读取文件、执行搜索，然后用工具返回判定结果。系统提示词定义了 Agent 的角色："Use the available tools to inspect the codebase and verify the condition. Use as few steps as possible — be efficient and direct."（译：使用可用工具检查代码库并验证条件。尽量少步骤——高效直接）。

Agent Hook 有严格的工具限制。注释说明了原因："Filter out disallowed agent tools to prevent stop hook agents from spawning subagents"（译：过滤不允许的 Agent 工具，防止 stop hook Agent 生成子 Agent）。没有这个限制，Agent Hook 可以无限递归地创建子 Agent，消耗所有资源。

执行方式	延迟	能力	安全限制	典型用途
command	~毫秒	执行 shell 命令	运行在用户权限下	审计日志
prompt	~1-2秒	单次 AI 推理	只读对话历史	上下文注入
http	~网络延迟	外部服务调用	URL 白名单 + 环境变量白名单	企业集成
agent	~10-60秒	完整多轮 AI 对话	工具受限，不可生成子 Agent	任务完成验证

权衡——Hooks 系统的 3 个设计代价

决策维度	选择 **A**（本系统）	选择 **B**	核心权衡
Hook 延迟容忍	agent Hook 可用 10-60 秒	只允许 command Hook	扩展能力 vs 用户体验
Hook 安全沙箱	工具限制 + URL 白名单	无限制	安全性 vs 灵活性
Hook 触发时机可观测性	27 个分散触发点	集中式注册表	覆盖面 vs 调试难度

代价一：**Hook 延迟**

每个 Hook 都在关键路径上。`PreToolUse` 在工具执行前触发——如果配置了 agent Hook，每次工具调用前都要等待 10-60 秒的 AI 对话。在频繁工具调用的场景中（一次任务 30 次工具调用），总延迟可能增加 5-30 分钟。

缓解方案：高频触发点只使用 command Hook（毫秒级），agent Hook 保留给低频触发点（如 `Stop`，每次会话最多触发一次）。

代价二：**Hook 安全性**

Agent Hook 被限制使用工具集（不能生成子 Agent），HTTP Hook 有 URL 白名单。这些限制是必要的——没有它们，一个被对抗性提示注入的 Hook 可以通过 HTTP 请求泄露内部数据，或通过 Agent 递归消耗所有资源。

代价是灵活性降低——开发者不能在 Agent Hook 中使用某些高级工具，不能在 HTTP Hook 中访问任意 URL。安全边界以牺牲灵活性为代价。

代价三：**Hook 触发时机分散**

27 个触发点分散在数千行代码中。调试 Hook 行为需要理解每个触发点的精确时机和上下文。例如，`PreCompact` 和 `PostCompact` 的区别是什么时候触发？`Stop` 和 `StopFailure` 的边界在哪里？

缓解方案：`HOOK_EVENTS` 数组本身是一张"触发点地图"——所有触发点集中定义在一处，开发者可以快速查找需要的时机。

踩坑指南——Hooks 中的常见错误

陷阱一：在高频触发点使用 **agent Hook**

`PreToolUse` 在每次工具调用前触发。一次典型任务可能触发 30-50 次工具调用。如果 `PreToolUse` 配置了 agent Hook，每次调用前增加 10-60 秒——总延迟可能从几分钟暴增到数十分钟。

❌ 错误做法：为 `PreToolUse` 配置 agent Hook "检查工具调用是否安全"。
✓ 正确做法：高频触发点（PreToolUse、PostToolUse）只用 command Hook。agent Hook 保留给 `Stop`、`PreCompact` 等低频触发点。

陷阱二：**HTTP Hook 未配置 URL 白名单**

`getHttpHookPolicy` 的 `allowedUrls` 如果为空或配置过于宽松（如 `https://*`），对抗性提示注入可以在对话中注入恶意 URL，Hook 会向这个 URL 发送包含内部数据的请求。

❌ 错误做法：HTTP Hook 配置 `allowedUrls: ["*"]` 或不配置白名单。
✓ 正确做法：精确配置白名单（如 `allowedUrls: ["https://internal-api.company.com/*"]`），只允许已知的安全 URL。

陷阱三：**Hook** 返回值格式错误被静默忽略

Hook 的返回值有严格的结构要求。如果返回的 JSON 格式不正确（如字段名拼写错误），Harness 静默忽略返回值，行为与 Hook 未触发一样——不会报错，只是不生效。

❌ 错误做法：Hook 返回自定义格式的 JSON，期望 Harness 能"理解"意图。
✓ 正确做法：严格按照 Hook 类型的返回值规范返回。command Hook 的 stdout 解析规则、agent Hook 的 `ok` 字段格式、http Hook 的响应结构——每种类型有独立的解析逻辑。

实证——Stop Hook 验证任务完成的完整路径

一个 Stop Hook 从触发到影响 Harness 继续/终止决策，经过了完整的 Agent 对话循环。这条路径验证了 Hooks 作为"神经末梢"的实际运作。

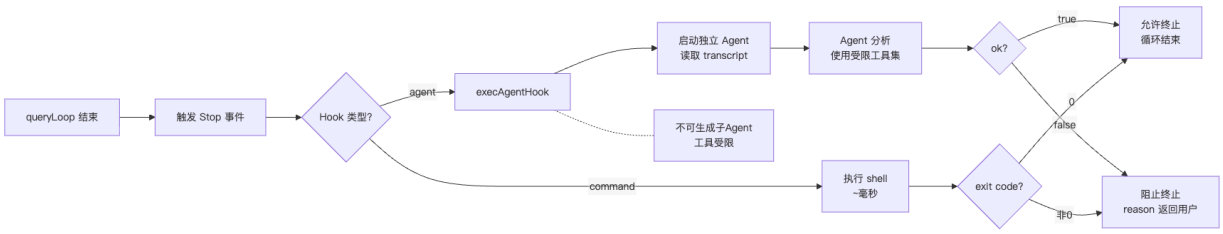
触发：当查询循环结束时（详见第 4 章的 `handleStopHooks`），Harness 检查 `Stop` 事件（`HOOK_EVENTS` 中的第 8 个触发点）是否有配置的 Hook。

调度：Hook 类型匹配到 `agent`，触发 `execAgentHook`（`src/utils/hooks/execAgentHook.ts:36`）。Harness 准备 Agent 的输入——当前对话历史的 transcript 路径和任务计划。

执行：Agent Hook 启动独立的多轮 AI 对话。系统提示词定义角色（`src/utils/hooks/execAgentHook.ts:108`）：验证停止条件。Agent 读取 transcript 文件，检查代码库变更，判断任务是否真正完成。工具集经过过滤（`src/utils/hooks/execAgentHook.ts:98`），移除可能生成子 Agent 的工具。

返回：Agent 通过专用工具返回判定：`ok: true`（任务完成）或 `ok: false`（任务未完成，附带 reason）。如果 `ok: false`，Harness 不终止循环，继续执行。

图 **10-2**：**Stop Hook** 完整执行路径



这条路径验证了 Hooks 的核心设计价值：外部代码（用户配置的 Hook）在 Harness 最关键的决策点（"循环是否应该终止"）插入了自定义逻辑，而核心循环代码无需任何修改。内部注册的 `registerPostSamplingHook`（`src/utils/hooks/postSamplingHooks.ts:31`）走同一条路径——内部和外部扩展共享同一套机制。

本章主成分：Hooks 系统的本质

本质：在 Harness 生命周期的 27 个预定义时机点提供双向接口——外部代码可以观察 Harness 状态，也可以改变 Harness 决策，而无需修改核心代码。

关键机制：

- `HOOK_EVENTS`：27 个触发点覆盖完整生命周期
- 4 种执行方式（command/prompt/agent/http）按能力层次递进
- Agent Hook 有工具限制（不能生成子 Agent），HTTP Hook 有 URL 白名单
- `registerPostSamplingHook` 内部注册机制与用户 Hooks 并列

适用边界：

- ✓ 适合：需要外部审计、权限扩展或上下文注入的企业级 Agent 系统
- ✓ 适合：需要在 Harness 决策点插入自定义逻辑的集成场景
- ✗ 不适合：核心逻辑（核心逻辑不应依赖 Hook，Hook 是可选扩展层）
- ✗ 不适合：高频低延迟场景（agent Hook 延迟不可控）

与其他模式的关系：

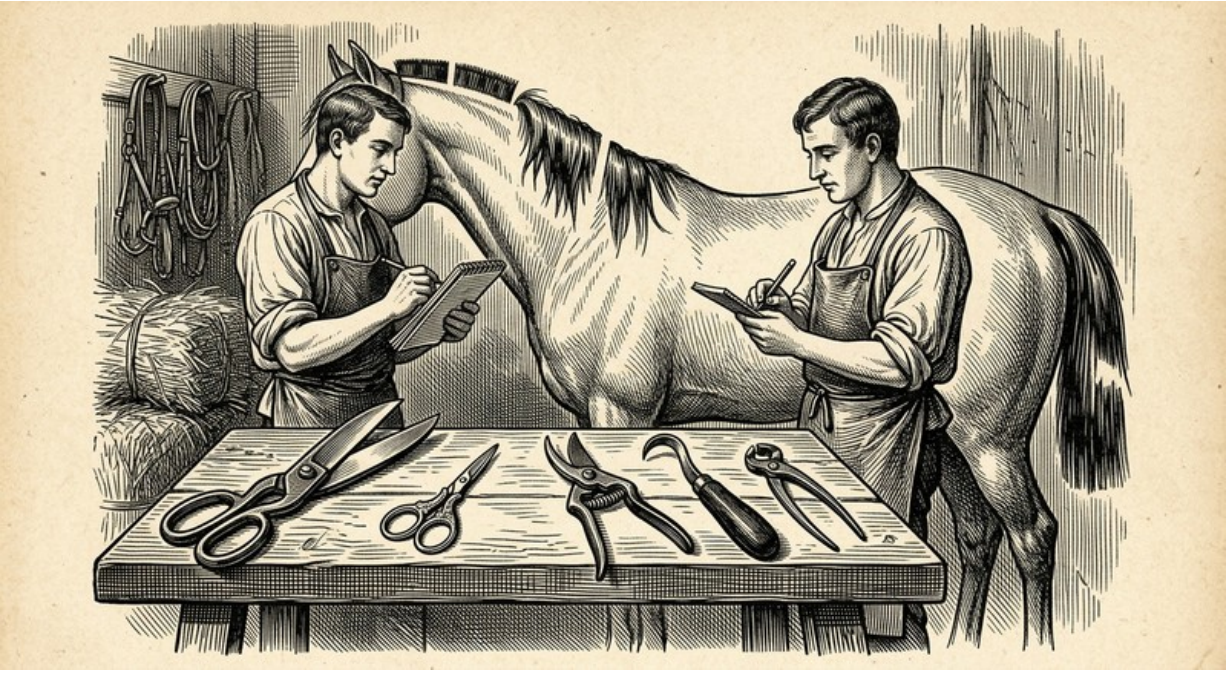
- 第 9 章（权限系统）的 `PermissionRequest` 事件可以被 Hook 接管
- 第 4 章（单轮执行）的 `handleStopHooks` 是 `Stop` Hook 的内部实现
- 第 13 章（多智能体）的 `SubagentStart/Stop` 触发点用于多智能体 Hooks

你能做什么

- 为 **PostToolUse** 注册 **command Hook** 实现审计日志。每次工具调用后写入外部日志——延迟毫秒级，不影响性能。
- 为 **PermissionRequest** 注册 **http Hook** 连接企业权限系统。返回 allow/deny 决策，让 Harness 的权限判断由外部系统驱动（详见第 9 章权限谱系）。
- 为 **Stop** 注册 **agent Hook** 验证任务完成。让 AI Agent 读取对话历史，判断任务是否真正完成——这是 command/prompt/http 都无法做到的能力。
- 为 **PreToolUse** 注册 **prompt Hook** 动态注入上下文。在工具调用前通过单次 AI 推理注入当前环境信息——延迟约 1-2 秒，换取动态感知能力。
- 在高频触发点只使用 **command Hook**。PreToolUse 每次工具调用都触发——agent Hook 在此处的延迟不可接受。agent Hook 保留给 `Stop` 等低频触发点。
- 为 **HTTP Hook** 配置 **URL** 白名单。精确配置 `allowedUrls`（如 `https://internal-api.company.com/*`），防止对抗性提示注入修改目标 URL。
- 参考 **`HOOK_EVENTS`** 的 **27** 个触发点，找到你需要干预的精确时机。不要猜测触发时机——所有触发点在 `coreTypes.ts` 中集中定义。

下一章导读：本章看到了 Hooks 如何在 Harness 生命周期的 27 个时机点提供外部扩展能力。但 Hooks 只影响行为决策——当上下文窗口不够用时，Harness 需要一种完全不同的机制来压缩对话历史。第 11 章将进入上下文管理的核心——五把"剪刀"如何在保留关键信息的同时压缩 token 消耗。

第六篇：记忆管理



五把剪刀，各有用途。这一篇深入 **Claude Code** 的压缩策略矩阵——为什么并存五种压缩方式，每种编码了什么模型能力假设，以及上下文重置如何兼顾缓存命中。

第 11 章：上下文的五把剪刀

> "一个上下文窗口的死亡方式不只有一种。5 种剪刀，5 种死法，5 种救法。"

同一个问题——"上下文太长了怎么办"——Claude Code 有 5 种不同的答案，每种答案针对不同的失败场景。理解为什么需要 5 种而非 1 种，比知道每种怎么用更重要。读完本章，你将理解每种压缩策略背后编码的"模型能力假设"，以及 queryLoop 中从轻到重的防线链如何在不丢失关键信息的前提下压缩 token 消耗。

问题——为什么需要 5 种压缩策略而非 1 种

如果上下文管理只有一个问题，就不需要 5 种策略。5 种策略的存在本身说明这是 5 个独立问题：

- **token** 超限——对话太长，超过上下文窗口，需要全量摘要
- 旧工具结果占位——早先轮次的文件读取、命令输出占据大量空间，但模型已经不需要它们了
- **cache TTL** 过期——超过 1 小时后，服务端缓存已经失效，继续发送旧的工具结果是纯浪费
- 历史轮次边界——某些历史消息块可以整段删除而不影响当前任务的连贯性
- **API 413** 响应——即使所有预防措施都做了，API 仍然返回"请求过长"，需要紧急压缩

queryLoop 中的执行顺序揭示了策略间的逻辑关系——源码注释说明："Apply snip before microcompact (both may run — they are not mutually exclusive)"（译：在 microcompact 之前应用 snip，两者可以同时运行——它们不是互斥的），紧接着 "Apply microcompact before autocompact"（译：在 autocompact 之前应用 microcompact）。轻量级策略先行，重量级策略托底。如果轻量级策略已经释放了足够的 token，重量级策略就不会触发。

策略	目标场景	触发频率	压缩深度
snipCompact	历史轮次边界	每轮检查	浅（删除整块消息）
microCompact	旧工具结果占位	每轮检查	中（清除工具输出）
contextCollapse	可折叠上下文投影	每轮检查	中（只读投影，不修改原始消息）
autoCompact	token 超限	阈值触发	深（全量对话摘要）
timeBasedMC	cache TTL 过期	时间阈值	中（清除过期工具结果）

原则 **11.1**：压缩策略按问题分而非按算法分 — 上下文压缩策略必须针对不同的失控场景分别设计，而非用一种"万能摘要"应对所有情况。禁止将不同触发条件的压缩合并为同一策略——合并意味着要么漏掉某些场景，要么在不必要的时候触发。

黄金法则——每种策略编码一个"模型能力假设"

每种压缩策略不是算法选择，而是一条工程断言——"在这种条件下，模型的记忆能力边界在这里"。源码中的常量就是这些断言的数值表达。

`AUTOCOMPACT_BUFFER_TOKENS = 13000` 是 autoCompact 的核心假设：给模型留 13000 token 的缓冲区来生成摘要。这个数字编码了断言"模型需要至少 13000 token 的空间才能完成一次高质量的对话摘要"——如果缓冲区太小，摘要生成会被截断，丢失关键信息。

`timeBasedMC` 的 60 分钟阈值直接映射到 cache TTL 的工程假设。注释说明了选择："60 is the safe choice: the server's 1h cache TTL is guaranteed expired"（译：60 是安全选择——服务端 1 小时的缓存 TTL 保证已过期）。这不是关于模型能力的假设，而是关于基础设施的假设——cache 过期后，旧的工具结果既占据 token 又不提供缓存加速效果。

`POST_COMPACT_MAX_FILES_TO_RESTORE = 5` 编码了断言"模型在压缩边界后需要哪些上下文才能继续工作"——最多恢复 5 个最近读取的文件。数字 5 来自工程经验：太少则模型丢失当前工作文件，太多则恢复成本抵消了压缩收益。

策略	编码的"能力假设"	源码常量
autoCompact	模型生成摘要需要 13000 token 缓冲区	`AUTOCOMPACT_BUFFER_TOKENS=13000`
autoCompact	连续 3 次失败应停止重试	`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES=3`
autoCompact	压缩后需要 50000 token 预算恢复上下文	`POST_COMPACT_TOKEN_BUDGET=50000`
microCompact	只有 8 类工具的结果值得清除	`COMPACTABLE_TOOLS`（8 种）
timeBasedMC	cache TTL 过期后旧内容无缓存价值	`gapThresholdMinutes=60`

原则 **11.2**：压缩参数即模型能力声明 — 每个压缩阈值和缓冲区必须基于对模型能力的工程判断，而非随意设置。如果模型能力变化（如上下文窗口扩大），这些参数必须相应调整——它们不是常量，是变量。

适用场景——什么条件下哪种压缩触发

5 种策略按照"触发频率 × 压缩深度"形成从轻到重的梯度。

snipCompact（浅层、每轮检查）——检查历史消息中是否存在可以整段删除的轮次边界。适合多轮对话中早期轮次与当前任务无关的场景。触发条件是 Feature Flag 控制。

microCompact（中层、每轮检查）——检查工具结果是否可以被清除。`COMPACTABLE_TOOLS` 白名单定义了 8 类可清除的工具：FileRead、Shell、Grep、Glob、WebSearch、WebFetch、FileEdit、FileWrite。白名单外的工具结果（如自定义 MCP 工具）不会被清除。

contextCollapse（中层、每轮检查）——与前两种不同，collapse 不修改原始消息数组，而是在读取时投影一个"折叠视图"。折叠后的消息不在 REPL 数组中，而在 collapse store 中——这意味着折叠可以跨轮次持久化。

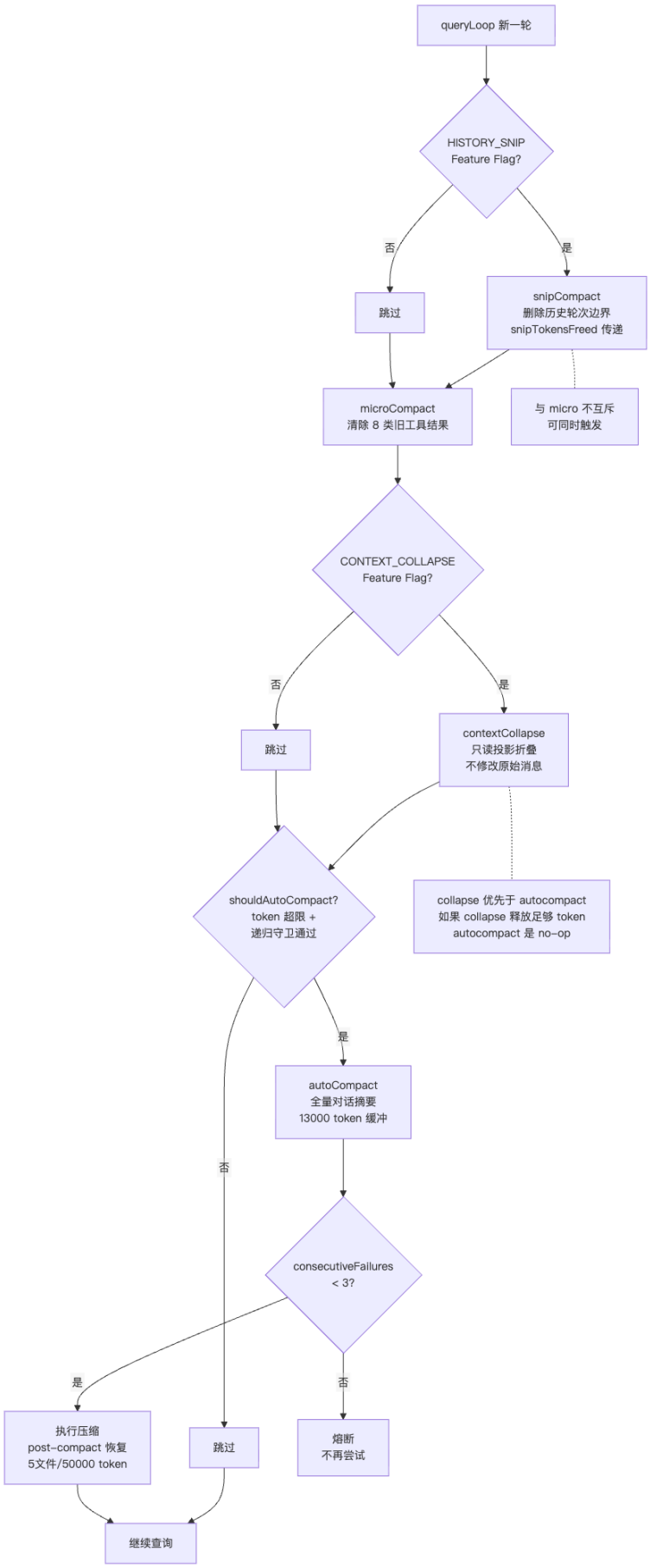
autoCompact（深层、阈值触发）——当 token 使用量超过 `getAutoCompactThreshold(model)` 时触发。可以通过 `DISABLE_AUTO_COMPACT` 环境变量完全禁用。`isAutoCompactEnabled()` 让这个开关独立于其他策略。

timeBasedMC（中层、时间阈值触发）——默认关闭（`enabled: false`）。适合会话生命周期超过 1 小时的长会话场景，清除 cache TTL 过期后的旧工具结果。需要用户手动配置才能启用。

工作原理——5 种策略的执行机制

5 种策略在 queryLoop 中以固定顺序执行——这个顺序不是随意的，而是经过设计的防线链。源码注释清晰地标记了每一步的检查点：`query_snip_start` → `query_microcompact_start` → `query_autocompact_start`。

图 **11-1**：**queryLoop** 中的压缩决策树



第一步：snipCompact

检查历史消息中是否存在轮次边界。如果存在，删除边界之前的消息，计算释放的 token 数（`snipTokensFreed`）。这个数字被传递给后续的 autoCompact——因为 snip 删除了消息，但存活的 assistant 消息的 usage 字段仍反映删除前的 token 计数。源码注释解释："Snip removes messages but the surviving assistant's usage still reflects pre-snip context, so tokenCountWithEstimation can't see the savings."（译：Snip 删除了消息但存活的 assistant 的 usage 仍反映 snip 前的上下文，所以 tokenCountWithEstimation 看不到节省量）。

第二步：**microCompact**

清除白名单中 8 类工具的旧结果。被清除的结果替换为占位消息 `'[Old tool result content cleared]'`。microCompact 与 snip 不是互斥的——同一轮可以两者都触发。microCompact 在 autoCompact 之前运行，是因为如果轻量级清除已经释放了足够 token，就不需要触发代价更高的全量摘要。

第三步：**contextCollapse**

与前两种策略不同，collapse 不修改原始消息数组。它在读取时投影一个"折叠视图"——折叠后的消息存储在独立的 collapse store 中。这个设计使得折叠可以跨轮次持久化：每次进入 queryLoop 时，`projectView()` 重放提交日志生成当前视图。源码注释解释了为什么 collapse 在 autoCompact 之前运行："Runs BEFORE autocompact so that if collapse gets us under the autocompact threshold, autocompact is a no-op and we keep granular context instead of a single summary."（译：在 autocompact 之前运行，这样如果 collapse 让 token 数低于 autocompact 阈值，autocompact 就不需要运行，我们保留细粒度的上下文而非一个摘要）。

第四步：**autoCompact**

最重量级的策略——生成完整的对话摘要，替换整个历史。触发条件是 token 使用量超过模型特定的阈值。执行前有 3 层递归守卫：`session_memory` 和 `compact` 查询源不触发（防止死锁），`marble_origami`（上下文 Agent）不触发（防止破坏主线程的已提交日志）。`REACTIVE_COMPACT` Feature Flag 可以关闭主动 autoCompact，改为仅响应 API 413 错误触发。

post-compact 恢复

压缩完成后，系统恢复最近读取的最多 5 个文件，使用 50000 token 的预算。这是压缩后"重建模型工作记忆"的关键步骤——没有它，模型在摘要后无法继续工作。

策略	触发条件	压缩范围	代价
snipCompact	Feature Flag + 轮次边界	删除整块历史消息	无 API 调用，丢失旧消息
microCompact	每轮检查 + 白名单工具	清除工具输出内容	无 API 调用，丢失工具结果
contextCollapse			无 API 调用，可逆

策略	触发条件	压缩范围	代价
	Feature Flag + 可折叠消息	只读投影，不修改原始	
autoCompact	token 超阈值 + 递归守卫	全量对话摘要	一次 API 调用，不可逆
timeBasedMC	cache TTL 过期 + 手动启用	清除过期工具结果	无 API 调用，默认关闭

权衡——多策略并存的 3 个设计代价

决策维度	选择 **A**（本系统）	选择 **B**	核心权衡
熔断机制	3 次失败停止	无限重试	保护 API 资源 vs 放弃压缩
策略并发	snip + micro 可同时触发	严格互斥	更细粒度控制 vs 行为不可预测
时间阈值	默认关闭（实验性）	默认开启	安全验证 vs 功能可达性

代价一：熔断机制是真实生产事故的产物

`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3` 不是理论设计——源码注释记录了真实的生产数据："BQ 2026-03-10: 1,279 sessions had 50+ consecutive failures (up to 3,272) in a single session, wasting ~250K API calls/day globally."（译：BQ 2026-03-10：1279 个会话有 50 次以上连续失败（最多 3272 次），每天全球浪费约 25 万次 API 调用）。没有熔断机制，autoCompact 会无限重试，每次都消耗 API token 但无法成功压缩。3 次上限是"快速放弃"与"给足够机会"之间的平衡。

代价二：多策略并发增加行为不可预测性

snip 和 micro 可以在同一轮都触发——用户可能在一轮中看到上下文被两种不同的策略同时压缩。这种并发设计让压缩更彻底（轻量级策略组合可能释放足够 token 避免重量级策略），但调试时需要同时考虑两种策略的交互效果。

代价三：时间阈值默认关闭限制功能可达性

timeBasedMC 是针对"cache TTL 过期后旧工具结果无缓存价值"的优化，但默认关闭。`enabled: false` 说明这是一个实验性功能——在多策略环境中引入新策略需要验证它不会与其他策略冲突。代价是用户需要手动配置才能获益。

踩坑指南——压缩策略中的常见错误

陷阱一：**autoCompact** 在 **compact** 查询源中触发死锁

`shouldAutoCompact` 有递归守卫：`session_memory` 和 `compact` 查询源直接返回 false。原因很直接——autoCompact 本身通过 forkedAgent 执行压缩，如果压缩查询再次触发 autoCompact，会无限递归。守卫不是"防御性编程"，是真实会触发的死锁预防。

❌ 错误做法：在自定义的 forkedAgent 中忽略查询源（`querySource`）参数，让压缩逻辑在所有查询源中无条件触发。
✓ 正确做法：所有压缩逻辑必须检查查询源——`compact`、`session_memory`、`marble_origami` 等查询源应跳过压缩。

陷阱二：期望 **microCompact** 清除自定义工具结果

`COMPACTABLE_TOOLS` 只包含 8 种内置工具。自定义 MCP 工具、自定义 Agent 工具的结果不在白名单中——microCompact 不会清除它们。如果一个自定义工具返回了 5000 token 的输出，它将一直保留在上下文中，直到 autoCompact 全量摘要。

❌ 错误做法：假设所有工具输出都会被 microCompact 自动管理，不控制自定义工具的输出长度。
✓ 正确做法：为自定义工具实现结果截断。如果工具输出可能很长，在工具内部限制返回长度——microCompact 的白名单不会扩展到自定义工具。

陷阱三：忽视熔断后上下文继续增长

autoCompact 连续失败 3 次后熔断，不再尝试压缩。但对话继续进行，token 持续增长。如果不监控熔断状态，上下文会一直增长到 API 的 `blocking_limit`，最终整个会话崩溃。

❌ 错误做法：在 autoCompact 熔断后继续正常使用，不调整行为。
✓ 正确做法：监控 `consecutiveFailures` 状态。如果熔断触发，考虑手动压缩（`/compact` 命令）或开新会话。

实证——从 token 计数到压缩触发的决策链

一次 autoCompact 触发的完整路径可以从 `calculateTokenWarningState` 追踪到 `compactConversation`。

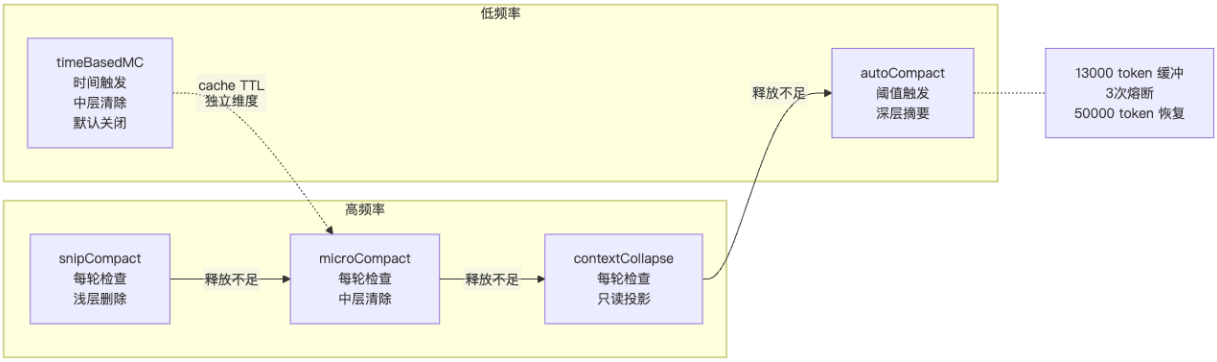
入口：`calculateTokenWarningState`（`src/services/compact/autoCompact.ts:93`）计算当前 token 使用百分比。它返回三个状态：`percentLeft`（剩余百分比）、`isAboveWarningThreshold`（超过警告阈值，使用 20000 token 缓冲）、`isAboveAutoCompactThreshold`（超过自动压缩阈值，使用 13000 token 缓冲）。

触发判断：`shouldAutoCompact`（`src/services/compact/autoCompact.ts:160`）执行 3 层递归守卫检查——查询源不是 `session_memory` 或 `compact`（防死锁），不是 `marble_origami`（防破坏主线程），Feature Flag 未禁用。如果全部通过，检查 token 数是否超过阈值，减去 `snipTokensFreed`（snip 已经释放的 token）。

执行：`autoCompactIfNeeded`（`src/services/compact/autoCompact.ts:241`）检查 `consecutiveFailures`——如果小于 3，调用 `compactConversation` 生成摘要。摘要使用 20000 token 的输出预算，保留系统提示词和用户上下文。

恢复：压缩后，`POST_COMPACT_MAX_FILES_TO_RESTORE = 5` 和 `POST_COMPACT_TOKEN_BUDGET = 50000`（`src/services/compact/compact.ts:122-123`）控制资源恢复。系统重新读取最近 5 个文件，使用最多 50000 token 的预算重建模型的工作记忆。

图 **11-2**：**5** 种策略的能力/频率矩阵



这条路径验证了多策略并存的核心理念：轻量级策略（snip、micro）在每轮都运行，如果它们释放了足够 token，重量级策略（autoCompact）就不需要触发。只有当所有轻量级策略都释放不足时，autoCompact 才作为最后一道防线执行全量摘要。

本章主成分：五种压缩策略的本质

本质：5 种策略分别针对 token 超限（autoCompact）、旧工具结果占位（microCompact）、cache TTL 过期（timeBasedMC）、历史轮次边界（snipCompact）和可折叠上下文（contextCollapse）5 种独立的上下文失控场景。每种策略的参数编码了对模型能力和基础设施的工程假设。

关键机制：

- 执行顺序：轻到重的防线链（snip → micro → collapse → autoCompact）
- 熔断机制：`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES=3`（生产数据驱动）
- 工具白名单：`COMPACTABLE_TOOLS` 只覆盖 8 种内置工具
- 缓冲区设计：13000 token 摘要缓冲 + 50000 token 恢复预算

适用边界：

- ✓ 适合：长会话多轮对话的 Agent 系统
- ✓ 适合：需要在不同压缩深度间灵活选择的场景
- ✗ 不适合：短会话单轮任务（不需要任何压缩策略）

与其他模式的关系：

- 本章的压缩边界是第 12 章（forkedAgent 上下文隔离）的前置概念——fork 前需要决定传递多少上下文

- 第 3 章（QueryEngine 循环）是压缩策略的触发宿主——每轮循环都检查是否需要压缩
- 第 10 章（Hooks）的 `PreCompact` 和 `PostCompact` 触发点在压缩前后提供拦截机会

你能做什么

- 监控上下文使用百分比。`calculateTokenWarningState` 的 `percentLeft` 可以在用户界面中显示——让用户知道压缩何时可能触发。
- 为自定义工具实现结果截断。microCompact 只覆盖 8 种内置工具——自定义工具的长输出不会被自动清除，在工具内部限制返回长度。
- 设置合理的摘要缓冲区。`AUTOCOMPACT_BUFFER_TOKENS=13000` 是保守选择——如果实现类似系统，宁可多留空间也不要让摘要生成被截断。
- 加入熔断机制。参考 `MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES=3`——生产数据证明没有熔断会浪费大量 API 调用。
- 长会话场景启用时间阈值压缩。如果会话生命周期超过 1 小时，timeBasedMC 可以清除 cache TTL 过期后的旧工具结果——但需要手动启用。
- 检查你的 **COMPACTABLE_TOOLS** 等价配置。不是所有工具结果都值得被 microCompact 处理——只有可能产生大输出的工具才需要进入白名单。
- 设计压缩后的资源恢复策略。参考 `POST_COMPACT_MAX_FILES_TO_RESTORE=5` 和 `POST_COMPACT_TOKEN_BUDGET=50000`——没有恢复，模型在摘要后无法继续工作。

下一章导读：本章看到了 5 种压缩策略如何在 queryLoop 中逐级收缩上下文。但当 Harness 需要同时执行多个子任务时，压缩不再是"在同一个上下文中腾空间"的问题——而是"如何为每个子任务隔离独立的上下文"。第 12 章将展示 forkedAgent 如何通过上下文隔离实现并行执行，以及这种隔离带来的"上下文分裂"代价。

第 12 章：隔离与交接

> "隔离不是遗忘，是选择性遗忘。遗忘的是历史，记住的是规则和能力。"

fork 一个子智能体，它的对话历史是空的，但 Prompt Cache 是热的——隔离了让模型分心的历史对话，保留了让 API 调用省钱的缓存。"清空"与"继承"同时发生。读完本章，你将理解 CacheSafeParams 如何定义这条分界线，以及 MEMORY.md 四层记忆架构如何在会话间传递"不可从项目状态推导的信息"。

问题——上下文重置和 Prompt Cache 如何兼顾

forkedAgent（分支智能体）必须同时满足两个看似矛盾的目标：清空对话历史（防止历史噪音干扰子任务）和保持 Prompt Cache 热（不浪费已缓存的 token）。

源码注释说明了 helper 的定位："Helper for running forked agent query loops with usage tracking"（译：用于运行分支智能体查询循环并跟踪使用情况的辅助工具）。矛盾的核心在于 API 缓存机制——缓存 key 由系统提示词、工具、模型、消息前缀和思考配置组成。如果 fork 时完全重置上下文，缓存 key 全部改变，每次 fork 都是"冷启动"；如果不重置上下文，子任务会被父智能体的历史对话干扰。

解法在 `CacheSafeParams` 的注释中直接揭示："Parameters that must be identical between the fork and parent API requests to share the parent's prompt cache. The Anthropic API cache key is composed of: system prompt, tools, model, messages (prefix), and thinking config."（译：在分支和父智能体 API 请求之间必须完全相同的参数，以共享父智能体的提示词缓存。Anthropic API 的缓存 key 由以下部分组成：系统提示词、工具、模型、消息前缀和思考配置）。

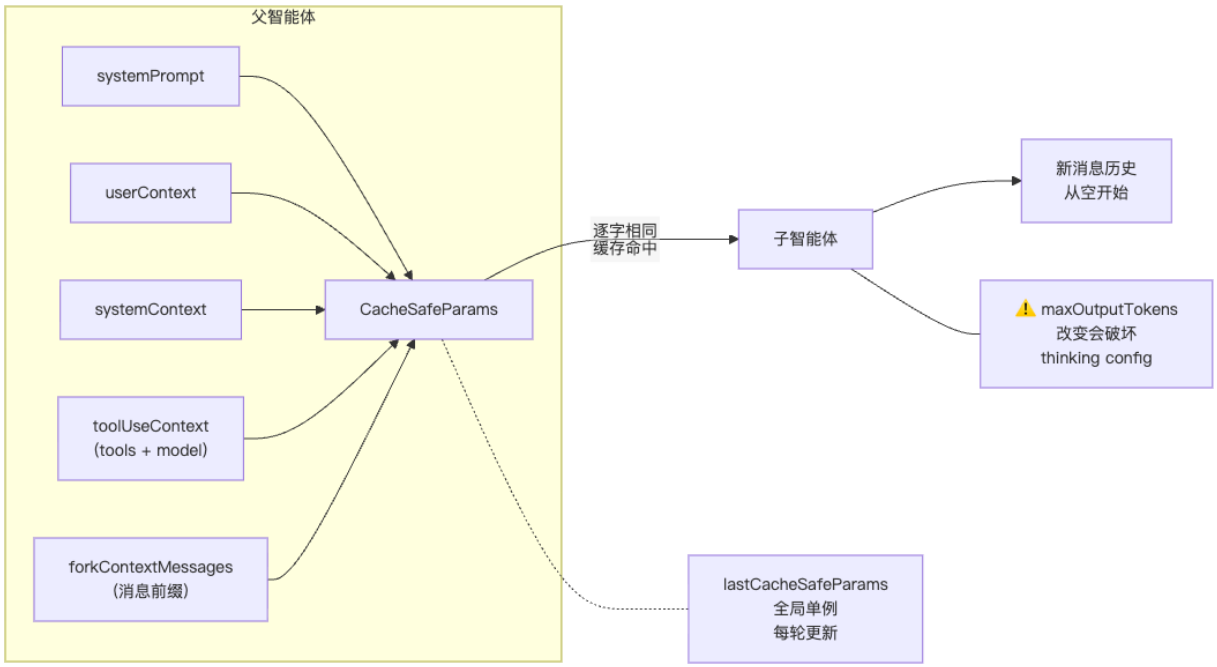
目标	操作	效果
清空历史	子智能体的消息历史从空开始	子任务不受父对话干扰
保持缓存	5 个 CacheSafeParams 逐字相同	API 缓存 key 不变，命中缓存
两者兼得	清空消息但保持系统级参数	隔离历史 + 继承缓存

原则 **12.1**：上下文隔离不等于缓存失效 — Agent 系统的上下文隔离必须区分"对话历史"和"API 缓存参数"两个维度。清空对话历史时，禁止同时改变影响缓存的参数——否则隔离的代价是缓存完全失效。

黄金法则——隔离历史，继承能力

forkedAgent 的核心原则是"对话历史归子智能体，Prompt Cache 属于 API 边界"——清空历史不等于清空缓存。`CacheSafeParams` 类型定义了这条分界线上的 5 个参数。

图 **12-1**：**forkedAgent** 参数继承关系



5 个参数各有明确的缓存角色：

- **systemPrompt**——系统提示词，必须与父智能体逐字相同。注释直接标记："must match parent for cache hits"（译：必须与父智能体匹配才能命中缓存）
- **userContext**——用户上下文，注入到消息前缀，影响缓存
- **systemContext**——系统上下文，追加到系统提示词，影响缓存
- **toolUseContext**——工具配置和模型选择，包含 tools 和 model
- **forkContextMessages**——父智能体的消息历史，作为缓存前缀传入。注释说明："Parent context messages for prompt cache sharing"（译：用于提示词缓存共享的父上下文消息）

`lastCacheSafeParams` 是一个全局单例——主循环每轮结束后写入，供后续所有 fork 复用。这意味着 Stop Hook 验证、记忆压缩、技能执行等所有 fork 场景都共享同一套缓存参数，不需要每个调用方重新传递。

原则 **12.2**：交接的内容是"能力"而非"记忆" — fork 时子智能体继承的是系统提示词、工具和模型（能力），而非对话历史（记忆）。对话历史是任务上下文，不应跨 fork 传递——每个子任务必须有独立的记忆空间。

适用场景——什么任务应该 fork 子智能体

需要独立对话历史但共享能力配置的任务都适合 fork。`forkLabel` 字段记录了 Claude Code 内部的 fork 使用场景——注释说明："Label for analytics (e.g.,

'session_memory', 'supervisor')"（译：用于分析的标签，如 'session_memory'、'supervisor'）。

场景	forkLabel	说明
记忆压缩	`session_memory`	压缩对话摘要，详见第 11 章
Stop Hook 验证	`supervisor`	验证任务是否完成，详见第 10 章
技能执行	`skill`	执行独立技能的 AI 对话
自动记忆提取	`extract_memories`	从对话中提取值得记忆的内容
上下文折叠	`marble_origami`	可折叠上下文的独立 Agent

每种场景都需要独立历史（子任务不应被父对话干扰），但共享相同的系统能力（工具、模型、提示词）。fork 的价值恰恰在于"能力继承 + 历史隔离"——如果不需要独立历史，直接在主循环中执行即可。

工作原理——fork 机制和分层记忆

fork 机制和分层记忆架构共同构成了 Claude Code 的"记忆管理系统"——前者解决当前对话中跨任务的上下文隔离，后者解决跨会话的信息持久化。

fork 机制

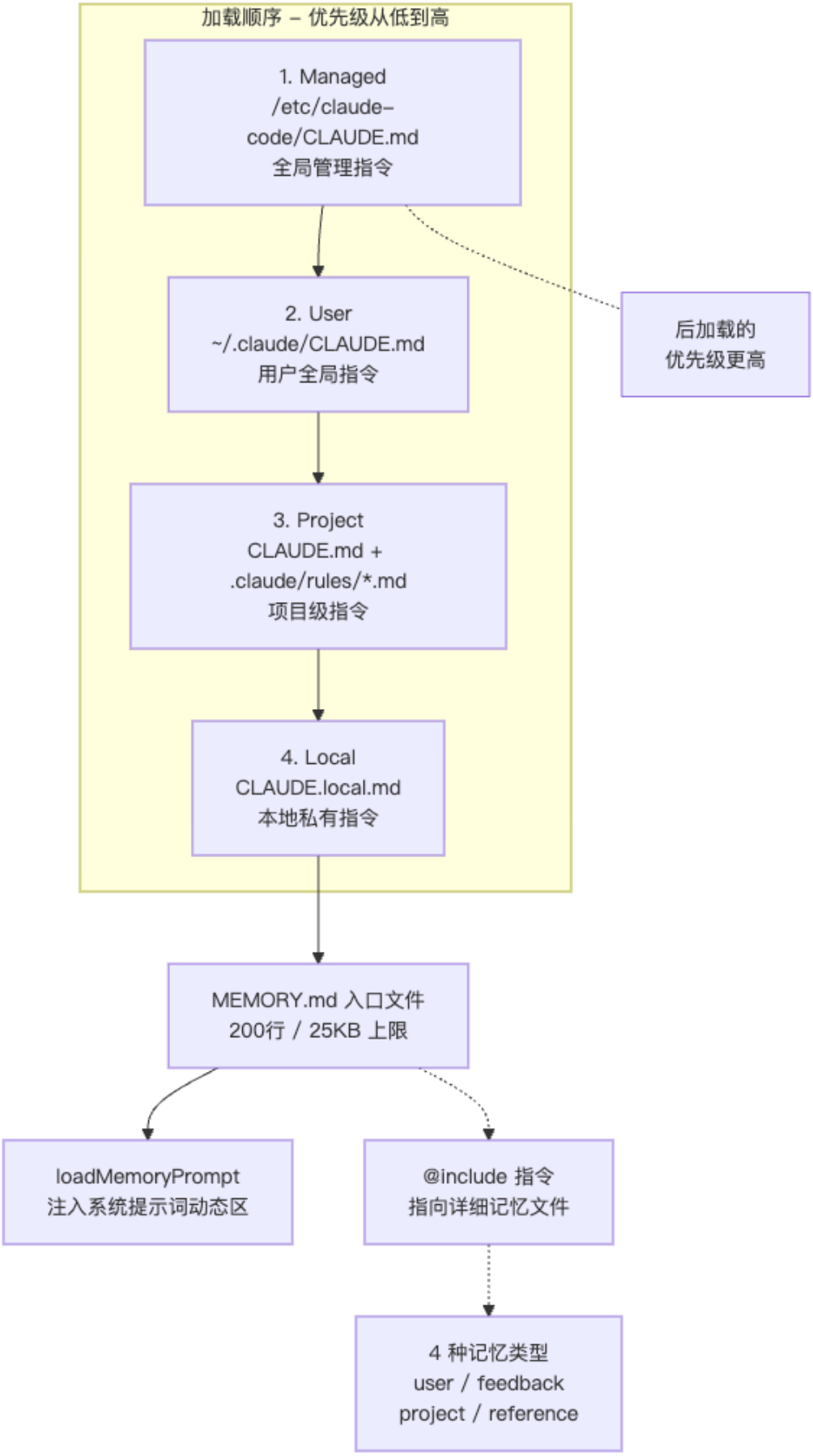
`runForkedAgent` 是 fork 的执行入口。它复用 `CacheSafeParams`（5 个参数完全相同），但消息历史从空开始——子智能体只能看到自己的任务提示和工具调用结果。`forkContextMessages` 以只读方式传入作为缓存前缀，子智能体不能修改这部分历史——这个设计保持了 API 缓存 key 的一致性。

四层记忆架构

源码注释定义了四层记忆的加载顺序："Files are loaded in the following order: 1. Managed memory — Global instructions for all users; 2. User memory — Private global instructions for all projects; 3. Project memory — Instructions checked into the codebase; 4. Local memory — Private project-specific instructions."（译：文件按以下顺序加载：1. 托管记忆——所有用户的全局指令；2. 用户记忆——所有项目的私有全局指令；3. 项目记忆——检入代码库的指令；4. 本地记忆——私有的项目特定指令）。

加载顺序是优先级的逆序——后加载的优先级更高。"Files are loaded in reverse order of priority, i.e. the latest files are highest priority with the model paying more attention to them."（译：文件按优先级的逆序加载，即最后加载的文件优先级最高，模型会更关注它们）。

图 12-2：MEMORY.md 四层记忆架构



MEMORY.md 入口文件

`ENTRYPOINT_NAME = 'MEMORY.md'`——记忆系统的入口文件。它有严格的大小限制：`MAX_ENTRYPOINT_LINES = 200`（最多 200 行）和 `MAX_ENTRYPOINT_BYTES = 25_000`（最多 25000 字节）。注释说明了为什么是 25KB："~125 chars/line at 200 lines. At p97 today; catches long-line indexes that slip past the line cap (p100 observed: 197KB under 200 lines)."（译：200 行约每行 125 字符。当前处于 p97 水平；捕获了绕过行数限制的长行索引（观察到 p100：200 行内 197KB））。

这个数字是真实数据驱动的——只限制行数不够（一行可以极长），必须同时限制字节数。25KB 是"足够存放重要记忆"与"不消耗过多 token"之间的平衡点。

4 种记忆类型

`MEMORY_TYPES = ['user', 'feedback', 'project', 'reference']`。源码注释定义了记忆的选择标准："Memories are constrained to four types capturing context NOT derivable from the current project state. Code patterns, architecture, git history, and file structure are derivable (via grep/git/CLAUDE.md) and should NOT be saved as memories."（译：记忆被限制为四种类型，捕获从当前项目状态不可推导的上下文。代码模式、架构、git 历史和文件结构是可推导的（通过 grep/git/CLAUDE.md），不应被保存为记忆）。

记忆类型	含义	示例
`user`	用户偏好	"用户喜欢函数式风格"
`feedback`	用户反馈	"上次生成的测试太简单，需要更多边界用例"
`project`	项目级知识	"这个项目使用 Effect-TS 错误处理"
`reference`	参考资料	"React Server Components 的核心限制"

自动记忆提取

`initExtractMemories` 实现了自动记忆提取——从对话中识别值得记忆的内容并写入记忆文件。它是异步的：对话结束后触发，不阻塞主流程。每次会话独立初始化（闭包设计），避免上一次会话的提取状态影响新会话。

权衡——fork 和记忆系统的 3 个设计代价

决策维度	选择 A（本系统）	选择 B	核心权衡
maxOutputTokens	CAUTION 标记，不推荐修改	自由设置	缓存安全 vs 输出控制
入口文件大小	25KB/200 行（p97 数据驱动）	无限制或更大	token 消耗 vs 信息完整性

决策维度	选择 A（本系统）	选择 B	核心权衡
自动提取	闭包隔离，每次会话独立	全局单例	隔离性 vs 实现简单

代价一：**maxOutputTokens** 的缓存陷阱

fork 时设置 `maxOutputTokens` 会意外改变 thinking config。注释用 CAUTION 标记了这个问题："setting this changes both max_tokens AND budget_tokens... a different budget_tokens will invalidate the cache"（译：设置此值会同时改变 max_tokens 和 budget_tokens，不同的 budget_tokens 会使缓存失效）。这个约束非常隐蔽——开发者可能只是想限制子智能体的输出长度，却无意间破坏了所有缓存。

代价二：入口文件大小的精确校准

25KB/200 行的上限基于 p97 数据——即 97% 的用户 MEMORY.md 在此范围内。超过此限制的内容被截断。这意味着极端情况下（项目非常复杂），重要的记忆信息可能被截断丢失。但无限制的代价更严重——如果入口文件不限制大小，它可能消耗数万 token，直接影响上下文窗口的可用空间。

代价三：自动提取的闭包隔离

`initExtractMemories` 每次会话重新初始化，不共享跨会话状态。这保证了隔离性（上一次提取不会影响本次），但代价是每次会话都要重新设置提取环境。如果提取逻辑需要大量初始化（如加载模型配置），重复初始化的成本不可忽略（推断）。

踩坑指南——fork 和记忆系统中的常见错误

陷阱一：**fork** 时修改 **CacheSafeParams** 中的任一参数

注释明确标记："must be identical between the fork and parent API requests"（译：在分支和父智能体 API 请求之间必须完全相同）。哪怕系统提示词中只多了一个空格，缓存就会全部失效。对于频繁 fork 的场景（如每次 Stop Hook 验证），缓存失效意味着每次都付出完整的 API 调用成本。

❌ 错误做法：fork 时"稍微调整"系统提示词，加入子任务特定的指令。
✓ 正确做法：子任务指令通过消息传入（不影响缓存），系统提示词保持逐字相同。子任务的特定需求用用户消息或系统上下文表达——这些不通过 CacheSafeParams 管理。

陷阱二：在记忆文件中记录可推导的信息

memoryTypes 注释明确说明："Code patterns, architecture, git history, and file structure are derivable (via grep/git/CLAUDE.md) and should NOT be saved as memories."（译：代码模式、架构、git 历史和文件结构是可推导的，不应保存为记

忆）。记忆的定位是"不可推导的上下文"——用户偏好、历史决策、反馈。如果记忆中填满了可推导的信息，入口文件的 25KB 配额很快耗尽。

❌ 错误做法：在记忆文件中记录"项目使用 React 框架，路由在 src/routes/"——这些信息通过看代码就能获得。
✓ 正确做法：记录"用户偏好函数式风格而非面向对象"、"上次重构时因为类型安全问题选择了 Zod 而非 io-ts"——这些是不可从代码推导的决策上下文。

陷阱三：**MEMORY.md 超过 25000 字节后重要信息被截断**

入口文件有双重限制（200 行 + 25KB）。超过限制后内容被截断——不在末尾的内容可能被保留，而在末尾的重要信息可能丢失。

❌ 错误做法：将所有记忆内容直接写入 MEMORY.md，不做分层组织。
✓ 正确做法：MEMORY.md 只放索引和最关键的条目（200 行以内），详细内容用 `@include` 指令指向独立的记忆文件。这样即使入口文件截断，详细文件仍可通过 @include 路径找到。

实证——自动记忆提取的完整流程

自动记忆提取从对话结束到下次会话可用，经过了异步的、增量的、闭包隔离的完整路径。

初始化：会话启动时，`initExtractMemories`（`src/services/extractMemories/extractMemories.ts:296`）创建独立的闭包状态——提取队列、已处理的消息 ID 集合、写入频率控制。闭包确保跨会话隔离。

触发：对话 Stop 后，提取机制异步触发。它扫描对话中的关键信息——用户反馈、项目决策、偏好表达。过滤标准是 memoryTypes 的定义：只提取"不可从项目状态推导的信息"。

写入：提取结果写入记忆文件目录。`MEMORY_TYPES` 的 4 种类型决定文件的分类——用户偏好写入 `user` 类型，项目决策写入 `project` 类型。每个记忆文件有 frontmatter 标记类型和元数据。

更新入口：`MEMORY.md` 入口文件（`src/memdir/memdir.ts:34`）更新索引。入口文件受 200 行/25KB 限制约束——如果新增记忆导致超限，较旧的条目被截断。

下次加载：新会话启动时，`loadMemoryPrompt`（`src/memdir/memdir.ts:419`）按四层顺序加载记忆文件，最后加载 MEMORY.md 入口文件。所有记忆内容注入系统提示词的动态区域，成为模型的"先验知识"。`MAX_MEMORY_CHARACTER_COUNT = 40000`（`src/utils/claudemd.ts:92`）控制记忆总量的上限。

这条路径验证了记忆系统的设计理念：记忆不是"把对话历史保存下来"，而是"从对话中提取不可推导的上下文，在下次会话中以最小的 token 代价注入"。入口文件的 25KB

限制、4 种类型的严格分类、闭包隔离的状态管理——每个设计决策都在"信息完整性"和"token 效率"之间寻找精确的平衡点。

本章主成分：隔离与交接的本质

本质：fork 时隔离对话历史（子智能体从零开始），交接 Prompt Cache（5 个参数逐字相同）——"清空"与"继承"同时发生。记忆系统通过 4 层架构和 MEMORY.md 入口文件在会话间传递"不可从项目状态推导的信息"。

关键机制：

- `CacheSafeParams`：5 个参数（systemPrompt/userContext/systemContext/toolUseContext/forkContextMessages）
- `lastCacheSafeParams`：全局单例，每轮更新，所有 fork 共享
- `MEMORY.md`：200 行/25KB 入口文件，`@include` 指向详细文件
- 4 种记忆类型：`user`/`feedback`/`project`/`reference`
- 自动记忆提取：闭包隔离，异步执行，只提取不可推导信息

适用边界：

- ✓ 适合：需要子任务隔离但共享缓存的并行 Agent 系统
- ✓ 适合：需要跨会话记忆持久化的长周期 Agent
- ✗ 不适合：单任务单会话的简单 Agent（不需要 fork 和记忆系统）

与其他模式的关系：

- 本章是第 11 章（压缩策略）的延伸——autoCompact 用 fork 生成摘要
- 第 10 章（Hooks）的 Stop Hook 通过 fork 验证任务完成
- 第 13 章（生成器与评估器）中每个 Agent 都是一次 fork

你能做什么

- 实现 **fork** 时确保 **CacheSafeParams** 逐字相同。系统提示词、用户上下文、系统上下文、工具配置、消息前缀——任何一个字不同都会破坏缓存。
- 避免在 **fork** 时设置 **maxOutputTokens**。它会改变 thinking config（budget_tokens），破坏缓存——用 CAUTION 标记的隐蔽陷阱。
- 用 **forkLabel** 标记不同场景的 **fork**。`session_memory`、`supervisor`、`skill`——标签让调试和监控成为可能。
- 为你的 **Agent** 建立分层记忆系统。区分全局规则（User 层）和项目规则（Project 层），后加载的优先级更高（详见第 5 章记忆加载机制）。
- 把 **MEMORY.md** 入口文件控制在 **200** 行**/25KB** 以内。超出部分用 `@include` 指向详细文件——入口文件是索引，不是全文。

- 在记忆文件中只记录不可推导的信息。用户偏好、历史决策、反馈——代码模式、架构、文件结构可以通过 grep/git 获得不应占用记忆配额。
- 实现自动记忆提取时使用闭包隔离状态。每次会话独立初始化，防止跨会话的状态泄漏。

下一章导读：本章看到了 fork 如何在隔离和继承之间找到平衡。但当 Harness 需要同时运行多个智能体协同工作时，单层的 fork 就不够了——需要一个"调度器"来编排它们的协作和竞争。第 13 章将展示生成器与评估器模式如何让多个智能体在共享上下文中分工协作，以及这种协作带来的"上下文争用"挑战。

第七篇：多智能体

从生成器与评估器的对抗协作，到并行世界的任务分发——这一篇揭示 **Harness** 如何突破单智能体的局限，通过多智能体架构实现更复杂的目标。

第 13 章：生成器与评估器

> "同一个模型，既当选手又当裁判，结果必然偏颇。GAN 用两个网络对抗解决这个问题——Harness 用两个 Agent。"

一个模型负责写代码，另一个模型负责验证——这不是简单的分工，而是一种对抗关系。planAgent 只能看不能改，verificationAgent 的工作不是确认"能运行"，而是"拼命让它崩溃"。理解这种 GAN 式的生成/验证分离，比知道具体有哪些内置智能体更重要。读完本章，你将理解 `disallowedTools` 如何实现比提示词更强硬的能力隔离，以及 Harness 如何用"让模型对抗模型"的方式解决自我评估偏差。

问题——为什么模型不能自己验证自己

模型生成代码后自我评估存在系统性偏差——这不是 bug，是 LLM 的固有特性。模型倾向于对自己的输出过度自信，跳过边界检查，用"看起来对"代替"实际运行了"。

verificationAgent 的系统提示词直接指出了这种偏差："Your job is not to confirm the implementation works — it's to try to break it."（译：你的工作不是确认实现能运行——而是试图让它崩溃）。更关键的是，提示词用了一个专门的章节标题来对抗这种偏差："RECOGNIZE YOUR OWN RATIONALIZATIONS"（译：识别你自己的合理化借口）。

这个章节列出了模型在验证时最常见的 6 种"偷懒借口"：

1. **"代码看起来正确"**——提示词反击："Reading is not verification. Run it."（译：读代码不是验证，运行它）
2. **"实现者的测试已经通过了"**——提示词反击："The implementer is an LLM. Verify independently."（译：实现者是个 LLM，独立验证）
3. **"大概没问题"**——提示词反击："Probably is not verified. Run it."（译：大概不是已验证，运行它）
4. **"启动服务器看看代码"**——提示词反击："Start the server and hit the endpoint."（译：启动服务器并请求端点）
5. **"我没有浏览器"**——提示词反击：检查是否有 MCP 工具（chrome/playwright），如果存在就使用，不要编造"做不到"的理由
6. **"这太耗时了"**——提示词反击："Not your call."（译：这不是你该决定的）

每一种借口都是一种可预见的失败模式——不是随机错误，而是 LLM 天性的系统表现。verificationAgent 的整个设计就是针对这些天性的对抗措施。

原则 **13.1**：自我评估是系统性偏差，不是随机错误 — 模型评估自己输出时禁止依赖自我确认。验证者与生成者必须是独立的智能体，验证者的提示词必须明确对抗已知的自我评估偏差模式。

黄金法则——能力隔离比角色隔离更重要

Claude Code 的智能体分离不是通过不同的"角色设定"实现的，而是通过硬性的工具禁用（`disallowedTools`）。planAgent 不能写文件不是因为提示词说"不要写"，而是因为写文件的工具根本不存在。

`disallowedTools` 是 `BaseAgentDefinition` 的一级配置字段——与 `agentType`、`model` 同级。这不是"建议不要使用"，而是"从工具列表中移除"。

智能体	禁用工具	效果
planAgent	Agent、ExitPlanMode、FileEdit、FileWrite、NotebookEdit（5 种）	完全只读，不能修改任何文件
verificationAgent	Agent、ExitPlanMode、FileEdit、FileWrite、NotebookEdit（5 种）	完全只读，不能修改任何文件，不能嵌套 Agent
exploreAgent	Agent、ExitPlanMode、FileEdit、FileWrite、NotebookEdit（5 种）	完全只读，快速搜索
generalPurposeAgent	无禁用	拥有全部工具，可读写

Claude Code 选择双保险：提示词约束（软约束）+ 工具移除（硬约束）。planAgent 的提示词仍然写着 "CRITICAL: READ-ONLY MODE - NO FILE MODIFICATIONS"（译：关键：只读模式——禁止文件修改），但同时通过 `disallowedTools` 移除了所有写文件工具。提示词是给模型的"告知"，工具移除是系统的"保障"——即使模型忽略了提示词，它也无法执行写操作。

原则 **13.2**：能力隔离优于角色隔离 — 智能体的权限边界必须通过工具集控制（硬约束），而非仅通过提示词角色设定（软约束）。提示词可以违反，工具不存在禁止违反。

适用场景——何时使用哪种内置智能体

6 种内置智能体按"权限范围 × 专精程度"形成从通用到专用的梯度。`getBuiltInAgents` 的注册顺序从 GENERAL_PURPOSE_AGENT 开始，逐步添加专精智能体。

智能体	权限	模型	专精	典型用途
GeneralPurpose	全部	inherit（继承主模型）	通用	通用子任务
Explore	只读	haiku（外部）/ inherit（内部）	搜索	快速查找文件、搜索关键词
Plan	只读	inherit	架构设计	设计方案、分析架构
Verification	只读	inherit	对抗性验证	验证实现是否真正完成

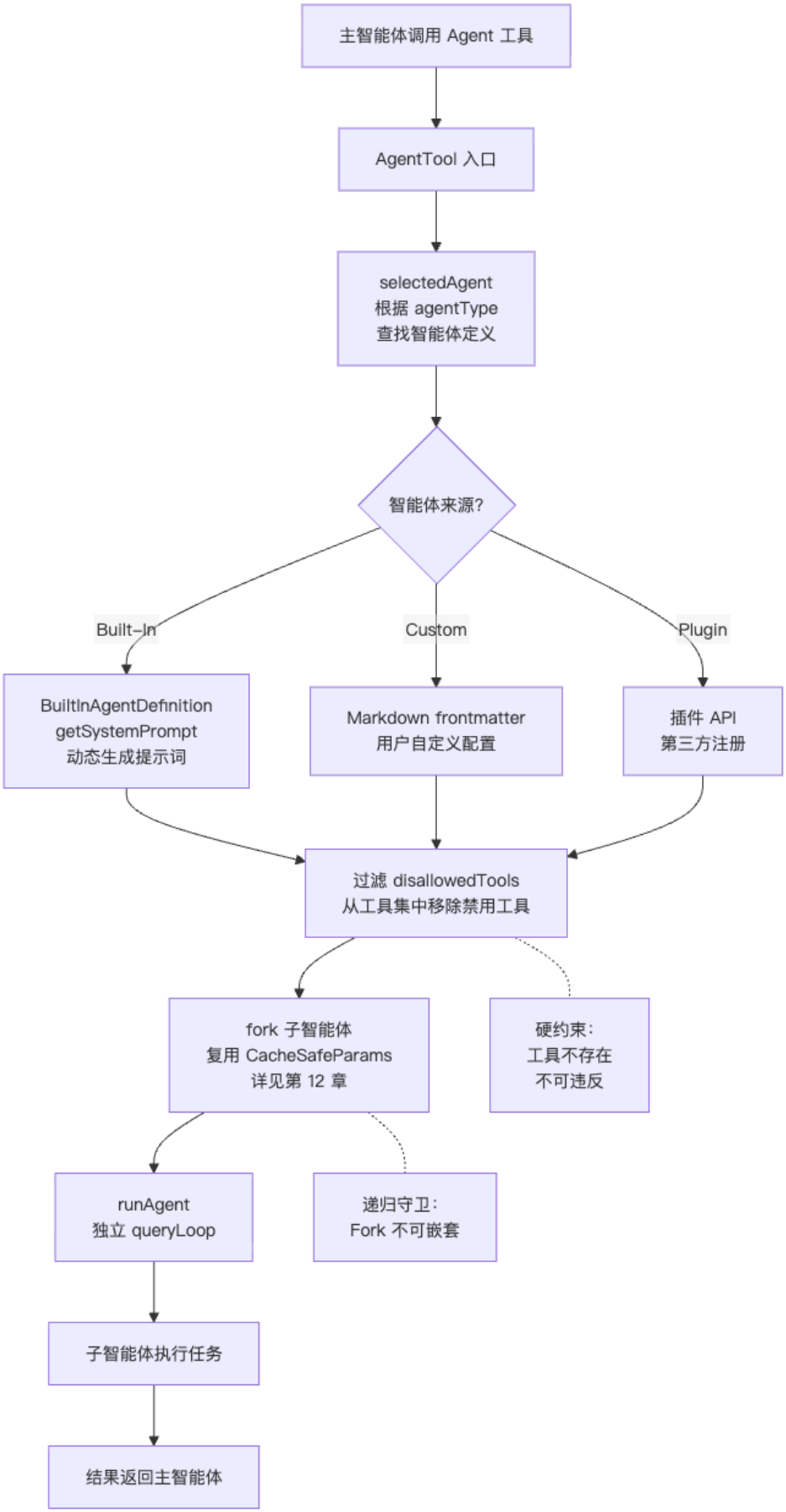
智能体	权限	模型	专精	典型用途
ClaudeCodeGuide	只读	inherit	文档查询	查询 Claude Code 文档
StatuslineSetup	只读	inherit	UI 配置	配置状态栏

Explore 和 Plan 通过 `areExplorePlanAgentsEnabled()` 控制启用状态。Verification 需要双重条件——`feature('VERIFICATION_AGENT')` 和 GrowthBook flag 同时开启。这种渐进式启用说明成本越高的能力越需要谨慎验证。

工作原理——AgentTool 从定义到执行的完整路径

AgentTool 的执行路径是"定义加载 → 工具过滤 → fork 子智能体 → 独立循环 → 结果返回"。每一步都有对应的隔离机制。AgentTool 自身也是一个标准工具——通过 `buildTool` 注册，与其他工具走同一条调度路径。

图 **13-1**：**AgentTool** 执行路径


主智能体调用 Agent 工具
AgentTool 入口
selectedAgent
根据 agentType
查找智能体定义
智能体来源?
Built-In
Custom
Plugin
BuiltInAgentDefinition
getSystemPrompt
动态生成提示词
Markdown frontmatter
用户自定义配置
插件 API
第三方注册
过滤 disallowedTools
从工具集中移除禁用工具
fork 子智能体
复用 CacheSafeParams
详见第 12 章
硬约束：
工具不存在
不可违反
runAgent
独立 queryLoop
递归守卫：
Fork 不可嵌套
子智能体执行任务
结果返回主智能体

第一步：定义加载

智能体有三种来源（`AgentDefinition` 联合类型）：BuiltIn（内置，代码定义）、Custom（自定义，Markdown frontmatter）、Plugin（插件，API 注册）。三者共享 `BaseAgentDefinition` 接口——`disallowedTools`、`model`、`omitClaudeMd` 等字段对所有来源统一。

BuiltIn 智能体通过 `getSystemPrompt()` 函数动态生成系统提示词——这意味着提示词可以根据运行时状态（如当前工作目录、Feature Flag）动态调整，而非写死在配置文件中。

第二步：工具过滤

`disallowedTools` 列表中的工具从可用工具集中移除。planAgent 和 verificationAgent 共享 5 种禁用工具——Agent、ExitPlanMode、FileEdit、FileWrite、NotebookEdit。这个列表确保了：不能生成子 Agent（防递归）、不能退出 Plan Mode（保持只读状态）、不能修改文件（保持只读）。

第三步：**fork** 子智能体

每个 Agent 在独立的 queryLoop 中执行（详见第 12 章 fork 机制）。复用 `CacheSafeParams` 保持 Prompt Cache 命中。递归守卫阻止嵌套——"Fork is not available inside a forked worker"（译：Fork 在 fork 的 worker 中不可用）。

第四步：独立循环 → 结果返回

`runAgent` 启动独立的 queryLoop。子智能体只看到自己的任务提示和工具调用结果，不受主智能体对话历史影响。结果返回给主智能体，作为当前工具调用的输出。

权衡——对抗式验证的 3 个设计代价

决策维度	选择 **A**（本系统）	选择 **B**	核心权衡
子智能体 token	omitClaudeMd 省 5-15 Gtok/周	完整加载 CLAUDE.md	token 成本 vs 信息完整性
验证执行方式	异步（background: true）	同步阻塞	用户体验 vs 验证及时性
Verification 启用	双重 Feature Flag	默认开启	成本控制 vs 功能可达性

代价一：**Token** 成本与 **omitClaudeMd** 权衡

Explore 智能体每周被调用超过 3400 万次。每次调用都加载完整的 CLAUDE.md 层级（包含 commit 规范、PR 模板、lint 配置）对只读智能体毫无意义——它只需要搜索代码，不需要知道 commit 规范。`omitClaudeMd` 注释说明了节省规模："Read-only agents (Explore, Plan) don't need commit/PR/lint guidelines — the main agent has full CLAUDE.md and interprets their results. Saves ~5-15 Gtok/week across 34M+

Explore spawns."（译：只读智能体不需要 commit/PR/lint 指南——主智能体有完整的 CLAUDE.md 并解释它们的输出。在 3400 万次以上的 Explore 调用中每周节省约 5-15 Gtoken）。

代价二：异步验证的延迟

Verification Agent 设置为 `background: true`——异步执行，用户不等待结果。主智能体在 Verification 完成前就返回"完成"状态。这意味着问题可能在用户看到"完成"后才被发现。代价是及时性换用户体验——如果 Verification 同步执行，每次实现都要等待完整的验证周期（可能数分钟）。

代价三：模型依赖

Verification 的有效性取决于模型是否真的遵循了对抗性提示词。而 "RECOGNIZE YOUR OWN RATIONALIZATIONS" 本身就在和模型的天性斗争——提示词告诉模型"你会想偷懒，不要偷懒"，但模型可能仍然偷懒。需要双重 Feature Flag 启用也暗示 Verification 的可靠性和成本尚未达到默认开启的水平（推断）。

踩坑指南——内置智能体中的常见错误

陷阱一：在 **planAgent** 中期望它创建临时文件

`disallowedTools` 包含 FileWrite——即使目标是临时文件也不能写入。planAgent 的"只读"是绝对的，不是"除了临时文件外只读"。

❌ 错误做法：在 planAgent 的提示中要求"先创建一个临时文件记录分析结果"，期望它能绕过只读限制。
✓ 正确做法：planAgent 的输出通过对话历史返回给主智能体——不需要文件作为中间存储。如果确实需要持久化，由主智能体在 planAgent 完成后写入。

陷阱二：在 **verificationAgent** 中期望它修复问题

verificationAgent 只能报告 `PASS/FAIL/PARTIAL`——它不能修改源代码来修复发现的问题。这个限制是刻意的：验证者和修复者是不同的角色，混淆两者会让验证者对"自己的修复"产生评估偏差。

❌ 错误做法：在 verificationAgent 的提示中加入"如果发现问题，请修复它"。
✓ 正确做法：verificationAgent 只负责报告问题。修复由主智能体根据 Verification 结果决定——生成和验证保持分离。

陷阱三：在子智能体中再次调用 **Agent** 工具

递归守卫阻止 Agent 嵌套调用——"Fork is not available inside a forked worker"。如果 generalPurposeAgent 尝试调用 Agent 工具创建子 Agent，会被递归守卫拦截。

❌ 错误做法：在子智能体的任务描述中要求"调用另一个 Agent 处理子任务"。
✓ 正确做法：子智能体执行原子任务。如果需要分解，由主智能体负责编排多个子智能体的调用顺序。

实证——Verification Agent 的对抗性验证流程

一次完整的 Verification 从接收到输出 VERDICT，需要经过构建、测试、对抗性探测、自我审查四个阶段。

接收：主智能体调用 Agent 工具，指定 `agentType: 'verification'`。传入任务描述和变更文件列表。`VERIFICATION_AGENT` 定义了它的属性：`background: true`（异步执行）、`color: 'red'`（红色标识，提示高风险）。

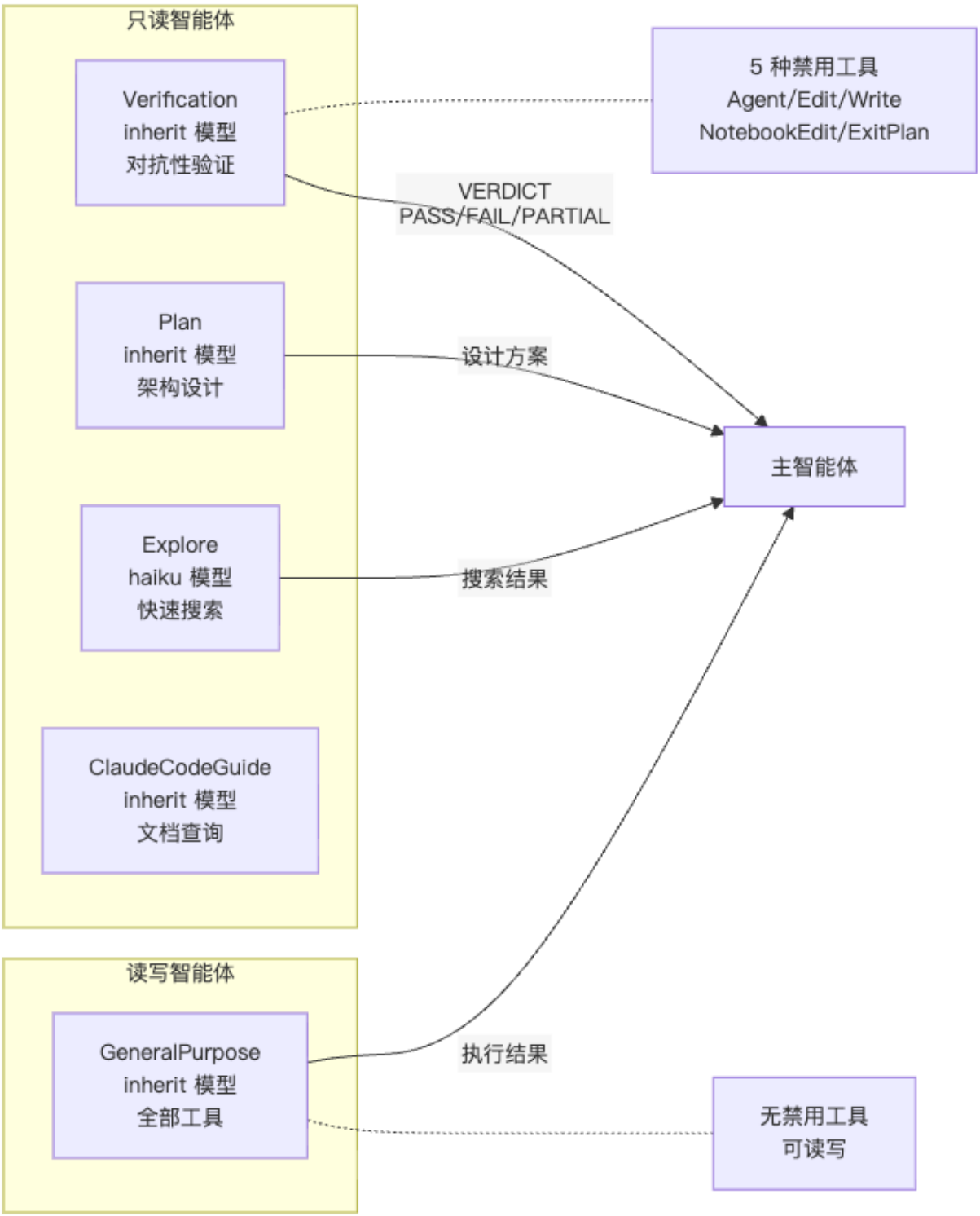
策略选择：验证策略按变更类型区分——Frontend、Backend、CLI、Infrastructure 等 11 种策略。每种策略定义了不同的验证重点：Frontend 关注渲染和交互，Backend 关注 API 契约和错误处理，Infrastructure 关注幂等性和资源清理。

执行：verificationAgent 在独立的 queryLoop 中运行，使用过滤后的工具集（无写操作工具）。它构建项目、运行测试、执行对抗性探测（并发请求、边界输入、幂等性检查、孤儿操作检测）。

自我审查：这是最关键的阶段。提示词强制模型进行自我审查——"RECOGNIZE YOUR OWN RATIONALIZATIONS"（`src/tools/AgentTool/built-in/verificationAgent.ts:53`）。模型必须检查自己是否在上一步中产生了偷懒行为——"If you catch yourself writing an explanation instead of a command, stop. Run the command."（译：如果你发现自己正在写解释而不是执行命令，停下来，运行命令）。

输出 **VERDICT**：最终输出三值判定——`PASS`（验证通过）、`FAIL`（验证失败，附带具体原因）、`PARTIAL`（部分通过，说明环境限制导致某些验证无法执行）。三值设计比二值（PASS/FAIL）更诚实——允许 Verification Agent 报告"我没有完全验证"而非被迫选择 PASS 或 FAIL。

图 **13-2**：**6** 种内置智能体权限矩阵



这条路径验证了 GAN 式生成/验证分离的核心价值：verificationAgent 不是在"帮助"主智能体确认结果，而是在"对抗"主智能体的输出。它的提示词设计、工具限制、三值输出——每一个细节都在对抗模型自我评估的系统性偏差。

本章主成分：生成器与评估器的本质

本质：通过 `disallowedTools` 实现能力隔离（硬约束），通过对抗性提示词（`RECOGNIZE YOUR OWN RATIONALIZATIONS`）对抗模型自我评估偏差。两者共同构成 GAN 式的生成/验证分离——验证者不是确认者，而是破坏者。

关键机制：

- `disallowedTools`：5 种工具硬性禁用，覆盖所有只读智能体
- VERDICT 三值输出：`PASS`/`FAIL`/`PARTIAL`——允许诚实报告验证不完整
- `omitClaudeMd`：只读智能体省略 CLAUDE.md，每周节省 5-15 Gtoken
- `AgentDefinition` 联合类型：BuiltIn | Custom | Plugin 三种来源统一接口

适用边界：

- ✓ 适合：需要独立验证的实现任务（特别是高风险修改）
- ✓ 适合：需要能力隔离的多角色 Agent 系统
- ✗ 不适合：单次简单修改（Verification 的成本不值得）
- ✗ 不适合：纯搜索/查询任务（不需要生成/验证分离）

与其他模式的关系：

- 本章的每个 Agent 都是第 12 章（forkedAgent）的具体使用场景
- 第 9 章（权限系统）的 `hasPermissionsToUseTool` 在子智能体中同样生效
- 第 14 章（并行世界）将展示多 Agent 并行执行

你能做什么

- 为只读智能体设置 **`disallowedTools`**。不要只用提示词约束——移除工具才是真正的能力隔离。提示词可以违反，工具不存在不可违反。
- 设计验证智能体时，明确列出模型的"合理化借口"并逐一对抗。6 种借口不是猜测——它们是 LLM 验证行为的系统性模式。
- 对高频调用的只读智能体使用 **`omitClaudeMd`**。Explore 每周 3400 万次调用证明了 token 节省的价值——5-15 Gtoken/周。
- 验证结果使用三值输出。`PASS`/`FAIL`/`PARTIAL` 允许环境限制的诚实报告——比二值更安全（详见第 7 章工具设计）。
- 为高风险智能体设置双重 **Feature Flag** 启用条件。Verification 的成本高，不应该是默认开启的——渐进式启用让成本可控。
- 使用 **`BaseAgentDefinition`** 统一接口。内置、自定义、插件共享相同的类型系统——新来源只需要实现统一接口。
- 为每个子智能体设置递归守卫。防止 Agent 嵌套调用 Agent 导致无限递归——"Fork is not available inside a forked worker"。

下一章导读：本章看到了单层的生成/验证分离——一个主智能体调用一个子智能体。但当任务可以拆分为多个独立子任务时，串行执行的延迟会成倍增长。第 14 章将展示并行世界——多个 worktree 如何让 Agent 同时在不同分支上独立工作，以及这种并行带来的"世界分裂"和"世界合并"挑战。

第 14 章:并行世界

> "单线程是 REPL 的命运,但不是 Agent 的命运。后台任务是暗线,Coordinator 是明线,两条线织出并行的世界。"

REPL 是单线程的--一轮对话、一个响应、一个任务。但用户的现实世界不是单线程的。Claude Code 用两种方式突破这个限制:后台任务让主循环不阻塞,Coordinator 模式让多个智能体真正并行工作。理解这两种并行的区别--以及它们按"进程内→进程外→远程"三个层次的隔离递进--比知道怎么 spawn 一个后台任务更重要。读完本章,你将理解 Harness 如何在单线程 REPL 之外开辟出并行的执行维度。

问题--REPL 是单线程的,但现实世界不是

REPL 的"一问一答"模型无法处理需要同时进行的任务。如果用户需要同时调研两个问题、同时运行测试和构建、同时修复多个文件--REPL 的单线程循环无法满足。

`TaskState` 联合类型揭示了任务系统的复杂度--7 种任务状态(LocalShell、LocalAgent、RemoteAgent、InProcessTeammate、LocalWorkflow、MonitorMcp、Dream),远比简单的"前台/后台"二元分类复杂。`isBackgroundTask` 函数的判断逻辑是:任务存在 `isBackgrounded` 属性且值为 `false` 时不计为后台任务--这意味着"运行中"和"后台"是独立的维度,一个任务可以正在运行但不在后台。

两种并行解法:

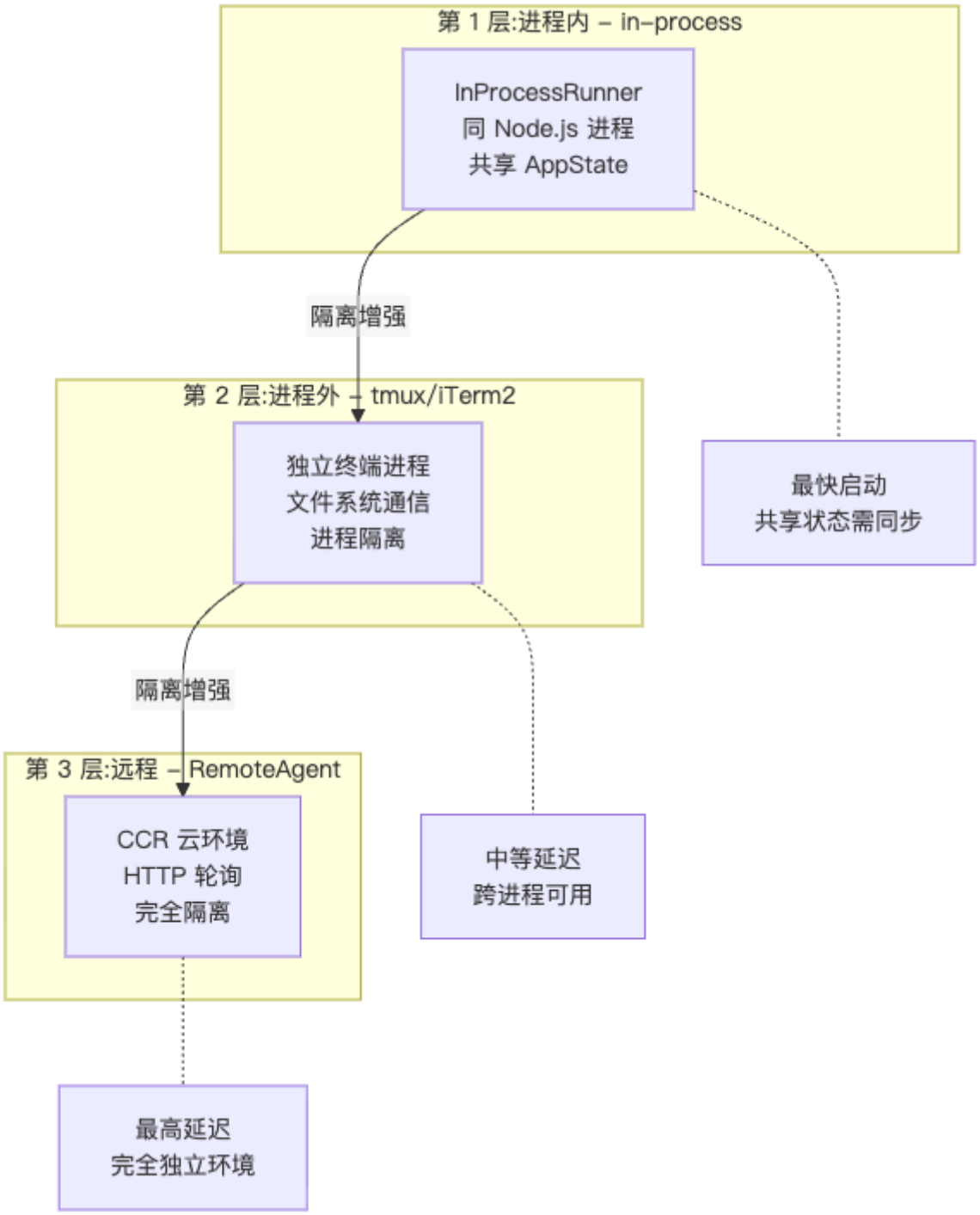
维度	后台任务	**Coordinator** 模式
目标	主对话不阻塞	多 Agent 协调工作
执行者	单个 shell 命令或 Agent	多个 worker Agent
通信	结果返回主对话	协调器综合 worker 结果
适用	独立子任务	多任务协调流水线

原则 **14.1:**并行不是单一能力,是分层能力 - Agent 系统必须按隔离程度提供不同层次的并行能力,而非一个"并行开关"。进程内并行、进程外并行、远程并行的隔离级别和代价完全不同--禁止用同一种机制覆盖所有场景。

黄金法则--并行是能力分层,不是功能开关

Claude Code 的并行能力按"进程内→进程外→远程"三个层次递进,每个层次的隔离和代价不同。`BackendType` 定义了三种后端--源码注释说明:"Types of backends available for teammate execution."(译:可用于队友执行的后端类型)。

图 14-1:三层并行架构



第 1 层:进程内(in-process)

队友在同一 Node.js 进程中运行，共享 AppState（`InProcessRunnerConfig`）。启动最快（无进程创建开销），但隔离最弱——共享状态需要同步机制。`LocalAgentTask` 和 `InProcessTeammate` 属于这一层。

第 2 层:进程外(tmux/iTerm2)

队友在独立终端进程中运行，通过文件系统通信。`SWARM_SESSION_NAME = 'claude-swarm'` 定义了 tmux 会话名。进程隔离意味着一个队友崩溃不影响其他进程，但通信需要通过文件系统的 `teammateMailbox`。

第 3 层:远程(RemoteAgent)

队友在 CCR（Cloud Compute Runtime）云环境中执行（`RemoteAgentTaskState`）。完全独立的环境，HTTP 轮询获取结果。隔离最强（不同机器），延迟最高（网络通信）。

并行层次	隔离级别	启动速度	通信方式	适用场景
进程内	共享内存	最快	函数调用	快速探索、轻量验证
进程外	进程隔离	中等	文件邮箱	长时间运行任务
远程	完全隔离	最慢	HTTP 轮询	云端大规模任务

原则 **14.2:**隔离越强,代价越高 - 并行层次的选择必须匹配任务的安全需求。简单的只读搜索不需要远程执行,高风险的代码修改禁止在共享状态的进程内运行。

适用场景--何时用后台任务,何时用 Coordinator

后台任务和 Coordinator 解决不同的问题--"主对话不被阻塞"和"多任务需要协调"。

后台任务适合:

- 独立 shell 命令(`LocalShellTask`)--构建、测试、lint,不需要主对话等待
- 独立 Agent 任务(`LocalAgentTask`)--探索代码库、验证实现,结果异步返回
- 远程 Agent 任务(`RemoteAgentTask`)--在云端执行,主对话继续工作

Coordinator 模式适合:

- 多文件并行修改--每个 worker 负责一组文件,协调器综合结果
- 研究→实现→验证流水线--协调器编排阶段转换,确保前序阶段完成后再启动后续阶段
- 需要多视角分析的任务--多个 worker 从不同角度并行研究,协调器综合不同视角

`isCoordinatorMode()` 需要显式启用(Feature Flag + 环境变量)--Coordinator 模式不是默认开启的,它有显著的资源成本和复杂度。

工作原理--Coordinator 模式的任务分配和通信机制

Coordinator 模式通过“协调器→worker→通知→综合”的循环实现多智能体协作。`TEAM_LEAD_NAME = 'team-lead'` 是协调器的固定名称。协调器的系统提示词定义了完整的工作流。

协调器角色

提示词明确定义了协调器的职责:"You are a coordinator. Your job is to: Direct workers to research, implement and verify code changes; Synthesize results and communicate with the user; Answer questions directly when possible - don't delegate work that you can handle without tools."(译:你是一个协调器。你的职责是:指导 worker 研究、实现和验证代码变更;综合结果并与用户沟通;尽可能直接回答问题--不要委托你自己能处理的工作)。

并发策略

提示词给出了明确的并发规则:"Parallelism is your superpower. Workers are async. Launch independent workers concurrently whenever possible - don't serialize work that can run simultaneously."(译:并行是你的超能力。Worker 是异步的。尽可能并发启动独立的 worker--不要把可以同时运行的工作串行化)。

同时有约束条件:"Read-only tasks - run in parallel freely; Write-heavy tasks - one at a time per set of files."(译:只读任务自由并行;写操作任务按文件集合一次一个)。这条规则编码了一个关键设计决策--写操作必须按文件集合串行化,否则多个 worker 同时修改同一文件会产生冲突。

通信机制

teammateMailbox:源码注释定义了它的定位:"Teammate Mailbox - File-based messaging system for agent swarms"(译:队友邮箱--基于文件系统的 Agent 群组消息系统)。`TeammateMessage` 类型定义了消息结构,`readMailbox` 函数读取队友的消息。文件系统的选择让通信跨进程可用--无论后端是 in-process、tmux 还是 iTerm2。

SendMessage 工具:队友间的实时通信通过 SendMessage 工具实现。`TEAMMATE_SYSTEM_PROMPT_ADDENDUM` 告诉队友:"IMPORTANT: You are running as an agent in a team. To communicate with anyone on your team: Use the SendMessage tool with `to: '<name>'` to send messages to specific teammates."(译:重要:你正在作为团队中的一个 Agent 运行。要与团队中的任何人通信:使用 SendMessage 工具发送消息给指定队友)。

task-notification XML:Worker 完成任务后,通过 task-notification XML 格式汇报结果--包含 task-id、status、summary、result 和 usage。协调器解析这些通知,综合 worker 的输出。

通信方式	适用范围	延迟	方向
teammateMailbox	跨进程队友	I/O 延迟	双向
SendMessage 工具	队友间实时	函数调用级	双向

通信方式	适用范围	延迟	方向
task-notification XML	Worker→协调器	随任务完成	单向

自包含提示词原则

协调器提示词中最关键的设计约束:"Workers can't see your conversation. Every prompt must be self-contained with everything the worker needs."(译:Worker 看不到你的对话。每个提示词必须自包含 worker 需要的所有信息)。这意味着协调器不能写"基于你刚才的研究结果"这样的模糊指令--worker 不知道"刚才的研究"是什么。

权衡--并行系统的 3 个设计代价

决策维度	选择 A(本系统)	选择 B	核心权衡
共享状态	进程内需同步	完全隔离	速度 vs 安全
通信方式	文件邮箱	进程内调用	跨进程可用性 vs 延迟
提示词设计	自包含(重复上下文)	共享对话历史	隔离性 vs token 成本

代价一:共享状态冲突

in-process 队友共享 AppState--文件系统状态、权限配置、工具上下文都在同一进程中。写操作必须按文件集合串行化,否则两个 worker 同时修改同一文件会产生冲突。这个约束不是"建议",是系统提示词中的硬规则--违反它会导致不可预测的文件损坏。

代价二:通信延迟

文件邮箱(teammateMailbox)用文件系统作为消息队列。好处是跨进程可用(tmux/iTerm2 后端),代价是每次消息传递都有 I/O 延迟。对于高频消息(如进度更新),这个延迟可能成为瓶颈。进程内后端不需要文件邮箱(直接函数调用),但通信方式的不统一增加了维护复杂度(推断)。

代价三:上下文重复

"Workers can't see your conversation"意味着每个 worker 都需要完整的自包含提示词--包括任务背景、代码上下文、约束条件。协调器必须把所有信息写进每个 worker 的提示词。对于复杂任务,一个 worker 的提示词可能消耗数千 token。多个 worker 并行时,token 成本成倍增长。

踩坑指南--并行系统中的常见错误

陷阱一:给 worker 的提示词不够自包含

"Workers can't see your conversation"是最常被忽视的约束。协调器写了"基于你的发现修复问题"这样的指令--但 worker 看不到协调器的对话历史,不知道"你的发现"是什么。

❌ 错误做法:在 worker 提示中引用之前的对话上下文("根据你刚才的研究"或"继续上面的任务")。 ✓ 正确做法:每个 worker 的提示词必须完全自包含--包括任务目标、相关文件路径、具体约束。假设 worker 是一个刚加入团队的新成员,对之前的对话一无所知。

陷阱二:多个 **worker** 同时修改同一组文件

协调器提示词的并发规则明确:"Write-heavy tasks - one at a time per set of files."违反这个规则会导致文件冲突--两个 worker 的修改互相覆盖。

❌ 错误做法:让两个 worker 同时修改同一个模块的不同部分,期望 git 能自动合并。 ✓ 正确做法:按文件集合分配任务--每个 worker 负责一组互不重叠的文件。如果两个 worker 必须修改同一文件,串行执行。

陷阱三:在 **in-process** 后端中忽视共享状态

in-process 队友共享 AppState。如果一个 worker 修改了全局状态(如切换工作目录、修改权限配置),会影响同一进程中的所有 worker。

❌ 错误做法:在 in-process worker 中修改全局状态(如切换 `cwd`),期望其他 worker 不受影响。 ✓ 正确做法:需要进程隔离的任务使用 tmux/iTerm2 后端。in-process 只用于只读或无状态的任务。

实证--Coordinator 的一次完整任务分配循环

一次完整的 Coordinator 任务循环包含"研究→综合→实现→验证"四个阶段,协调器在每个阶段做出不同的并行策略选择。

阶段一:**Research(**并行只读**)**

协调器接收到用户任务"为登录模块添加 OAuth 支持"。它分析任务,决定启动 3 个并行只读 worker:Worker A 调研现有登录代码,Worker B 调研 OAuth 库的最佳实践,Worker C 调研现有测试覆盖。只读任务自由并行--提示词明确允许:"Read-only tasks - run in parallel freely"。

阶段二:**Synthesis(**协调器综合**)**

3 个 worker 通过 task-notification XML 汇报结果。协调器读取 `readMailbox`(`src/utils/teammateMailbox.ts:84`)收集所有结果。提示词的核心指令:"Always synthesize - your most important job is to synthesize findings into a specific prompt."(译:始终综合--你最重要的工作是将发现综合为具体的提示词)。协调器不是简单转发 worker 结果,而是理解后重新组织为自包含的实现规格。

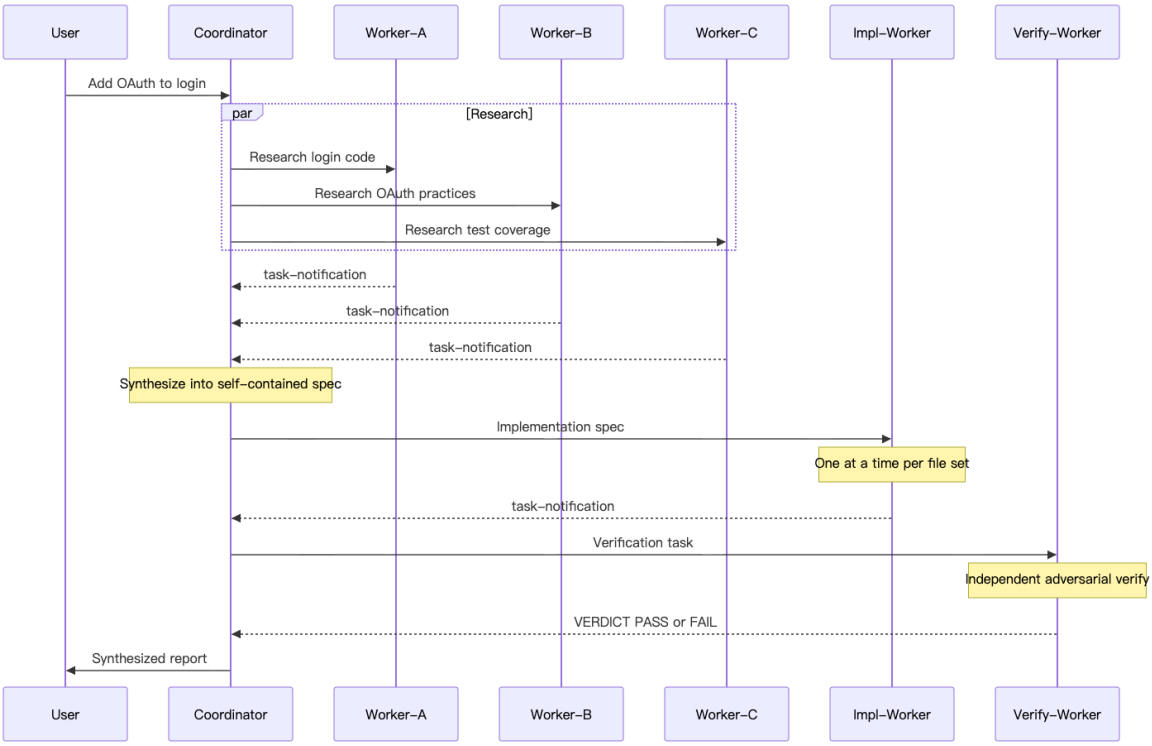
阶段三**:Implementation(**按文件集合串行**)**

协调器根据综合结果,编写自包含的实现提示词,启动实现 worker。如果涉及多组文件,按文件集合串行分配--同一组文件只有一个 worker 在修改。每个 worker 的提示词包含完整的实现规格,不引用对话历史。

阶段四**:Verification(**独立验证**)**

协调器启动独立的验证 worker--最好使用全新的 worker 以避免实现偏差(详见第 13 章的对抗性验证理念)。验证结果以 `PASS`/`FAIL`/`PARTIAL` 返回,协调器决定是否需要重新实现。

图 **14-2:Coordinator** 任务分配循环



这条路径验证了 Coordinator 模式的核心价值:协调器不做具体工作("don't delegate work that you can handle without tools"是例外),它的核心职责是"综合"--理解多个 worker 的输出,重新组织为下一个阶段的自包含输入。综合质量直接决定并行系统的整体效果。

本章主成分:并行世界的本质

本质:并行不是单一能力,是按隔离程度分层的(进程内→进程外→远程),每一层有不同的通信机制和代价。后台任务解决"主对话不被阻塞",Coordinator 模式解决"多任务需要协调"--两种并行的目标、机制和适用场景完全不同。

关键机制:

- `BackendType`:3 种后端(in-process/tmux/iTerm2),递进的隔离策略
- `TaskState`:7 种任务状态,覆盖从本地 shell 到远程 Agent 的全部场景
- Coordinator 系统提示词:角色定义 + 并发规则 + 自包含约束
- `teammateMailbox`:基于文件系统的消息队列,跨进程可用
- "Workers can't see your conversation":自包含提示词原则

适用边界:

- ✓ 适合:需要并行执行多个独立子任务的场景
- ✓ 适合:需要协调多阶段流水线(研究→实现→验证)的复杂任务
- ✗ 不适合:单任务单步骤的简单操作(并行开销不值得)
- ✗ 不适合:所有步骤都有强依赖关系的线性任务(无法并行)

与其他模式的关系:

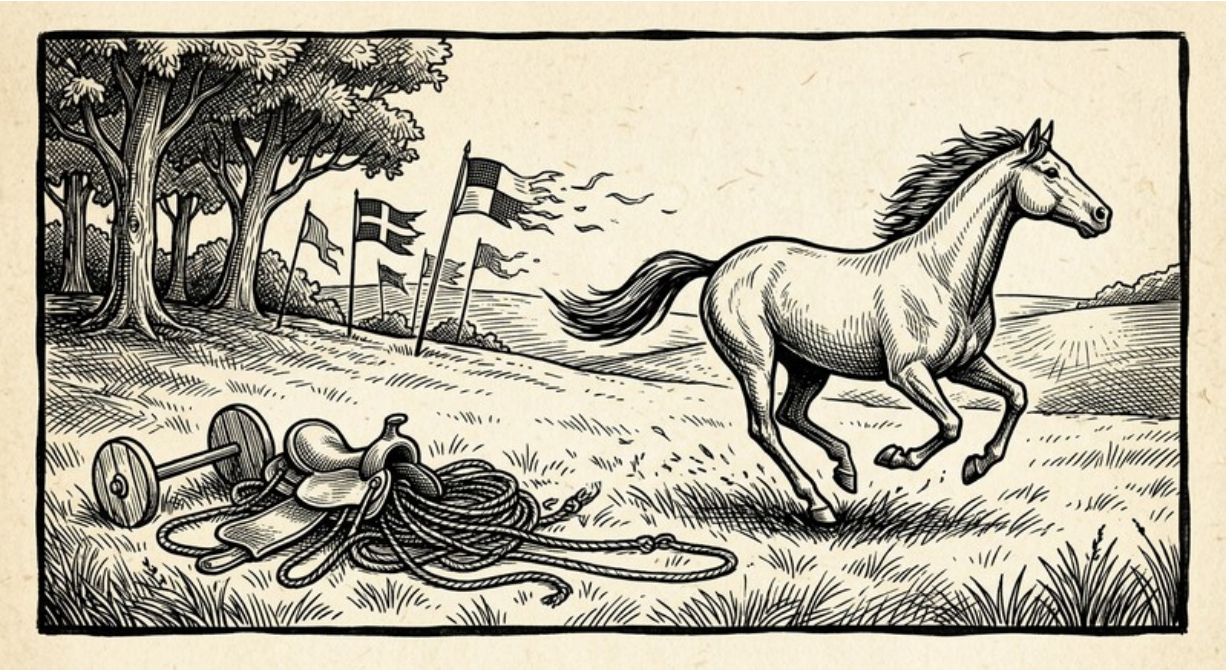
- 第 13 章(生成器与评估器)的内置智能体在 Coordinator 模式中就是 worker
- 第 12 章(fork 机制)是每个 worker 的底层执行机制
- 第 10 章(Hooks)的 `SubagentStart/Stop` 触发点在 Coordinator 中触发

你能做什么

- 区分"后台任务"和"**Coordinator** 模式"。前者是主对话不被阻塞,后者是多个 Agent 协调工作。混淆两者会导致设计方向错误。
- 为每个 **worker** 提供自包含的提示词。Worker 看不到你的对话,所有上下文必须写进提示词--"Workers can't see your conversation"。
- 只读任务自由并行,写操作按文件集合串行。这是 Coordinator 系统提示词中的显式规则--违反它会导致文件冲突。
- 协调器的核心职责是"综合"。不是简单转发 worker 结果,而是理解后重新组织为下一个阶段的自包含输入。
- 选择后端时按隔离需求决定。进程内(最快)→ 进程外(隔离好)→ 远程(完全隔离)--不要对所有任务使用同一种后端。
- 为队友间通信实现基于文件的消息队列。参考 `teammateMailbox`--文件系统通信让跨进程可用,I/O 延迟在大多数场景中可接受。
- 在协调器提示词中明确"什么是好的提示词"和"什么是坏的提示词"。减少 worker 的无效调用--每个 worker 的提示词质量直接决定并行系统的效率。

下一章导读:本章看到了 Coordinator 模式如何让多个 Agent 并行工作。但 Agent 能力的边界在哪里?第 15 章将展示 Claude Code 的扩展机制--MCP(Model Context Protocol)如何让 Harness 连接外部世界,以及插件的动态加载系统如何在不修改核心代码的情况下添加新能力。

第八篇：演进



最好的脚手架是自己消失的脚手架。最后两章从两个方向收束全书——向后看，**Feature Flag** 如何让 **Harness** 自我精简；向前看，从 **Claude Code** 的具体决策中提炼出通用工程原则。

第 15 章：脚手架会消失

> "最好的 Harness 是你正在拆的那个。每个 feature flag 都是一张拆除许可证：盖章日期待定。"

89 个 feature flag，828 处调用——每一处都是一个赌注："有一天这个 flag 会关掉，这段代码会消失。"理解 feature flag 不是开关，而是"脚手架拆除计划"，比知道有哪些 flag 更重要。读完本章，你将理解 `feature()` 如何在编译时物理消除代码，以及为什么 Harness 的大部分功能是在等待模型变得足够好以至于自己变得多余。

问题——Harness 会随着模型进步变得多余吗

如果模型有一天能完美管理上下文、评估输出、协调任务，那么压缩策略、Verification Agent、Coordinator 模式都不再需要。89 个 feature flag 的存在本身就说明 Harness 的大部分功能是在弥补模型能力的不足——当模型变得更好，这些功能应该消失。

`DEFAULT_FEATURES` 只有 3 个条目：`KAIROS`、`COORDINATOR_MODE`、`CLAUDE_CODE_SIMPLE`——全部默认 `false`。这意味着大部分 feature 在默认配置中是关闭的，只有在特定构建中才启用。`feature()` 函数的实现更直接：`currentFeatures[featureName] ?? false`——未配置的 flag 返回 `false`，代码不执行。

更关键的是 `import { feature } from 'bun:bundle'`——这不是普通的配置读取，而是 Bun 编译器的 DCE（Dead Code Elimination，死代码消除）钩子。当 `feature('X')` 返回 `false` 时，Bun 编译器在构建产物中物理消除 `false` 分支的代码——不是注释掉，不是跳过，是从产物中彻底消失。

统计维度	数字	含义
Feature flag 总数	89 个	89 个"模型能力假设"
`feature()` 调用总数	828 处	828 处条件代码路径
`DEFAULT_FEATURES`	3 个	默认全关闭
main.tsx 中的调用	60 处	入口文件最密集

原则 **15.1**：每个补偿模型不足的功能都必须可消除 — Harness 中每个弥补模型能力缺陷的功能必须受 feature flag 控制。禁止将补偿性代码硬编码为无条件行为——当模型进步时，这些代码必须能被安全移除。

黄金法则——每个 feature flag 编码一个"模型能力假设"

feature flag 不是功能开关，而是一条工程断言——"在当前模型能力下，这段 Harness 代码是必需的；当模型超越这个阈值，这段代码可以安全移除"。源码中的注释反复强调 DCE 目的，而非"开关"语义。

Feature flag	编码的能力假设	关联章节
`REACTIVE_COMPACT`	模型可能产生超长输出导致 API 413 错误	第 11 章
`HISTORY_SNIP`	模型处理完整长历史的能力不足	第 11 章
`CACHED_MICROCOMPACT`	API cache 机制不完美，需要手动管理	第 11 章
`VERIFICATION_AGENT`	模型无法可靠评估自己的输出	第 13 章
`COORDINATOR_MODE`	单 Agent 无法高效处理并行任务	第 14 章
`CONTEXT_COLLAPSE`	简单压缩策略释放空间不够	第 11 章
`KAIROS`（24 处调用）	需要独立的 assistant 模块	main.tsx

每个 flag 的 `true` 状态意味着"模型在这方面还不够好，需要 Harness 补偿"；`false` 状态意味着"这段补偿代码不需要"。autoCompact 源码中的注释直接标注了这种语义："Inside feature() so the string DCEs from external builds (it's in excluded-strings.txt)."（译：放在 feature() 内部以使字符串从外部构建中通过 DCE 消除（它在 excluded-strings.txt 中））。

microCompact 的延迟导入也遵循同样的原则："The imports and state live inside feature() checks for dead code elimination."（译：导入和状态放在 feature() 检查内部以实现死代码消除）。不只是运行时行为，连模块的物理存在都受 flag 控制。

原则 **15.2**：**Flag** 消除的是代码，不是行为 — Feature flag 必须通过编译时 DCE 物理消除代码，而非仅通过运行时 `if` 跳过。代码物理消失意味着产物更小、启动更快、攻击面更小——这些是运行时 `if` 无法提供的优势。

适用场景——哪些 Harness 功能最可能先消失

按"模型进步速度 × Harness 代码量"排列，压缩策略最可能先被移除。

消失速度	功能	理由
最快	`REACTIVE_COMPACT`	模型的输出长度控制正在改善
最快	`HISTORY_SNIP`	上下文窗口在增长，长历史不再是致命问题
中等	`CACHED_MICROCOMPACT`	API cache 机制在改善
中等	`microCompact`（整体）	cache TTL 优化后，手动清除的价值降低

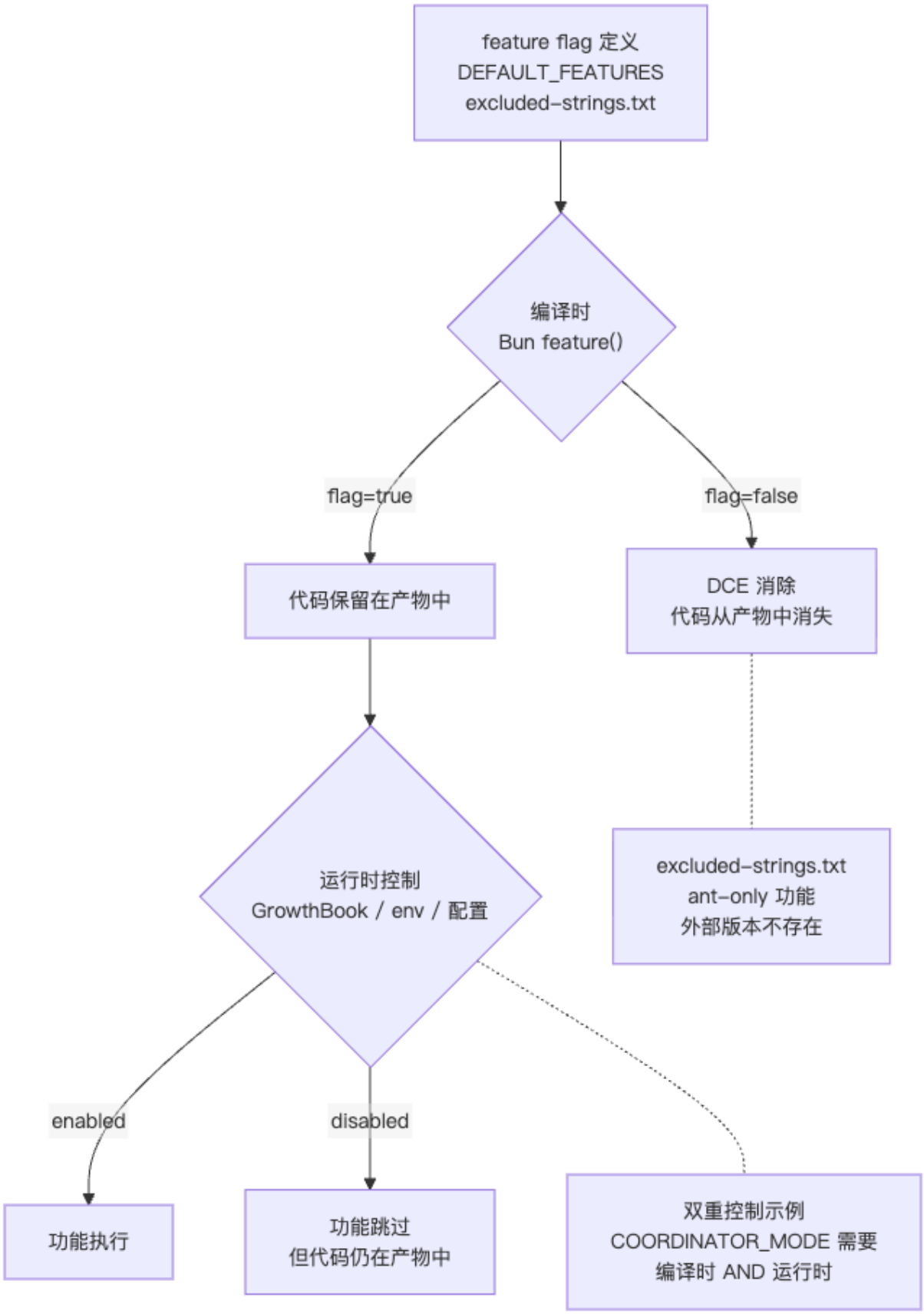
消失速度	功能	理由
最慢	`VERIFICATION_AGENT`	自我评估是模型的根本困难
最慢	`COORDINATOR_MODE`	并行协调是架构问题，不只是能力问题

`REACTIVE_COMPACT` 标记为 "ant-only"——源码注释明确说："REACTIVE_COMPACT is ant-only"。ant-only 功能只在内部构建中存在，外部版本通过 DCE 完全消除。这意味着这个功能可能已经在评估是否可以永久关闭。

工作原理——feature flag 的编译时和运行时两层机制

Claude Code 的 feature flag 有两层——编译时 DCE（Bun feature）和运行时控制（GrowthBook/环境变量/配置），两层配合实现"开发时灵活、生产时精简"。

图 **15-1**：**feature flag** 两层机制


feature flag 定义
DEFAULT_FEATURES
excluded-strings.txt
编译时
Bun feature()
flag=true
flag=false
代码保留在产物中
DCE 消除
代码从产物中消失
excluded-strings.txt
ant-only 功能
外部版本不存在
运行时控制
GrowthBook / env / 配置
enabled
disabled
功能执行
功能跳过
但代码仍在产物中
双重控制示例
COORDINATOR_MODE 需要
编译时 AND 运行时

编译时 DCE

`feature()` 函数在 Bun 编译时被编译器识别。当 `feature('X')` 返回 `false` 时，Bun 编译器将 `if (feature('X')) { ... }` 中的整个代码块从产物中消除。这比运行时 `if` 更彻底：

- 产物更小——828 处 `feature()` 调用中，大部分在外部版本中返回 `false`，对应代码不进入产物
- 启动更快——不需要初始化不存在的模块
- 攻击面更小——代码物理不存在，无法被利用

`excluded-strings.txt` 是 Bun 的 DCE 配置文件——列出所有只存在于内部构建的 flag 名称。autoCompact 源码中的注释解释了它的工作方式："Inside feature() so the string DCEs from external builds (it's in excluded-strings.txt)"。flag 名称字符串本身也从外部版本中消除——连字符串常量都不留。

运行时控制

即使代码通过了编译时 DCE，运行时仍有多层控制。`COORDINATOR_MODE` 是最典型的双重控制：编译时需要 `feature('COORDINATOR_MODE')` 为 `true`，运行时还需要 `isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE)` 为 `true`。两者缺一不可。

运行时控制来源包括：

- **GrowthBook Feature Flags**——远程配置，可动态切换
- 环境变量——如 `DISABLE_AUTO_COMPACT`、`CLAUDE_CODE_COORDINATOR_MODE`
- 用户配置——如 `autoCompactEnabled`（详见第 11 章）

两层配合的价值

两层机制不是冗余，而是互补。编译时 DCE 解决"外部版本不应该有这段代码"的安全和性能问题；运行时控制解决"即使代码存在，也应该可以动态开关"的灵活性问题。Coordinator 模式需要两层都启用——编译时确保外部版本不包含 Coordinator 代码（功能尚未成熟），运行时允许内部用户按需启用。

控制层	时机	机制	效果
编译时 DCE	构建时	`bun:bundle` feature() + excluded-strings.txt	代码物理消除
运行时	执行时	GrowthBook / env / 配置	行为开关

权衡——DCE 驱动架构的 3 个设计代价

决策维度	选择 A（本系统）	选择 B	核心权衡
代码分割	条件 require 切断模块图	统一构建	产物精简 vs 构建复杂
测试矩阵	89 个 flag 的排列组合	固定配置	覆盖面 vs 测试成本
延迟导入	feature-gated 模块按需加载	全量导入	安全性 vs 开发便利

代价一：代码分割

main.tsx 中的条件 `require()` 切断了模块依赖——`coordinatorModeModule` 只在 `feature('COORDINATOR_MODE')` 为 `true` 时存在。不同构建的模块图不同。这意味着开发者需要理解"这段代码在哪个构建中存在"——microCompact 的注释直接说明了这个问题："Lazy-initialized cached MC module and state to avoid importing in external builds."（译：延迟初始化缓存 MC 模块和状态，以避免在外部构建中导入）。

代价二：测试矩阵

89 个 flag 的所有组合不可能穷尽——2^89 是天文数字。团队只能聚焦高风险组合测试（推断）。每个新增 flag 都增加测试负担，但移除 flag 需要评估模型能力是否已经足够好——这个评估本身就有成本。

代价三：延迟导入的开发便利性

顶层 `import` 更直观，但 external builds 中某些模块不存在，直接 import 会导致编译错误。开发者必须使用延迟导入（条件 `require` 或动态 `import()`），增加了代码的间接性。

踩坑指南——feature flag 的常见错误

陷阱一：在 **feature()** 外的顶层代码中引用 **feature-gated** 模块

DCE 消除的不是变量，是整个导入链。如果顶层 `import` 引用了只存在于内部构建的模块，external builds 会编译失败。

❌ 错误做法：在文件顶部直接 `import { someInternalAPI } from './internalModule.ts'`——即使只在 `feature()` 内部使用，顶层 import 也会在 external builds 中失败。
✓ 正确做法：使用条件 `require()` 或动态 `import()` 在 `feature()` 检查内部延迟导入。microCompact 的延迟导入模式是标准参考。

陷阱二：假设 **feature flag** 只控制运行时行为

`feature('X')` 返回 `false` 时，代码不仅不执行——它在产物中不存在。这意味着你不能通过运行时日志或错误追踪来调试不存在的代码。

❌ 错误做法：在 external builds 中调试 feature-gated 功能，期望看到"flag is false"的日志。
✓ 正确做法：理解 DCE 消除的代码在产物中完全不存在。调试时需要确认构建配置中目标 flag 是否为 `true`。

陷阱三：忘记 **lazy import** 导致 **external build** 失败

microCompact 的注释是教训："The imports and state live inside feature() checks for dead code elimination."——如果你忘记了这一点，把 import 放在 `feature()` 外面，external builds 会在编译时失败。

❌ 错误做法：在 `feature()` 检查外面声明依赖 feature-gated 模块的变量或函数。
✓ 正确做法：所有依赖 feature-gated 模块的代码都放在 `feature()` 检查内部。使用 lazy import 模式确保 external builds 不引入不存在的模块。

实证——从 feature flag 到代码消失的完整路径

追踪 `REACTIVE_COMPACT` 从定义到使用到 DCE 的完整路径，展示一个 feature flag 如何控制代码的物理存在。

定义：`REACTIVE_COMPACT` 不在 `DEFAULT_FEATURES` 中（默认不存在），通过 `excluded-strings.txt` 注册为 ant-only 功能。

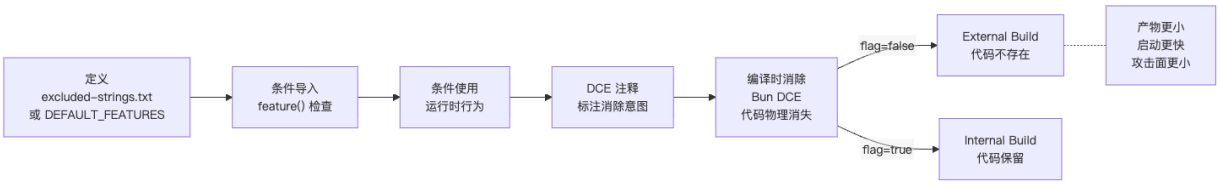
条件导入：`src/query.ts` 中通过 `const reactiveCompact = feature('REACTIVE_COMPACT')` 条件加载。如果 flag 为 `false`，`reactiveCompact` 为 `false`（falsy），相关代码路径不执行。

条件使用：autoCompact 的 `shouldAutoCompact` 中，`REACTIVE_COMPACT` 为 `true` 时关闭主动压缩，让 reactive compact 在 API 返回 413 错误时触发。注释标注了 DCE 目的："feature() wrapper keeps the flag string out of external builds (REACTIVE_COMPACT is ant-only)."（译：feature() 包装保持 flag 字符串不出现在外部构建中（REACTIVE_COMPACT 是仅限内部的））。

DCE 注释：query.ts 中的注释进一步说明："Gated behind feature() so the string is eliminated from external builds."（译：用 feature() 门控以使字符串从外部构建中消除）。

编译时消除：Bun 构建时，`feature('REACTIVE_COMPACT')` 返回 `false`，整个条件分支从产物中消除——不是注释掉，不是空函数，是彻底消失。

图 **15-2**：**Feature flag** 生命周期



这条路径验证了脚手架设计的核心价值：每个 feature flag 是一条从"补偿模型不足"到"代码物理消失"的完整生命周期。flag 关闭不是"功能关闭"——是代码不存在。这种设计让 Harness 随模型进步而自我精简，而非越做越臃肿。

本章主成分：脚手架会消失的本质

本质：89 个 feature flag 是 89 个"模型能力假设"，每个 flag 的 `false` 状态意味着"这段 Harness 代码在外部版本中不存在"——不是关闭，是消失。Harness 不是越做越大的护城河，而是一座设计好拆除方案的脚手架。

关键机制：

- 编译时 DCE：Bun `feature()` + `excluded-strings.txt`，代码物理消除
- 运行时控制：GrowthBook / 环境变量 / 用户配置，行为动态开关
- 双重控制：`COORDINATOR_MODE` 等需要两层同时启用
- 828 处 `feature()` 调用，覆盖从压缩到验证到协调的全部 Harness 功能

适用边界：

- ✓ 适合：需要随模型进步逐步精简的 Harness 系统
- ✓ 适合：有 internal/external 构建差异的多版本系统
- ✗ 不适合：一次性工具（不需要渐进式架构）
- ✗ 不适合：所有功能都是永久性的系统

与其他模式的关系：

- 本章综合了第 3-14 章所有 feature flag 的用法
- 第 11 章（压缩策略）的 5 种策略各有对应的 feature flag
- 第 13 章（Verification Agent）需要 `VERIFICATION_AGENT` flag 双重启用
- 第 14 章（Coordinator 模式）需要 `COORDINATOR_MODE` 编译时 + 运行时控制
- 第 16 章将把这些观察提炼为通用的设计原则

你能做什么

- 为你的 **Harness** 系统建立 **feature flag** 机制。每个补偿模型不足的功能都用 flag 控制——flag 关闭时，补偿代码应该可以安全移除。
- 使用编译时 **DCE** 而不是运行时 **if**。产物大小和启动时间都受益——代码物理消失比运行时跳过更彻底。

- 在 **feature flag** 代码中标注 **DCE** 注释。参考 query.ts 的 "Gated behind feature() so the string is eliminated from external builds" 模式——帮助维护者理解哪些代码可能不存在。
- 定期审查 **feature flag**。如果模型在某方面已经足够好，就关掉对应的 flag——flag 的目标是消失，不是永久存在。
- 对 **ant-only** 功能使用 **excluded-strings.txt**。从外部版本中彻底消除——连 flag 名称字符串都不留。
- 延迟导入 **feature-gated** 模块。参考 microCompact 的 lazy import 模式——避免顶层 import 导致 external builds 失败。
- 为 **flag** 的排列组合建立优先级矩阵。不可能测试所有 2^89 种组合——聚焦高风险的 flag 组合测试。

下一章导读：本章看到了 89 个 feature flag 如何编码"模型能力假设"，以及 DCE 如何让 Harness 随模型进步而自我精简。第 16 章是全书的最后一章——将前 15 章的所有观察提炼为 7 条通用设计原则，一套可迁移到任何 AI Agent 系统的 Harness 架构蓝图。

第 16 章：从 Claude Code 到通用原则

> "15 章之后，问题没有变——什么交给模型，什么交给代码？但答案变了：这不是一个二选一，而是一个有 7 条原则的决策框架。"

15 章源码分析，数百处锚点，最终归于一个问题：什么该交给模型，什么必须由 Harness 控制？本章不教你怎么复制 Claude Code 的实现——它给你一套决策框架，让你在任何 LLM 应用中回答同一个问题。每条原则都有源码中的真实决策支撑，每条都有明确的适用边界。

问题——"模型 vs Harness"是一个伪问题

前 15 章的分析反复证明，"模型"和"Harness"不是竞争关系。Harness 编码的是模型在当前版本中做不到的事，而 feature flag 机制确保这些代码在模型足够好时会消失（详见第 15 章）。

模型能力不足	**Harness** 补偿	章节
上下文窗口有限	5 种压缩策略按深度执行	第 11 章
摘要生成不稳定	`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES=3` 熔断	第 11 章
自我评估偏差	Verification Agent 对抗性验证	第 13 章
单 Agent 无法并行	Coordinator 模式多智能体协作	第 14 章
旧工具结果占位	microCompact 清除白名单工具输出	第 11 章
权限判断不完美	4 层纵深防御 + AI 分类器	第 9 章
缓存失效浪费 token	CacheSafeParams 保持缓存命中	第 12 章

"模型 vs Harness"的正确理解是：Harness 是模型能力的历史快照。89 个 feature flag 中的每一个都记录了一个"模型在某个版本中还做不到"的事实（详见第 15 章）。当模型进步时，flag 关闭，代码消失——Harness 缩小，不是扩大。

黄金法则——7 条 Harness 工程原则

从 15 章的源码分析中，可以提炼 7 条可复用的原则。每条都不是理论设计，而是 Claude Code 源码中真实存在的工程决策。

原则 **16.1**：能力边界硬编码，行为指导软编码 — 用工具禁用（`disallowedTools`）而非提示词约束来隔离能力。提示词可以违反，工具不存在不可违反。

planAgent 的 `disallowedTools` 列出了 5 种被硬性移除的工具（Agent、ExitPlanMode、FileEdit、FileWrite、NotebookEdit）。planAgent 不能写文件不是因为提示词说"不要写"，而是因为写文件的工具根本不存在（详见第 13 章）。

原则 **16.2**：补偿措施分层，轻量先行，重量托底 — 5 种压缩策略按深度排序执行，轻量级策略释放足够 token 时重量级策略不触发。

queryLoop 中的执行顺序是 `snip → microCompact → contextCollapse → autoCompact`——每一步都尝试释放 token，如果足够就停止。只有所有轻量策略都释放不足时，autoCompact 才执行全量摘要（详见第 11 章）。

原则 **16.3**：缓存是约束，不是优化 — `CacheSafeParams` 的 5 个参数必须逐字相同才能命中缓存。改一个字，缓存全部失效。

fork 时隔离对话历史但不破坏缓存——这要求 systemPrompt、userContext、systemContext、toolUseContext、forkContextMessages 完全相同。哪怕系统提示词只多了一个空格，缓存就失效，每次 fork 都是"冷启动"（详见第 12 章）。

原则 **16.4**：对抗优于自评 — 用独立的 Verification Agent 对抗模型的自我评估偏差。验证者的工作不是确认，是破坏。

模型的 6 种"合理化借口"（"代码看起来对"、"大概没问题"、"太耗时了"）是系统性的，不是随机的。Verification Agent 的系统提示词用 "RECOGNIZE YOUR OWN RATIONALIZATIONS" 对抗这些借口——这不是改进模型的评估能力，而是用一个独立模型对抗另一个模型的偷懒倾向（详见第 13 章）。

原则 **16.5**：并行按隔离分层 — in-process → tmux → remote 三层，每层的隔离和代价递增。不是所有并行都需要最大隔离。

三种 BackendType 编码了三种不同的信任假设：进程内共享状态（信任度高、延迟低）、tmux/iTerm2 进程隔离（中等信任、I/O 延迟）、远程云环境（零信任、网络延迟）。选择后端时按任务的安全需求匹配，不是一律用最安全的（详见第 14 章）。

原则 **16.6**：每个补偿措施都有拆除计划 — feature flag 确保 Harness 代码可以随模型进步而消失。补偿是临时的，不是永久的。

828 处 `feature()` 调用中的大部分在外部版本中返回 `false`——代码物理消除，不是注释掉。autoCompact 的 DCE 注释直接标注："Inside feature() so the string DCEs from external builds"。flag 的目标不是"控制功能开关"，而是"标记这段代码应该在未来消失"（详见第 15 章）。

原则 **16.7**：记忆只存不可推导的信息 — MEMORY.md 只存用户偏好、历史决策和反馈，不存代码架构。可推导的信息通过工具实时获取。

4 种记忆类型（user/feedback/project/reference）的选择标准是"不可从项目状态推导"。代码模式、架构、git 历史通过 grep/git 获得不应占用记忆配额。这个约束确保记忆系统的入口文件（200 行/25KB）不会填满可推导的信息（详见第 12 章）。

原则	源码锚点	关联章节	适用边界
16.1 能力硬编码	planAgent `disallowedTools`	第 13 章	需要不可违反的权限边界
16.2 补偿分层	queryLoop 压缩执行顺序	第 11 章	多种补偿策略并存时
16.3 缓存是约束	`CacheSafeParams` 5 参数	第 12 章	有 Prompt Cache 的 API
16.4 对抗优于自评	RATIONALIZATIONS 提示词	第 13 章	需要验证输出质量
16.5 并行按隔离	`BackendType` 三种类型	第 14 章	需要并行执行子任务
16.6 拆除计划	`feature()` DCE 注释	第 15 章	所有补偿性代码
16.7 只存不可推导	memoryTypes 4 种类型	第 12 章	有跨会话记忆的系统

适用场景——什么时候你需要一个 Harness

不是所有 LLM 应用都需要完整的 Harness。需要 Harness 的信号是"模型输出有不可接受的失败模式"。

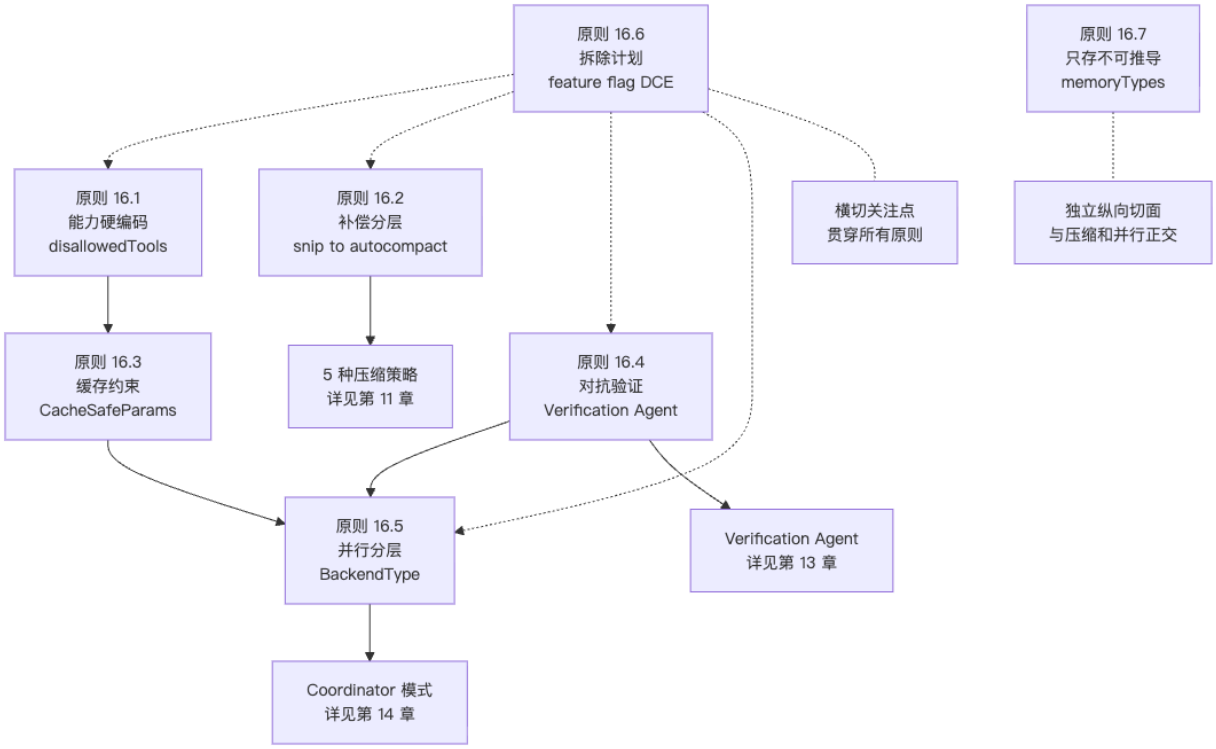
失败模式	是否需要 **Harness**	对应原则	参考
单轮问答不准确	可能不需要	—	模型可能已经够好
上下文窗口溢出	是	16.2 分层补偿	第 11 章
输出质量不可验证	是	16.4 对抗验证	第 13 章
需要同时执行多个子任务	是	16.5 并行分层	第 14 章
子任务有不同权限需求	是	16.1 能力硬编码	第 13 章
有 Prompt Cache 成本	是	16.3 缓存约束	第 12 章
跨会话需要记忆	是	16.7 只存不可推导	第 12 章

简单的单轮问答不需要 Harness。长会话 Agent 需要压缩策略。代码生成需要验证。多任务需要协调。当失败模式可以容忍时（如创意写作中的偶尔不完美），Harness 的成本可能不值得。

工作原理——7 条原则的协同关系

7 条原则不是独立的检查清单，它们之间存在依赖和协同。能力隔离（原则 16.1）是 fork 机制的前提，fork 是多 Agent 并行（原则 16.5）的基础，feature flag（原则 16.6）是所有原则的退出机制。

图 **16-1**：**7** 条原则的依赖关系



依赖链条是这样的：

能力隔离 → **fork** 安全 → 多 **Agent** 并行：planAgent 和 verificationAgent 的 `disallowedTools` 确保子智能体不会越权（原则 16.1），`CacheSafeParams` 确保隔离历史时不破坏缓存（原则 16.3），`runForkedAgent` 在安全隔离的基础上实现并行（原则 16.5）。没有能力隔离，fork 出的子智能体可能修改不应该修改的文件；没有缓存约束，每次 fork 都是冷启动。

对抗验证作为独立切面：Verification Agent 不依赖压缩策略或并行机制——它是验证阶段的独立关注点。但它的 `disallowedTools`（不能修改文件、不能生成子 Agent）仍然遵循原则 16.1。

feature flag 作为横切关注点：原则 16.6（拆除计划）不参与具体的执行路径，但贯穿所有层级——从压缩策略（`REACTIVE_COMPACT`）到验证（`VERIFICATION_AGENT`）到并行（`COORDINATOR_MODE`），每个补偿措施都有对应的 feature flag。

记忆系统作为独立纵向切面：原则 16.7（只存不可推导信息）与压缩和并行正交——它解决的是跨会话的信息持久化，不是当前会话的执行效率。

权衡——7 条原则的适用边界

7 条原则各有适用边界——超出边界时遵循原则反而会增加复杂度。

原则	适用边界	过度应用的代价
16.1 能力硬编码	需要不可违反的权限边界	所有工具都禁用 = Agent 无法完成任务
16.2 补偿分层	多种策略并存的场景	策略太多 = 调试困难、行为不可预测
16.3 缓存约束	有 Prompt Cache 的 API	无缓存时过度维护 CacheSafeParams 是浪费
16.4 对抗验证	输出质量有高风险的场景	简单任务用 Verification = token 成本不值得
16.5 并行分层	子任务真正独立的场景	有依赖关系的任务强行并行 = 结果冲突
16.6 拆除计划	所有补偿性代码	简单功能加 feature flag = 增加维护负担
16.7 只存不可推导	有明确"可推导 vs 不可推导"边界的场景	边界模糊时 = 自动提取质量差

原则 16.2（补偿分层）在短会话中完全不触发——5 种压缩策略的阈值都不满足。为一个 5 轮对话的系统设计 5 种压缩策略是过度设计。

原则 16.4（对抗验证）默认关闭——Verification Agent 需要双重 Feature Flag 启用（详见第 13 章）。这不是缺陷，而是对成本的诚实评估：验证的 token 成本在简单任务中不值得。

原则 16.5（并行分层）的 in-process 后端需要处理共享状态的并发访问——如果任务之间有文件依赖，强行并行会产生冲突。Coordinator 提示词明确要求"Write-heavy tasks — one at a time per set of files"（详见第 14 章）。

踩坑指南——设计 Harness 时的 3 个常见误区

误区一：为模型还不需要补偿的地方建补偿

模型的能力在快速进步。在构建 Harness 之前，先用当前最好的模型测试——如果模型在某个场景中已经足够好，就不需要补偿。`DEFAULT_FEATURES` 只有 3 个条目，说明大部分补偿是实验性的，默认关闭（详见第 15 章）。

❌ 错误做法：预判"模型可能在这里出错"，提前构建复杂的补偿机制。
✓ 正确做法：先让模型尝试，收集失败数据，只为有证据的失败模式构建补偿。

误区二：把 **Harness** 功能做成永久架构

没有 feature flag 的补偿会变成技术债务——当模型进步后，这些代码仍然存在，增加维护负担和产物大小。autoCompact 的 DCE 注释明确标注了代码应该从外部版本中消除（详见第 15 章）。

❌ 错误做法：将补偿逻辑硬编码为核心路径的一部分，没有退出机制。
✓ 正确做法：每个补偿措施都受 feature flag 控制。定期审查 flag——模型在进步，Harness 应该在缩小。

误区三：忽视 **Harness** 自身的 **token** 成本

Harness 的每个子系统都有 token 消耗。Explore 智能体每周被调用 3400 万次，`omitClaudeMd` 通过省略不必要的 CLAUDE.md 内容每周节省 5-15 Gtoken（详见第 13 章）。如果忽视 Harness 的 token 成本，补偿措施本身可能比它解决的问题更昂贵。

❌ 错误做法：为子智能体加载完整的上下文（包括不需要的 CLAUDE.md 层级）。
✓ 正确做法：只读智能体使用 `omitClaudeMd` 省略不相关的上下文。计算 Harness 补偿的 token 成本，与它解决的问题的代价比较。

实证——用 7 条原则审视一个新项目

用 7 条原则检查一个假设的 LLM 应用——"代码审查 Agent"，展示原则如何指导具体决策。

场景：设计一个自动代码审查 Agent，读取 PR diff 并输出审查意见。

原则 **16.1** 检查（能力硬编码）：审查 Agent 只能读代码和输出意见，不能修改代码。用 `disallowedTools` 移除 FileEdit 和 FileWrite——硬性保证审查者不会"顺手"修改代码（参考 planAgent 的设计，详见第 13 章）。

原则 **16.2** 检查（补偿分层）：大型 PR 的 diff 可能很长。先尝试截断 diff（轻量），不够时压缩历史对话（中等），最后全量摘要 diff（重量）。不需要 5 种策略——3 种可能就够。

原则 **16.3** 检查（缓存约束）：如果多个文件需要独立审查，可以用 fork 并行。fork 时确保 CacheSafeParams 完全相同——系统提示词、工具配置、模型选择逐字匹配（详见第 12 章）。

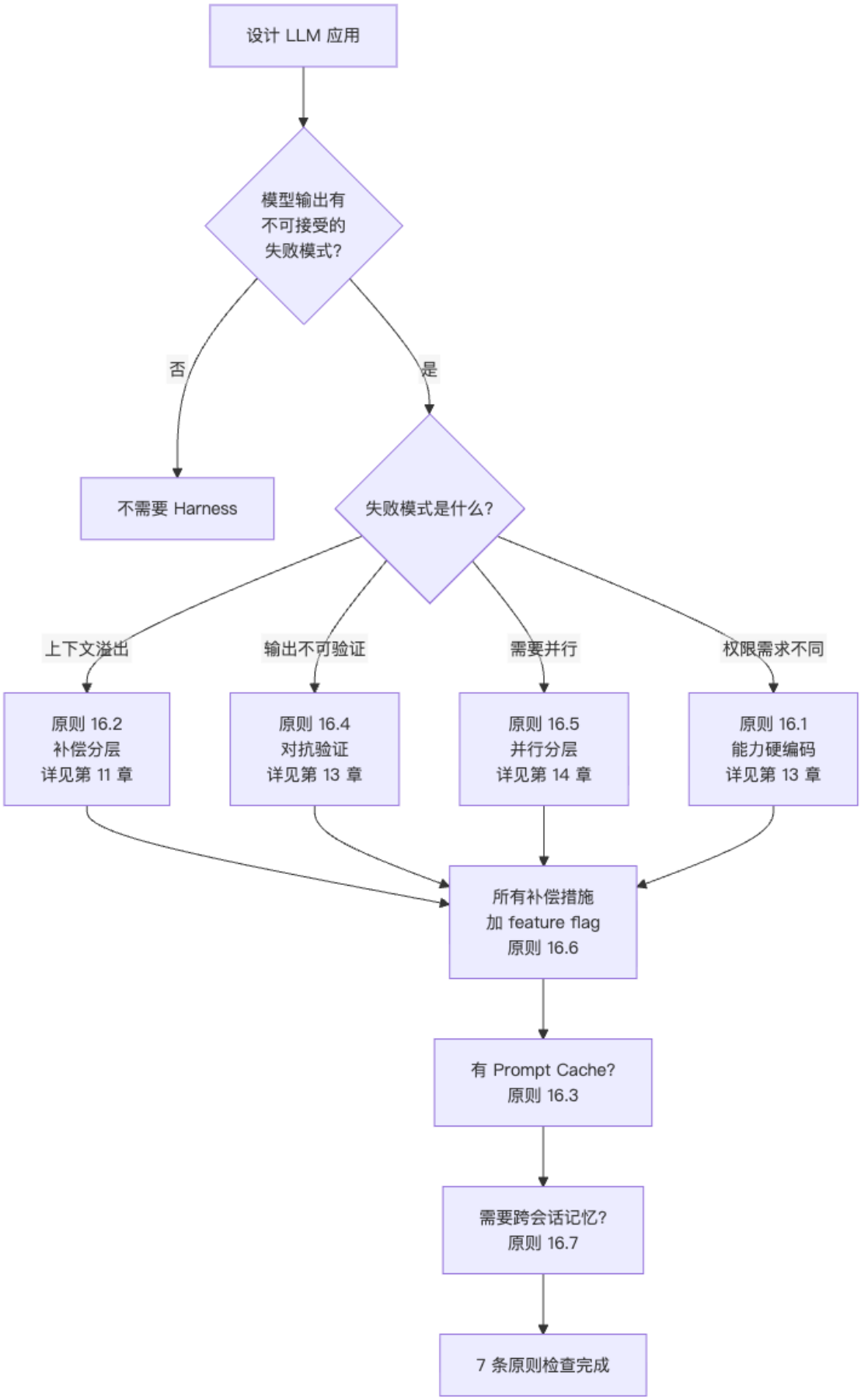
原则 **16.4** 检查（对抗验证）：审查质量需要独立验证。用一个独立的 Agent 检查审查意见是否有遗漏——它的工作不是确认"审查不错"，而是找审查漏掉的问题（参考 Verification Agent 的对抗性设计，详见第 13 章）。

原则 **16.5** 检查（并行分层）：多文件审查可以并行——但只读审查用 in-process 就够了（不需要进程隔离）。如果审查涉及运行测试（写操作），才需要进程外或远程后端（详见第 14 章）。

原则 **16.6** 检查（拆除计划）：所有补偿措施受 feature flag 控制。当模型的代码审查能力足够好时，Verification Agent 可以关闭。当模型能处理任意长度的 diff 时，截断策略可以移除。

原则 **16.7** 检查（只存不可推导）：记忆用户的审查偏好（"关注性能而非风格"），不存代码模式（代码本身就在仓库里）。

图 **16-2**：**Harness** 适用性决策树


设计 LLM 应用
模型输出有
不可接受的
失败模式?
否
是
不需要 Harness
失败模式是什么?
上下文溢出
输出不可验证
需要并行
权限需求不同
原则 16.2
补偿分层
详见第 11 章
原则 16.4
对抗验证
详见第 13 章
原则 16.5
并行分层
详见第 14 章
原则 16.1
能力硬编码
详见第 13 章
所有补偿措施
加 feature flag
原则 16.6
有 Prompt Cache?
原则 16.3
需要跨会话记忆?
原则 16.7
7 条原则检查完成

这个决策树不是一次性的——它是迭代使用的。随着模型进步，某些失败模式可能消失，对应的 Harness 组件应该被移除。定期回到这个决策树，重新评估每条原则是否仍然适用。

本章主成分：从 Claude Code 到通用原则

本质：Harness 不是产品的永久部分，是产品在"模型还不够好"阶段的临时支架。7 条原则是设计支架的工程方法，不是产品设计方法。当模型能力超越 Harness 补偿的阈值时，对应的 Harness 代码应该消失。

关键机制：

- 能力隔离（`disallowedTools`，硬约束）
- 分层补偿（snip → micro → collapse → auto，轻→重）
- 缓存一致性（`CacheSafeParams`，5 参数逐字相同）
- 对抗性验证（Verification Agent，独立破坏者）
- 并行分层（in-process → tmux → remote，按需隔离）
- 拆除计划（`feature()` DCE，代码物理消失）
- 记忆约束（4 种类型，只存不可推导信息）

全书结论：Harness 工程的最高目标不是建更好的 Harness，而是让 Harness 变得不需要。89 个 feature flag 中的每一个都在等待这一天。

你能做什么

- 列出你的 **LLM** 应用中**"模型还不够好"**的具体场景。每个场景都是一条 Harness 候选——但只有不可接受的失败模式才值得补偿。
- 为每条 **Harness** 设计 **feature flag**。确保将来可以安全移除——当模型进步时，你的 Harness 应该在缩小。
- 用 **`disallowedTools`**（或等价机制）硬性隔离能力。不要只靠提示词——提示词可以违反，工具不存在不可违反。
- 为补偿措施设计分层。先轻量后重量，先尝试微补偿再考虑全量重做——不是所有问题都需要最重的解决方案。
- 如果需要并行执行，按隔离需求选择后端。进程内（最快）→ 进程外（隔离好）→ 远程（完全隔离）——不要对所有任务使用同一种后端。
- 为每个子 **Agent** 提供自包含的提示词。它看不到你的对话——所有上下文必须写进提示词。
- 定期审查 **feature flag**。模型在进步，你的 Harness 应该在缩小。如果一个 flag 已经存在一年没有更新，要么它补偿的问题已经解决，要么你需要重新评估。

全书结语：16 章，从一次 API 调用的生命周期到 7 条通用原则。我们看到了 Harness 如何在查询循环中编排模型行为，如何用工具系统约束模型能力，如何用压缩策略管理

有限上下文，如何用对抗性验证弥补自我评估偏差，如何用并行机制突破单线程限制，以及如何用 feature flag 确保这一切都是临时的。Claude Code 的源码是一本工程教科书——它教的不只是"怎么用 Claude"，而是"怎么设计一个让 AI 安全、高效、可控地工作的系统"。而这 7 条原则，就是这本教科书的目录。

附录 A：术语表

本书使用的核心术语，按主题分组。每个术语给出在 Claude Code 语境下的精确定义。

架构与系统

Harness：围绕 LLM/Agent 搭建的运行框架，涵盖提示词管理、工具编排、上下文压缩、多智能体协调、权限护栏等。Claude Code 是目前公开可见的最完整的 Harness 实现。

QueryEngine：Claude Code 的主循环引擎。管理用户输入 → API 调用 → 工具执行 → 响应输出的循环，控制循环的继续、压缩和终止条件。

REPL（Read-Eval-Print Loop）：交互模式——读取用户输入、模型处理、输出响应、等待下一输入。Claude Code 的 REPL 基于 Ink（React for CLI）渲染。

Feature Flag：编码"模型能力假设"的编译时/运行时开关。Claude Code 有 89 个 Feature Flag、828 处调用。

DCE（Dead Code Elimination，死代码消除）：Bun 编译时将 `feature()=false` 的代码从产物中物理消除。不是注释掉，不是跳过——代码不存在。

excluded-strings.txt：Bun 的 DCE 配置文件，列出只存在于内部构建的 Feature Flag 名称。flag 名称字符串本身也从外部版本中消除。

上下文管理

autoCompact：上下文接近上限时触发的全量压缩策略。用一次 API 调用生成完整对话摘要，替换整个历史。缓冲区 13000 token，失败 3 次触发熔断。

microCompact：清除旧工具结果的轻量压缩策略。只处理 COMPACTABLE_TOOLS 白名单中的 8 类工具（FileRead/Shell/Grep/Glob/WebSearch/WebFetch/FileEdit/FileWrite）。

snipCompact：在历史轮次边界处裁剪消息的压缩策略。删除边界之前的消息块，计算释放的 token 数。

contextCollapse：投影式上下文折叠。不修改原始消息数组，而是提供一个"折叠视图"。折叠状态存储在独立的 collapse store 中，跨轮次持久化。

timeBasedMC（Time-Based MicroCompact）：基于 cache TTL 过期的压缩策略。阈值 60 分钟，默认关闭。适合长会话场景。

Prompt Cache : Anthropic API 的缓存机制。相同前缀的消息可以复用之前的计算结果，减少 token 消耗和延迟。缓存 key 由系统提示词、工具、模型、消息前缀和思考配置组成。

CacheSafeParams : fork 时必须与父智能体完全相同的 5 个参数（systemPrompt/userContext/systemContext/toolUseContext/forkContextMessages）。任何一字不同都会导致 Prompt Cache 失效。

Agent 与并行

forkedAgent : 在主循环之外启动的独立 Agent 会话。隔离对话历史但共享 Prompt Cache。通过 `runForkedAgent` 执行，复用 `lastCacheSafeParams` 全局单例。

AgentDefinition : 智能体的定义结构，包含 agentType、disallowedTools、model、系统提示词等。三种来源：BuiltIn（内置）、Custom（Markdown frontmatter）、Plugin（插件 API），共享 BaseAgentDefinition 接口。

disallowedTools : 智能体的硬性工具禁用列表。被禁用的工具不存在于智能体的工具池中——不是"建议不用"，是"不存在"。planAgent 和 verificationAgent 共享 5 种禁用工具。

Verification Agent : 对抗性验证智能体。目标不是确认代码能运行，而是"拼命让代码崩溃"。系统提示词明确列出模型的 6 种"合理化借口"并逐一对抗。异步执行（background: true）。

planAgent : 只读规划智能体。通过 disallowedTools 硬性保证不能修改文件。系统提示词强调 "CRITICAL: READ-ONLY MODE"。

exploreAgent : 快速搜索智能体。使用 haiku 模型（外部用户）或 inherit（内部用户），每周被调用超过 3400 万次。

Coordinator Mode : 多智能体协调模式。协调器分配任务，worker 独立执行并汇报结果。需要 Feature Flag + 环境变量双重启用。

BackendType : 队友执行后端类型。三种：in-process（同进程）、tmux（独立终端）、iterm2（iTerm2 原生分屏）。控制进程隔离级别。

teammateMailbox : 基于文件系统的 Agent 间消息队列。源码注释定义："File-based messaging system for agent swarms"。跨进程可用，I/O 延迟。

SendMessage : 队友间实时通信工具。通过 `to: "<name>"` 发送给指定队友，`to: "*"` 广播。

提示词与记忆

System Prompt（系统提示词）：分层构建的完整系统提示词。通过 DYNAMIC_BOUNDARY 分为静态区（缓存友好）和动态区（每次重新生成）。

DYNAMIC_BOUNDARY：系统提示词中静态区与动态区的分界标记。静态区包含角色定义和工具说明（缓存命中），动态区包含当前工作目录、文件列表等（每次不同）。

CLAUDE.md：四层记忆体系：Managed（全局管理）→ User（用户全局）→ Project（项目级）→ Local（本地私有）。后加载的优先级更高。

MEMORY.md：记忆系统的入口文件。限制 200 行 / 25000 字节。通过 `@include` 指向详细记忆文件。

memoryTypes：4 种记忆类型：user（用户偏好）、feedback（用户反馈）、project（项目知识）、reference（参考资料）。选择标准：只存"不可从项目状态推导的信息"。

omitClaudeMd：子智能体省略 CLAUDE.md 加载的配置。Explore 智能体每周 3400 万次调用，省略不必要的 CLAUDE.md 每周节省 5-15 Gtoken。

自动记忆提取（initExtractMemories）：从对话中自动识别值得记忆的内容并写入记忆文件。每次会话独立初始化（闭包隔离），异步执行。

安全与权限

Permission Mode（权限模式）：权限谱系模式。5 个外部模式（default/acceptEdits/plan/dontAsk/bypassPermissions）+ 2 个内部模式（auto/bubble）。默认值是 default（最严格）。

auto 模式：AI 分类器自动判断工具调用是否安全的内部模式。通过 `USER_TYPE === 'ant'` 判断和 Feature Flag 控制，仅内部用户可用。

4 层纵深防御：auto 模式下的权限判断链：规则分类器 → 危险模式库 → AI 分类器 → 否决追踪。每层独立判断，互为兜底。

DENIAL_LIMITS：否决追踪上限：maxConsecutive=3（连续否决不超过 3 次）、maxTotal=20（总计否决不超过 20 次）。超限后 shouldFallbackToPrompting=true，强制回退人工确认。

CROSS_PLATFORM_CODE_EXEC：危险模式库。硬编码的代码执行入口点列表（python/node/bash/sh 等），在 AI 分类器之前提供不可绕过的硬拒绝。

Hook：在 Harness 生命周期特定时机点插入自定义逻辑的事件系统。27 种事件类型（HOOK_EVENTS），4 种执行方式（command/prompt/agent/http）。

Stop Hook : 查询循环结束时触发的事件。通常配置为 agent Hook，用 AI 验证任务是否真正完成。

HOOK_EVENTS : 27 个预定义触发点的完整列表。覆盖工具调用（Pre/PostToolUse）、会话边界（SessionStart/End）、权限（PermissionRequest）、压缩（Pre/PostCompact）等。

工具系统

buildTool : 工具的工厂函数。统一接口定义 isEnabled（是否可用）、isAllowed（是否被允许）、execute（执行逻辑）。所有工具共享相同的注册和调度路径。

StreamingToolExecutor : 并发工具执行器。支持工具的流式输出（yield），只读工具可并发执行，写操作工具串行执行。

SkillTool : 技能工具。延迟加载技能定义，将 100+ 技能文件缩减为 1% 上下文预算。模型看到技能摘要，需要时才加载全文。

COMPACTABLE_TOOLS : microCompact 可清除的工具白名单。8 类：FileRead、Shell、Grep、Glob、WebSearch、WebFetch、FileEdit、FileWrite。白名单外的工具结果不会被清除。

压缩常量

AUTOCOMPACT_BUFFER_TOKENS : autoCompact 的摘要生成缓冲区，13000 token。编码假设"模型需要至少 13000 token 才能完成高质量摘要"。

POST_COMPACT_TOKEN_BUDGET : 压缩后资源恢复预算，50000 token。压缩后重新读取最近 5 个文件，重建模型的工作记忆。

MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES : autoCompact 连续失败熔断阈值，3 次。源码注释记录真实生产数据：曾有 1279 个会话 50+ 次连续失败，每天浪费约 25 万次 API 调用。

> 参见正文各章节获取每个术语的完整分析和源码锚点。